Metaprogramming in .NET 10

Runtime Introspection, Reflection, and Dynamic Code Generation

Complete Study Guide

About this guide

A 37-minute technical session, written up in full. It assumes working C# knowledge and no prior exposure to metaprogramming.

Target framework	**.NET 10 (`net10.0`)**
Verified on	.NET SDK 10.0.400, runtime 10.0.11, Windows 11 x64
Runnable code	12 runnable examples (14 projects) in `Code-Examples/` — all built and executed
Session length	37 minutes (6 + 10 + 12 + 9)

How to read this

Every console output quoted in this guide is **real**. It was captured by running the corresponding project, not written from expectation. The captured transcripts are in `Code-Examples/expected-output/`.

Where this guide contradicts an older tutorial — particularly about `Reflection.Emit` — the difference is almost always that the tutorial describes .NET Framework APIs that no longer exist. Those cases are flagged explicitly and were checked by running code on .NET 10.

Contents

1. **Part 1 — Understanding Metaprogramming** *(6 min)* normal vs metaprogramming · why it exists · developer benefits · compile-time vs runtime · implicit vs explicit metadata
2. **Part 2 — Runtime Introspection** *(10 min)* .NET metadata · the Reflection API · assembly and type discovery · `BindingFlags` · `Activator` · attributes · plugin discovery
3. **Part 3 — Runtime Code Generation** *(12 min)* CIL and the evaluation stack · IL instructions · `DynamicMethod` · `AssemblyBuilder`/`TypeBuilder` · persistence on .NET 10 · invalid IL
4. **Part 4 — Real-World Applications** *(9 min)* ASP.NET Core MVC · dependency injection · EF Core 10 · serialization · plugins · dynamic proxies · cross-cutting concerns · expression trees
5. **Quick Revision Notes** — for the 15 minutes before presenting
6. **Interview Questions** — 22 questions with accurate answers
7. **Glossary** — every term, with common misuses called out
8. **References** — official Microsoft documentation, all links verified

The one idea

> *A .NET assembly is **self-describing**. Metaprogramming reads that description (Part 2) or writes new code from it (Part 3). Every framework in Part 4 does it **once** at startup, **caches** the result, and then runs **compiled code**.*

Part 1 — Understanding Metaprogramming

Session time: 6 minutes (slides 3–7) Runnable example: `Code-Examples/01-Basics`

Terminology first

Three words are used throughout this guide. Learn them here and the rest reads easily.

Term	Meaning
Metadata	Data *about* your program — the names, types and signatures of everything you declared. The C# compiler writes it into your `.dll` alongside the executable code.
Introspection	*Reading* metadata while the program runs ("what properties does this type have?").
Code generation	*Creating* new executable code while the program runs.

"Meta-" means "about". **Metaprogramming is programming about programs.** Reading metadata and generating code are its two halves, and they map onto Parts 2 and 3 of this session.

1. Normal Programming vs Metaprogramming

1.1 Definition

Normal programming writes code that operates on **data** — numbers, strings, records, rows. The *structure* of the program is fixed when you compile it, and knowledge of that structure lives in the developer's head.

Metaprogramming writes code that operates on **the program itself** — its types, members, signatures and behaviour. The program holds knowledge about its own structure and can act on it.

1.2 In simple English

In normal programming you tell the computer *"take this customer's email and send a message to it."*

In metaprogramming you tell the computer *"look at whatever object I just handed you, find out what properties it has, and print each one."*

The second instruction does not mention customers or emails. It works on objects whose types you had never seen when you wrote it — including types written next year.

1.3 Why the concept exists

Because **library authors cannot know their users' types.**

The author of a JSON serializer, a DI container, a unit-test runner or an ORM is writing code in 2019 that must work with a class you will write in 2027. There are only two options:

1. Make every user hand-write adapter code for every one of their types; or
2. Write code that *asks each type about itself* at run time.

Option 2 is metaprogramming, and it is why `JsonSerializer.Serialize(anything)` works.

1.4 How it works internally

When `csc` compiles your C#, it produces a `.dll` containing **two** things:

```
MyApp.dll
■■■ IL (Intermediate Language)  ← the executable instructions
■■■ METADATA TABLES             ← the description of everything you declared
    ■■■ TypeDef      : every type you declared
    ■■■ MethodDef    : every method, with its signature
    ■■■ Field        : every field
    ■■■ Property     : every property
    ■■■ Param        : every parameter, with its name
    ■■■ CustomAttribute : every attribute you applied
    ■■■ (and about 40 more)
```

That metadata is not debug information and it is not optional. It is a required part of every managed assembly — the CLR needs it to do its own job (resolving calls, verifying types, garbage-collecting correctly). Reflection simply **reads the tables that are already there.**

This is the single most important mental model in the whole session:

> ***A .NET assembly is self-describing. Metaprogramming is reading the description.***

1.5 Important APIs

API	Purpose
`object.GetType()`	The runtime type of an actual object

API	Purpose
`typeof(T)`	The `Type` for a compile-time-known type
`Type.GetProperties()`	The properties a type declares
`PropertyInfo.GetValue(obj)`	Read a property's value from an instance
`Type.GetType("Name")`	Find a type from a string, at run time

1.6 Syntax

```
// Normal: the property is named in the source.
string email = customer.Email;

// Metaprogramming: the property is discovered at run time.
PropertyInfo property = customer.GetType().GetProperty("Email")!;
object? email2 = property.GetValue(customer);
```

1.7 Complete example

From `01-Basics/Program.cs`. The hand-written version first:

```
internal static class HandWritten
{
    public static IEnumerable<string> Describe(Customer c)
    {
        yield return "Customer.Id = " + c.Id;
        yield return "Customer.FullName = " + c.FullName;
        yield return "Customer.Email = " + c.Email;
        yield return "Customer.IsActive = " + c.IsActive;
    }

    public static IEnumerable<string> Describe(Product p)
    {
        yield return "Product.Sku = " + p.Sku;
        yield return "Product.Name = " + p.Name;
        yield return "Product.Price = " + p.Price;
    }
}
```

And the metaprogramming version that replaces **all** of it:

```
internal static class Reflective
{
    public static IEnumerable<string> Describe(object instance)
    {
        ArgumentNullException.ThrowIfNull(instance);

        Type type = instance.GetType();

        yield return "-- " + type.Name + " (" + type.GetProperties().Length + " public properties)";

        foreach (PropertyInfo property in type.GetProperties(BindingFlags.Public | BindingFlags.Instance))
        {
            if (!property.CanRead)
            {
                continue;
            }

            object? value = property.GetValue(instance);
            yield return type.Name + "." + property.Name + " = " + value;
        }
    }
}
```

1.8 Line-by-line

Line	What happens			
`ArgumentNullException.ThrowIfNull(instance)`	Guard. Reflection on `null` gives a `NullReferenceException` deep inside the method otherwise.			
`Type type = instance.GetType()`	Gets the **runtime** type. If the variable is declared `object` but holds a `SavingsAccount`, this returns `SavingsAccount`. This is the difference between `GetType()` and `typeof(T)`.			
`type.GetProperties()`	Reads the `Property` metadata table for this type and returns one `PropertyInfo` per property.			
`BindingFlags.Public \	BindingFlags.Instance`	Restricts the result to public, non-static properties. Omitting the flags means `Public \	Inst anc e \	St ati c`.
`if (!property.CanRead)`	A write-only property has no getter; calling `GetValue` on it throws.			
`property.GetValue(instance)`	Invokes the property's `get` accessor through the runtime and boxes the result into `object`.			

1.9 Expected output

Verified by running the example (`Code-Examples/expected-output/01-Basics.txt`):

```
One reflection-based describer, used for three unrelated types:
  -- Customer (4 public properties)
  Customer.Id = 7
  Customer.FullName = Asha Menon
  Customer.Email = asha@example.com
  Customer.IsActive = True

  -- Product (3 public properties)
  Product.Sku = 101
  Product.Name = Mechanical Keyboard
  Product.Price = 4999.00

  -- Order (3 public properties)
  Order.OrderId = 5001
  Order.PlacedOn = 11/10/2026
  Order.LineCount = 2
```

One method, three unrelated types, and the method's source mentions none of them.

1.10 Real-world applications

Every one of these is `GetProperties()` plus a loop, at heart:

- `System.Text.Json` — walks your type's properties to write JSON.
- EF Core — walks them to build a table schema.
- AutoMapper — matches source properties to destination properties by name.
- `[Required]` validation in ASP.NET Core — walks properties looking for attributes.
- xUnit / NUnit — walks assemblies looking for `[Fact]` / `[Test]` methods.
- `ToString()` on a `record` — the compiler generates it, but conceptually the same job.

1.11 Advantages

- **One implementation serves unlimited types**, including types that do not exist yet.
- **Less code**, therefore fewer places for a bug to hide.

- **No drift.** A hand-written describer silently goes stale when a property is added; the reflective one picks it up automatically.
- **Enables whole categories of library** that are otherwise impossible.

1.12 Limitations

- **No compile-time safety.** `GetProperty("Emial")` compiles perfectly and returns `null` at run time.
- **Slower.** Measured in Part 3: a reflective property read is roughly 7–8× a direct one on the machine that produced this package.
- **Opaque.** "Go to definition" leads nowhere useful; stack traces go through `Invoke`.
- **Hostile to trimming and Native AOT.** The trimmer cannot see reflective usage, so it may remove members your reflection needs. Covered in Part 3.
- **Can leak internals.** `BindingFlags.NonPublic` reads private state, so reflective code can couple itself to things that were never part of the public contract.

1.13 Performance and security

Performance. The cost is *per call*, not per program. The fix is always the same: do the reflection **once**, cache the result, and execute cached/compiled code thereafter. Every framework in Part 4 does exactly this.

Security. `private` is a **compiler** rule, not a runtime permission. `BindingFlags.NonPublic` reads private fields, and it is allowed. Accessibility is **encapsulation**, not a **security boundary**. Never rely on `private` to protect a secret from code running in the same process.

1.14 Common mistakes

1. **Using reflection when generics would do.** If the type is known at compile time, `T` is faster, safer and clearer.
2. **Calling `GetProperties()` inside a loop.** It allocates an array every time. Hoist it out and cache it.
3. **Reflecting to avoid typing.** Reflection is for when the types are *unknown*, not when they are merely *tedious*.
4. **Forgetting `BindingFlags`** and then being surprised that a private or static member is missing.
5. **Assuming `GetProperties()` returns a stable order.** It does not guarantee one; sort explicitly if order matters.

1.15 Audience questions

Q: Is this the same as `dynamic`? No. `dynamic` defers *binding* of an expression to run time but you still write the member name in source (`obj.Email`). Reflection lets the *name itself* be data (a string, a config value, a database row). Also, `dynamic` relies on the much heavier DLR; reflection is the lower-level primitive.

Q: Is metaprogramming the same thing as reflection? Reflection is one *kind* of metaprogramming — the read-only, run-time kind. Part 3 covers run-time code *generation*, and source generators do it at compile time. Reflection is the most common form, not the whole category.

Q: Should I use this in application code, or only in libraries? Mostly libraries and infrastructure. In application code the types are usually known, so generics and plain C# are better. The honest exceptions are plugin systems and genuinely data-driven features (a report builder whose columns come from config).

1.16 Documentation

- Reflection and attributes (C# guide) — https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes/
- Reflection in .NET — https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/reflection
- Managed code and the CLR — https://learn.microsoft.com/en-us/dotnet/standard/managed-code

1.17 Comparison table

	Normal programming	Metaprogramming
Operates on	values and data	types, members, behaviour
Structure known	when you write the code	only when the program runs
Checked by	the compiler	nothing — you find out at run time
Typical speed	full	slower per call, unless cached
Typical author	application developer	library / framework developer
Failure mode	build error	runtime exception, or silent wrong behaviour
Example	`customer.Email`	`type.GetProperty("Email")!.GetValue(c)`
Debuggability	step straight in	opaque frames through `Invoke`
AOT / trimming	always fine	needs care, sometimes impossible

Why metaprogramming is called a "higher-level" technique: not because it is more advanced in the sense of better, but because it sits one level *above* ordinary code — its input is the program. Normal code reasons about customers; metaprogramming code reasons about the *idea* of a customer type.

---

2. Why Do We Need Metaprogramming?

2.1 Definition

The need arises whenever the **set of types a piece of code must handle is not known when that code is written.**

2.2 The problem, demonstrated first

Say you must log every property of every domain object for an audit trail. There are 40 domain types.

Without metaprogramming you write 40 methods:

```
void Audit(Customer c) { /* 4 lines */ }
void Audit(Product p)  { /* 3 lines */ }
void Audit(Order o)    { /* 3 lines */ }
// ... 37 more
```

Count the costs:

Cost	Detail
Volume	40 methods, ~150 lines of pure mechanism
Drift	Add `Product.Weight` and the audit log is silently wrong. The compiler does **not** warn you.
Scaling	Every new type needs a new method. 41st type, 41st method.
Review burden	150 lines of near-identical code hide the one that is wrong
Impossible case	A type from a plugin DLL loaded at run time — you *cannot* write its method in advance

That last row is the decisive one. Repetition is annoying; **impossibility** is a hard block.

2.3 The metaprogramming solution

One method, from §1.7 above, handles all 40 types and every future one. The output is in §1.9.

2.4 When the problem is NOT this

Be fair about the alternatives. Repetition alone does not justify reflection:

Situation	Better tool than reflection
Types known at compile time, behaviour identical	**Generics** — `Audit<T>(T item)` with a constraint
Types known, behaviour differs slightly	**An interface** — `IAuditable.Describe()`
Boilerplate known at build time	**A source generator** — zero run-time cost
A fixed, small set of types	**Just write the 3 methods.** Honestly.

Reflection earns its place at the point where the type set is **open**.

2.5 Real-world applications

- A plugin host loading DLLs discovered on disk (example 04).
- A serializer handling your types and your users' types (example 09).
- A DI container constructing types it has never seen (example 07).
- A test runner finding tests in any assembly.
- A report engine whose columns come from a database table.

2.6 Common mistakes

Using reflection to work around a design problem. If you find yourself reflecting over your *own* types inside your *own* application, the usual root cause is a missing interface or a missing generic parameter. Fix the design instead.

2.7 Audience questions

Q: Could a source generator solve the 40-types problem instead? Yes, and better — *if* all 40 types are in your solution at build time. It would generate the 40 methods for you, with zero run-time cost and full trimming support. Source generators do not help with the plugin case, because that type does not exist at build time. See §4.

Q: We have 40 types but they are all ours. Which should we pick? Source generator or generics. Reflection is the answer only for the open-set case. Choosing reflection for a closed set means paying the costs in §1.12 for benefits you do not need.

Q: Isn't 150 lines of boring code fine? It is explicit and fast. It is a real position and sometimes the right one — explicit code is debuggable and the trimmer loves it. The decisive question is not volume but **drift**: can the code go silently wrong when someone adds a property? If yes, mechanism should be derived, not written by hand.

---

3. Developer Benefits

Four benefits, each stated on its own, with the counterweight.

3.1 Automation

Definition. Deriving mechanism from declarations instead of writing it.

Example. ASP.NET Core model validation. You declare:

```
public sealed class CreateReportRequest
{
    [Required]
    [StringLength(60, MinimumLength = 3)]
    public string Title { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Owner { get; set; } = string.Empty;
}
```

You write **no validation code at all**. Example 10 POSTs an invalid body and the framework answers:

```
POST /api/reports   body: {"title":"x","owner":"not-an-email"}
  -> 400 Bad Request
  -> {"title":"One or more validation errors occurred.","status":400,
      "errors":{"Owner":["The Owner field is not a valid e-mail address."],
                "Title":["The field Title must be a string with a minimum length of 3
                          and a maximum length of 60."]}}
```

Advantage. The rule is declared once, next to the thing it constrains, and cannot be forgotten at a call site.

Limitation. The rules are limited to what the attributes can express. Anything conditional ("required only when Status is Draft") escapes the model and needs real code.

3.2 Reducing boilerplate

Definition. Removing code whose content is fully implied by a declaration.

Example. `[JsonPropertyName]` and `[JsonIgnore]` (example 09) replace a hand-written DTO-mapping layer. One attribute keeps a field out of an API response:

```
[JsonIgnore]
public string? InternalNotes { get; set; }
```

Advantage. Less code, and the intent is visible on the member itself.

Limitation. Boilerplate is *explicit*. Replacing it moves the logic somewhere invisible. A new team member can read a hand-written mapper; they cannot read an attribute's effect without knowing the framework.

3.3 Maintainability

Definition. Change costs stay proportional to the change, not to the codebase.

Example. Example 03's validator:

```
foreach (PropertyInfo property in type.GetProperties(BindingFlags.Public | BindingFlags.Instance))
{
    var range = property.GetCustomAttribute<ValidateRangeAttribute>();
    if (range is not null && value is not null) { /* check it */ }
}
```

Add a 41st annotated class and the validator needs no edit. Output:

```
INVALID payment: Amount=-5 CurrencyCode="RUPEES" Reference=""
  ERROR: Amount must be between 0.01 and 1000000 (was -5)
  ERROR: CurrencyCode must be exactly 3 characters (was 6)
  ERROR: Reference is required
```

Advantage. One place to fix a bug in the mechanism; N places become one.

Limitation. When the generic mechanism *is* the bug, debugging is harder. And because nothing references the attribute explicitly, "find all usages" tells you nothing.

3.4 Extensibility

Definition. Third parties can add behaviour without modifying — or rebuilding — your code.

Example. Example 04's host:

```
Loaded PluginDiscovery.Plugins.Sales v1.0.0.0
  into load context: PluginALC:PluginDiscovery.Plugins.Sales
  accepted MonthlySalesPlugin as "Monthly Sales Summary" v2.1.0
  accepted TopCustomersPlugin as "Top Customers" v2.1.0
  skipped ExperimentalForecastPlugin - [ReportPlugin(Enabled = false)]
  skipped UnannotatedHelperPlugin - implements IReportPlugin but has no [ReportPlugin]
```

The host's source names no plugin class. Drop a DLL in a folder, get a feature.

Advantage. This is the only one of the four that is *qualitative*. The others make existing work cheaper; this one makes previously impossible things possible.

Limitation. You have just created a plugin API, with everything that implies: versioning, trust, failure isolation. And `AssemblyLoadContext` **is not a security sandbox** — see Part 2 §8.

3.5 When metaprogramming increases complexity instead of reducing it

Be honest about this during the talk; it buys credibility for everything else.

Reach for something simpler when:

1. **The type set is closed.** Generics or a source generator win outright.
2. **It is called once.** Generation and JIT cost more than the reflection saved.
3. **The abstraction has one implementation.** A reflective "engine" with exactly one caller is a hand-written method with extra steps.
4. **The team will not maintain it.** IL and visitor chains are write-mostly. A clever 50-line emitter that nobody dares touch is worse than 200 obvious lines.
5. **You are shipping Native AOT.** Much of Part 3 simply does not run there.
6. **The failure mode matters more than the convenience.** A compile error at 3pm beats a `NullReferenceException` at 3am.

A reasonable rule:

> ***Metaprogramming trades compile-time safety for run-time flexibility.*** *Make that trade when you genuinely need the flexibility — and not one step sooner.*

3.6 Audience questions

Q: Our team banned reflection for performance reasons. Reasonable? Partly. Banning *per-call* reflection on a hot path is sound. Banning it at *startup* is not — you would have to ban ASP.NET Core, EF Core and the DI container, all of which reflect at startup and then execute cached code. The useful rule is "no reflection per request", not "no reflection".

Q: How do I know whether I have gone too far? Two tests. Could a competent colleague change the behaviour safely without you explaining it? And does a typo in a *string* become a production incident rather than a build failure? Two yeses mean you have gone too far.

Q: Which of the four benefits matters most? Extensibility, because it is the only one you cannot get another way. Automation and reduced boilerplate are better served by source generators when the shape is known at build time.

4. Compile-Time vs Runtime Metaprogramming

4.1 Definition

Metaprogramming happens at one of three **stages**, and confusing them is the most common conceptual error in this subject.

Stage	When	Tool	Produces
Compile-time generation	during the build	**Roslyn source generators**	new **C# source**, compiled with yours
Runtime introspection	while running	**Reflection**	*information* about existing code
Runtime generation	while running	**Reflection.Emit**, expression trees	new **IL**, in memory

4.2 In simple English

- A **source generator** is a colleague who writes extra code for you just before the compiler runs. By the time the program starts, that code is ordinary compiled C#.
- **Reflection** is reading the table of contents of a finished book.
- **Reflection.Emit** is writing a new chapter while the book is being read.

4.3 Why three stages exist

Because the information needed arrives at different times.

- If the shape is known at **build time** → generate then. Zero run-time cost, full type checking, trimming-safe. Always prefer this.
- If the shape is known only at **run time** (a plugin, a config file, a user query) → build time cannot help you.

4.4 How each works internally

Source generators. A generator is an analyzer implementing `IIncrementalGenerator`. Roslyn runs it during compilation, passing a model of your syntax and symbols. It adds source files to the same compilation. The output is normal IL — the generator is not present at run time and costs nothing then.

Reflection. Reads the metadata tables described in §1.4. Nothing is created; the cost is lookups, allocations for the `*Info` objects, and per-call binding on `Invoke`.

Runtime generation. You build IL (or an expression tree, which is compiled to IL). The JIT compiles it to machine code on first call. Requires a JIT, which is why it fails under Native AOT.

4.5 Key differences

	Source generators	Reflection	Reflection.Emit
Runs at	build time	run time	run time
Output	C# source	information	IL
Run-time cost	**zero**	per call	once, then ~direct
Type-checked	yes, by the compiler	no	no (and not always verified)
Native AOT	**yes**	partly	**no**
Trimming	safe	needs annotation	unsafe
Debuggable	yes — real source	frames via `Invoke`	poorly
Handles types unknown at build time	**no**	yes	yes

	Source generators	Reflection	Reflection.Emit
Startup cost	none	first-use reflection	generation + JIT

4.6 The same job, both ways

Example 09 does this deliberately, and the outputs are **byte-for-byte identical**:

```
// RUNTIME: reflects over Invoice on first use, then caches the plan.
string a = JsonSerializer.Serialize(invoice, options);

// COMPILE-TIME: the plan was generated during the build.
string b = JsonSerializer.Serialize(invoice, InvoiceJsonContext.Default.Invoice);
```

The source-generated context is declared with no body at all:

```
[JsonSourceGenerationOptions(WriteIndented = true, PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase)]
[JsonSerializable(typeof(Invoice))]
internal partial class InvoiceJsonContext : JsonSerializerContext;
```

Verified output:

```
Byte-for-byte identical: True
```

Same goal, same result. **The only difference is when the metaprogramming ran.**

4.7 Choosing

```
Is the shape known at BUILD time?
■■■ YES → source generator            (zero runtime cost, AOT-safe)  ← prefer
■■■ NO  → is it a FUNCTION or a TYPE you need?
          ■■■ function → expression trees   (readable, type-checked as you build)
          ■■■ type     → Reflection.Emit / DispatchProxy
          ■■■ or just read metadata → Reflection
```

4.8 Real-world: .NET is moving run-time work to build time

Feature	Runtime version	Build-time version
JSON	reflection-based	`JsonSerializerContext` source generator
Regex	`RegexOptions.Compiled` (emits IL)	`[GeneratedRegex]` source generator
Logging	reflective formatting	`[LoggerMessage]` source generator
Config binding	reflective binding	configuration binder generator
DI	reflection + emitted engine	compile-time container experiments

The direction is consistent, and the motivation is Native AOT and startup time.

4.9 Common mistakes

1. **"Source generators replace reflection."** They replace it for *closed* type sets only. Plugins remain reflection's territory.
2. **Thinking a source generator runs at run time.** It does not exist at run time.
3. **Expecting `Reflection.Emit` to work under Native AOT.** It throws.
4. **Forgetting expression trees need `Compile()` — and `Compile()` needs a JIT.** Building and *reading* a tree works under AOT (which is why EF Core can translate queries there); compiling one does not.

4.10 Audience questions

Q: Are source generators strictly better? Better where they apply, and they apply to most boilerplate. They cannot handle types that do not exist at build time, cannot see run-time configuration, and they add build time and a learning curve.

Q: Do source generators slow my build? Yes, measurably but usually mildly. Incremental generators are designed to cache aggressively. Trading build time for run-time cost and AOT compatibility is normally a good trade.

Q: Where do T4 templates and IL weaving (PostSharp, Fody) fit? T4 is an older compile-time text generator, pre-Roslyn and not integrated with the compiler's semantic model. IL weavers run *after* compilation and rewrite the IL in your assembly — a fourth stage, "post-compile", which this session does not cover but which is how some AOP frameworks work.

4.11 Documentation

- Source generators overview — https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/source-generators-overview
- `System.Text.Json` source generation — https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/source-generation
- Native AOT limitations — https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/#limitations-of-native-aot-deployment

---

5. Implicit vs Explicit Metadata

5.1 A necessary caveat first

"Implicit" and "explicit" metadata are teaching labels, not CLR categories.

The CLR has no such distinction. It has metadata tables, one of which is `CustomAttribute`. Everything is simply *metadata*, read through the same API. The split used here is about **who put it there**:

- **Implicit** — the compiler emitted it because you declared a type.
- **Explicit** — you deliberately attached it with an attribute.

Say this out loud when presenting. It is a genuinely useful distinction *and* it is not a formal one, and conflating the two confuses people later when they read the ECMA-335 spec and find no "implicit metadata" table.

5.2 Implicit metadata

You wrote no metadata here, yet all of this is queryable:

```
internal sealed record Customer(int Id, string FullName, string Email, bool IsActive);
```

Verified output:

```
IMPLICIT metadata - emitted by the compiler purely because you declared the type:
  Id        : Int32    (CanRead=True, CanWrite=True)
  FullName  : String   (CanRead=True, CanWrite=True)
  Email     : String   (CanRead=True, CanWrite=True)
  IsActive  : Boolean  (CanRead=True, CanWrite=True)
```

Names, types, accessibility, arity, parameter names, base types, interfaces — all emitted because the compiler must.

5.3 Explicit metadata

```
internal sealed record Customer(
    int Id,
    [property: Required] [property: StringLength(100)] string FullName,
    [property: Required] [property: EmailAddress] [property: JsonPropertyName("email_address")] string Email,
    bool IsActive);
```

Verified output:

```
EXPLICIT metadata - attributes you deliberately attached:
  FullName  : RequiredAttribute, StringLengthAttribute
  Email     : RequiredAttribute, EmailAddressAttribute, JsonPropertyNameAttribute
```

5.4 Attributes as declarative metadata

An attribute does not *do* anything. It is **inert data** until something reads it.

```
[JsonIgnore]
public string? InternalNotes { get; set; }
```

`[JsonIgnore]` has no behaviour. `System.Text.Json` looks for it and chooses to skip the property. If nothing reads an attribute, it is a comment the compiler stores.

This is why attributes are called **declarative**: you declare *what is true*, and separate machinery decides *what to do*.

5.5 Metadata tables

The compiler stores an applied attribute as a row in the `CustomAttribute` table recording the attribute type and the constructor arguments. The attribute **object does not exist** in the assembly — only the recipe for building one.

Which explains a result that surprises people (example 03):

```
  ReferenceEquals(first, second) = False
```

Two calls to `GetCustomAttribute<TableAttribute>()` return **different objects**, because each call runs the constructor again.

And it is why `GetCustomAttributesData()` exists — it reads the row *without* constructing anything:

```
GetCustomAttributesData() returns the raw metadata rows:
  NullableContextAttribute(1)
  NullableAttribute(0)
  TableAttribute(invoice_lines, Schema = sales)
  TagAttribute(billing)
  TagAttribute(generated-nightly)
  TagAttribute(pii-free)
```

Note the first two rows: `NullableContextAttribute` and `NullableAttribute` are attributes **the compiler** applied, to record nullable reference annotations. A neat illustration that the implicit/explicit line is about authorship, not mechanism — these are compiler-authored *attributes*.

5.6 Important APIs

API	Purpose
`GetCustomAttribute<T>()`	One attribute, **constructed**. Throws `AmbiguousMatchException` if several.
`GetCustomAttributes<T>()`	All of them, constructed. Use when `AllowMultiple = true`.
`GetCustomAttributesData()`	The raw rows, **nothing constructed**
`IsDefined(type)`	Cheapest existence check — no construction

5.7 Advantages and limitations

Implicit: free, always present, impossible to forget, cannot express intent. **Explicit:** expresses intent precisely, but must be remembered, can drift from reality, and couples your domain type to a framework's attribute.

That last point drives a real design decision: EF Core offers data annotations **and** a fluent API precisely so you can keep persistence attributes out of a domain class. Part 4 §3 shows both.

5.8 Performance and security

Performance. `GetCustomAttributes` constructs objects on every call — do not put it on a hot path. Cache the result, or use `IsDefined` / `GetCustomAttributesData` when you only need to *know*, not to *have*.

Security. Reading attributes **runs their constructors**. For an assembly you do not trust, that is executing its code. Use `GetCustomAttributesData()`, or `MetadataLoadContext`, to inspect untrusted assemblies without running anything.

5.9 Common mistakes

1. `GetCustomAttribute<T>()` **(singular) on an** `AllowMultiple = true` **attribute** → `AmbiguousMatchException`. Example 03 triggers it on purpose.
2. **Forgetting** `inherit: true`**.** Both the attribute's `Inherited` setting *and* the call's `inherit` argument must agree. Example 03 shows all four combinations.
3. **Expecting attribute instances to be cached.** They are not.
4. **Putting logic in an attribute.** Attributes are data; the reader holds the behaviour.

5. **Omitting `[AttributeUsage]`.** The defaults (`All`, `AllowMultiple = false`, `Inherited = true`) are rarely what you meant.

5.10 Audience questions

Q: Do unused attributes cost anything at run time? Essentially nothing. They add a few bytes of metadata and are never constructed unless something asks. They are not free for *trimming*, though — an attribute referencing a type can keep that type alive.

Q: Can I add attributes at run time? Not to an existing type. Metadata of a loaded type is immutable. You can apply attributes to types you **generate** (`TypeBuilder.SetCustomAttribute`, Part 3). For "run-time configuration" the usual answers are a fluent API (as EF Core does) or a `TypeDescriptionProvider`.

Q: Attributes or a fluent API, for my own library? Offer both if you can. Attributes are discoverable and sit next to the member; a fluent API keeps framework types out of domain classes, supports conditional configuration, and can express things attributes cannot (attribute arguments must be compile-time constants). EF Core's choice to support both is the reference example.

5.11 Documentation

- Writing custom attributes — https://learn.microsoft.com/en-us/dotnet/standard/attributes/writing-custom-attributes
- `AttributeUsageAttribute` — https://learn.microsoft.com/en-us/dotnet/api/system.attributeusageattribute
- `CustomAttributeData` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.customattributedata
- Assemblies in .NET — https://learn.microsoft.com/en-us/dotnet/standard/assembly/

Part 1 summary

1. Normal code operates on data; metaprogramming operates on the program's own structure.
2. It exists because library authors cannot know their users' types.
3. It works because **every .NET assembly is self-describing** — metadata is required, not optional.
4. Benefits: automation, less boilerplate, maintainability, and — uniquely — extensibility.
5. Three stages, and keeping them apart matters: compile-time generation, runtime introspection, runtime generation.
6. Implicit vs explicit metadata is about **authorship**, not a CLR category.
7. The trade is always the same: **compile-time safety for run-time flexibility.**

Part 2 — Runtime Introspection

Session time: 10 minutes (slides 8–15) Runnable examples: `02-Reflection`, `03-Attributes`, `04-Plugin-Discovery`

1. What Is Runtime Introspection?

1.1 Definition

Runtime introspection is a program examining its own structure — assemblies, types and members — while it is executing, using the metadata stored in its assemblies.

In .NET this is done with the **Reflection API** in `System.Reflection`.

1.2 In simple English

The program asks questions about itself and gets answers:

- "What types are in this DLL?"
- "What methods does this type have?"
- "Is this method marked `[Obsolete]`?"
- "What arguments does this constructor need?"

It answers them by reading the description the compiler already wrote into the assembly. It is **not** reading your `.cs` files — source is not needed and is usually absent.

1.3 Runtime type information

The CLR keeps, for every loaded type, a runtime structure describing it. `System.Type` is your handle on that structure. Every object on the heap carries a pointer to its type's information, which is how `GetType()` can work on an `object` reference with no other context — and how `is`, `as` and virtual dispatch work.

Introspection is therefore not bolted on. It exposes bookkeeping the runtime must do anyway.

1.4 Use cases

Use case	Who does it
Serialization	`System.Text.Json`, XML, protobuf
ORM mapping	EF Core, Dapper
Dependency injection	`Microsoft.Extensions.DependencyInjection`
Test discovery	xUnit, NUnit, MSTest
Plugin loading	MEF, custom hosts, MSBuild, Roslyn analyzers
Validation	ASP.NET Core model validation, FluentValidation
Mapping	AutoMapper, Mapster
Debuggers and profilers	Visual Studio, `dotnet-dump`
Admin/diagnostic UIs	Swagger, health-check dashboards

1.5 Limitations

- **Read-only.** Introspection cannot change a loaded type. (Creating *new* types is Part 3.)
- **Needs the assembly loaded** (or a `MetadataLoadContext`).
- **Slow per call** relative to direct access.
- **Invisible to the trimmer**, so trimming can delete what you intend to reflect over.
- **No compile-time checking** — every error is a run-time error.

- **Cannot see inside method bodies.** Metadata describes *signatures*, not implementations. You can see that `Withdraw(decimal)` exists; you cannot ask "does it validate its argument?" The body is IL, and reading IL is a different job (`MethodBody.GetILAsByteArray()`, or a decompiler).

That last point is worth saying in the talk — people often assume reflection can read logic. It cannot.

1.6 Documentation

- Reflection in .NET — https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/reflection

2. Understanding .NET Metadata

2.1 Definition

Metadata is the structured, binary description of every type and member in an assembly, emitted by the compiler and stored in the assembly file itself.

2.2 The hierarchy

```
AppDomain / process
■■■ Assembly            MyApp.dll — the deployment and versioning unit
    ■■■ Manifest        name, version, culture, referenced assemblies
    ■■■ Module          usually exactly one, the PE file itself
        ■■■ Type        class, struct, interface, enum, delegate
            ■■■ Constructor   ConstructorInfo
            ■■■ Method        MethodInfo
            ■■■ Property      PropertyInfo  (name bound to accessor methods)
            ■■■ Field         FieldInfo
            ■■■ Event         EventInfo
            ■■■ Nested type   Type
```

Every level can carry **custom attributes**.

2.3 Verified, from example 02

```
Simple name      : ReflectionDemo
Version          : 1.0.0.0
FullName         : ReflectionDemo, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
Location on disk : ReflectionDemo.dll
IsDynamic        : False
EntryPoint       : Metaprogramming.ReflectionDemo.Program.Main
Modules          : ReflectionDemo.dll
Referenced       : 3 assemblies, e.g. System.Runtime, System.Linq, System.Console
```

Everything there was read from the assembly's own tables.

2.4 Each level explained

Assembly. The unit of deployment, versioning and identity. One or more modules plus a manifest. `Assembly.GetName()` gives the identity; `GetReferencedAssemblies()` gives the dependencies recorded at compile time.

Assembly manifest. The part of the metadata recording the assembly's own identity, its referenced assemblies and their expected versions, and its resources. This is what the loader uses to resolve dependencies.

Module. A physical container. Multi-module assemblies are legal but essentially unused in modern .NET, so assembly ≈ module in practice.

Type. `TypeDef` rows for types you declared, `TypeRef` rows for types you referenced. `Type` also represents constructed generics (`List<int>`), arrays and by-ref types, which is why `Type.Name` can be `List\1`.

Method. Name, signature, calling convention, accessibility, and the location of the IL body. `MethodInfo` exposes the signature; the body is separate.

Field. Name, type, accessibility. Note that **auto-properties produce a backing field** — reflection shows it, with a compiler-generated name:

```
FIELDS (FieldInfo) - including the compiler's backing fields:
  private  String     _auditTrail
  private  Decimal    <AnnualRate>k__BackingField
```

The `<...>k__BackingField` name is deliberately unspeakable in C# so you cannot collide with it.

Property. *Not* a storage location. A property is metadata binding a name to a getter and/or setter method. Those accessors are real methods, flagged `IsSpecialName`:

```
METHODS (MethodInfo) - DeclaredOnly to hide inherited object members:
  Decimal    get_AnnualRate()  (property accessor)
  Decimal    ProjectInterest(Int32 months)
  Decimal    AddInterest(Int32 months)
  String     Describe()
```

Constructor. A method named `.ctor` (instance) or `.cctor` (static), surfaced as `ConstructorInfo`.

Custom attributes. Rows in the `CustomAttribute` table — Part 1 §5.5.

2.5 How metadata is stored

A managed assembly is a **PE file** (the Windows executable format) with extra sections. The CLI header points at metadata streams:

Stream	Contents
`#~`	the compressed metadata tables (`TypeDef`, `MethodDef`, …)
`#Strings`	identifier names
`#US`	user strings (your string literals)
`#Blob`	signatures and attribute arguments
`#GUID`	GUIDs

Tables reference each other by index, which is what makes lookups fast. The format is specified in **ECMA-335**.

Two practical consequences:

1. **Metadata cannot be stripped.** It is load-bearing. Trimming removes *unused members*; it does not remove the description of what remains.
2. **Anyone with your DLL can read your API** — and decompile it. Reflection is not the privacy risk; the IL being right there is. Obfuscation renames things but the structure is still present.

2.6 Common mistakes

1. Thinking a property is a field. It is a pair of methods.
2. Being surprised by `<X>k__BackingField` and compiler-generated `<>c` closure types.
3. Expecting `GetReferencedAssemblies()` to list *loaded* assemblies. It lists what was referenced at compile time; use `AppDomain.CurrentDomain.GetAssemblies()` for what is actually loaded.

2.7 Audience questions

Q: Does metadata make my assemblies much bigger? It is a modest fraction of a typical assembly and it is not optional. You cannot trade it away for size.

Q: Can I read metadata without loading and executing the assembly? Yes, and you should for untrusted or cross-architecture assemblies: use `System.Reflection.Metadata` (low-level reader) or `MetadataLoadContext` (reflection-like API that never runs code). Example 03's `GetCustomAttributesData()` is the same idea at member level.

Q: Is reflection reading my source files? No. Source is not deployed and is not needed. Everything comes from the DLL.

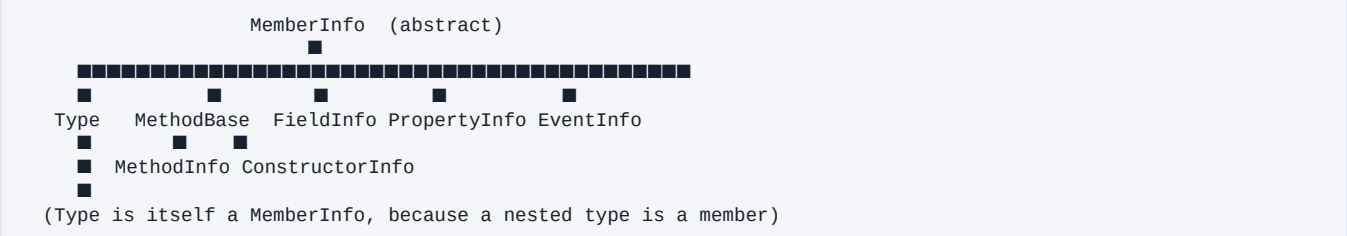
2.8 Documentation

- Assemblies in .NET — https://learn.microsoft.com/en-us/dotnet/standard/assembly/

- `MetadataLoadContext` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.metadataloadcontext

3. Introduction to the Reflection API

3.1 The object model



Everything derives from `MemberInfo`, which is why they all have `Name`, `DeclaringType` and `GetCustomAttributes`.

3.2 The core types

Type	Represents	Key members
`Assembly`	a loaded assembly	`GetTypes`, `GetName`, `GetType(string)`, `Location`, `IsDynamic`
`Module`	a module	`Name`, `GetTypes`
`Type`	a type	`GetMethods`, `GetProperties`, `GetFields`, `GetConstructors`, `IsAssignableFrom`, `BaseType`, `GetInterfaces`
`MethodInfo`	a method	`Invoke`, `ReturnType`, `GetParameters`, `CreateDelegate`
`ConstructorInfo`	a constructor	`Invoke`, `GetParameters`
`PropertyInfo`	a property	`GetValue`, `SetValue`, `CanRead`, `CanWrite`, `PropertyType`
`FieldInfo`	a field	`GetValue`, `SetValue`, `IsPrivate`, `IsInitOnly`
`ParameterInfo`	a parameter	`Name`, `ParameterType`, `HasDefaultValue`
`BindingFlags`	a filter	see §6

3.3 How they relate

```
Assembly.GetTypes()        → Type[]
Type.GetMethods()          → MethodInfo[]
MethodInfo.GetParameters() → ParameterInfo[]
MethodInfo.ReturnType      → Type        ← back to Type: the graph is navigable
PropertyInfo.DeclaringType → Type
```

The graph is fully connected: from any node you can reach any other. That is what makes generic framework code possible.

3.4 Common mistakes

1. **Not caching.** `GetProperties()` allocates a fresh array each call.
2. **Using `==` on `Type` for assignability.** Use `IsAssignableFrom` so inheritance counts.
3. **Confusing `typeof(T)` and `GetType()`.** The first is the static type, the second the runtime type. Example 02 shows them disagreeing.
4. **Forgetting generics complicate `Name`.** `List<int>.Name` is `List\1; use` GetGenericArguments()` for the rest.

3.5 Audience questions

Q: Are two `typeof(X)` calls the same object? Yes — for a given type in a given load context the runtime returns the same `Type` instance, so `ReferenceEquals` is true and `==` is safe. Across *different* load contexts they are different types (example 04 §3).

Q: Why is there both `MethodBase` and `MethodInfo`? `MethodBase` is what methods and constructors share (`Invoke`, `GetParameters`). `MethodInfo` adds `ReturnType`, which a constructor does not have.

Q: Thread safety? Reading reflection objects is thread-safe. Your own caches around them are not — use `ConcurrentDictionary`.

3.6 Documentation

- `System.Reflection` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection
- `Type` — https://learn.microsoft.com/en-us/dotnet/api/system.type

---

4. Assembly and Type Discovery

4.1 The four ways to get a `Type`

```
Type a = typeof(BankAccount);                                    // compile time
Type b = someObject.GetType();                                   // runtime type of an instance
Type? c = Type.GetType("Metaprogramming.ReflectionDemo.BankAccount");// from a string
Type? d = Assembly.GetExecutingAssembly().GetType("...SavingsAccount");// from one assembly
```

Verified output:

```
(a) typeof(BankAccount)           -> Metaprogramming.ReflectionDemo.BankAccount
(b) instance.GetType()            -> Metaprogramming.ReflectionDemo.SavingsAccount   (note: NOT BankAccount)
    static type of the variable is object, runtime type is SavingsAccount
(c) Type.GetType("...BankAccount") -> Metaprogramming.ReflectionDemo.BankAccount
(d) assembly.GetType("...")        -> Metaprogramming.ReflectionDemo.SavingsAccount
```

Way	Resolved	Checked	Use when
`typeof(T)`	compile time	**yes**	the type is known — always prefer this
`GetType()`	run time	n/a	you have an instance
`Type.GetType(string)`	run time	no	the name is data (config, plugin manifest)
`Assembly.GetType(string)`	run time	no	you know which assembly to look in

4.2 Assembly-qualified names, and why `Type.GetType` returns `null`

This is the most common reflection bug. `Type.GetType(string)` searches only:

1. the **calling** assembly, and
2. `System.Private.CoreLib` (the core library).

A type in any other assembly needs an **assembly-qualified name**. Verified:

```
Unqualified name from another assembly:
  Type.GetType("System.Text.Json.JsonSerializerOptions") = null  <- this is the trap
Assembly-qualified name:
  Type.GetType("...JsonSerializerOptions, System.Text.Json") = JsonSerializerOptions
```

The canonical form is on the type itself:

```
typeof(JsonSerializerOptions).AssemblyQualifiedName =
  System.Text.Json.JsonSerializerOptions, System.Text.Json, Version=10.0.0.0,
  Culture=neutral, PublicKeyToken=cc7b13ffcd2ddd51
```

Name grammar:

Form	Example
Namespace + name	`MyApp.Customer`
Assembly-qualified	`MyApp.Customer, MyApp`
Fully qualified	`MyApp.Customer, MyApp, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null`
Nested type	`MyApp.Outer+Inner` ← a `+`, not a `.`
Generic	`System.Collections.Generic.List\`1[[System.Int32, System.Private.CoreLib]]`
Array	`MyApp.Customer[]`

The five reasons `Type.GetType` returns `null`:

1. The name is not assembly-qualified and the type is elsewhere. *(most common)*
2. The containing assembly is not loaded, and nothing triggers a load.
3. A nested type written with `.` instead of `+`.
4. A typo — including the wrong namespace, or omitting it.
5. The assembly version/culture/key in a fully qualified name does not match.

Make it diagnosable. Pass `throwOnError: true` when a missing type is a bug:

```
Type.GetType("Totally.Missing.Type", throwOnError: true);
// -> TypeLoadException, with a message naming what it looked for
```

Verified:

```
  TypeLoadException: Could not load type 'Totally.
```

A silent `null` that becomes a `NullReferenceException` three frames later is far worse than an immediate, named exception.

4.3 Enumerating types

```
Assembly current = Assembly.GetExecutingAssembly();
Type[] all = current.GetTypes();
```

Verified:

```
assembly.GetTypes() returned 8 types (compiler-generated ones included).

The types we actually declared:
  BankAccount            abstract class  base=Object
  CurrentAccount         class           base=BankAccount
  FixedDepositAccount    class           base=BankAccount
  IInterestBearing       interface       base=-
  LedgerEntry            class           base=Object
  Program                static class    base=Object
  SavingsAccount         class           base=BankAccount
```

Note `GetTypes()` includes **compiler-generated** types — closure classes (`<>c`), iterator state machines, async state machines. Filter them (`!t.Name.StartsWith('<')`) or your "list my types" feature looks bizarre.

Method	Returns
`GetTypes()`	every type, public and not
`GetExportedTypes()`	only publicly visible types
`DefinedTypes`	every type as `TypeInfo`

4.4 Finding implementations — the core of plugin discovery

```
all.Where(t => typeof(IInterestBearing).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract)
```

Verified:

```
  SavingsAccount implements IInterestBearing
  FixedDepositAccount implements IInterestBearing
```

Three details that are all mistakes if you skip them:

1. `IsAssignableFrom` — accounts for inheritance. `t == typeof(I)` would find nothing.
2. `!t.IsInterface` — the interface itself is assignable from itself.
3. `!t.IsAbstract` — an abstract base cannot be instantiated. Note that `IsAbstract` is also `true` for `static` classes.

4.5 `ReflectionTypeLoadException` — the production detail

`GetTypes()` **throws** if any type in the assembly cannot be loaded, typically because a dependency is missing. The types that *did* load are still available:

```
private static IEnumerable<Type> SafeGetTypes(Assembly assembly)
{
    try
    {
        return assembly.GetTypes();
    }
    catch (ReflectionTypeLoadException ex)
    {
        Console.WriteLine("  warning: " + ex.LoaderExceptions.Length + " type(s) failed to load in "
            + assembly.GetName().Name + "; continuing with the rest.");
        return ex.Types.Where(t => t is not null).Cast<Type>();
    }
}
```

Any host that scans assemblies it did not build **must** do this. One plugin with a missing dependency would otherwise take down discovery for all of them.

4.6 Audience questions

Q: How do I scan every assembly in my application? `AppDomain.CurrentDomain.GetAssemblies()` returns the *loaded* ones — and assemblies load lazily, so one you have not touched yet will be missing. That is why scanning libraries ask you to pass the assemblies explicitly, or scan DLL files in a directory.

Q: Does `GetTypes()` force dependent assemblies to load? It can. Resolving a base type or an interface requires loading the assembly that defines it. This is one reason type scanning costs startup time.

Q: Can I scan without loading anything? Yes — `MetadataLoadContext`. Use it to vet a plugin before committing to loading it.

4.7 Documentation

- `Assembly` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.assembly
- `Type` — https://learn.microsoft.com/en-us/dotnet/api/system.type

---

5. Inspecting Types

5.1 The four families

```
Type t = typeof(SavingsAccount);

ConstructorInfo[] ctors = t.GetConstructors();
PropertyInfo[] properties = t.GetProperties();
MethodInfo[] methods = t.GetMethods();
FieldInfo[] fields = t.GetFields();
```

Verified:

```
CONSTRUCTORS (ConstructorInfo):
  .ctor(String accountNumber, Decimal balance, Decimal annualRate)

PROPERTIES (PropertyInfo):
  Decimal    AnnualRate     {get}   declared on SavingsAccount
  String     AccountNumber  {get}   declared on BankAccount
  Decimal    Balance        {get;set}   declared on BankAccount
  String     Nickname       {get;set}   declared on BankAccount
```

Note `DeclaringType`: `GetProperties()` returns **inherited** members too. Use `BindingFlags.DeclaredOnly` to see only what this type declares.

Also note `Balance` reports `{get;set}` even though its setter is `protected`. `CanWrite` is true if *a* setter exists, regardless of accessibility — to find a *public* setter use `GetSetMethod()` (which returns `null` for a non-public one) or `GetSetMethod(nonPublic: true)`.

5.2 Reading and writing

```
PropertyInfo balance = t.GetProperty(nameof(SavingsAccount.Balance))!;
Console.WriteLine(balance.GetValue(account));

PropertyInfo nickname = t.GetProperty(nameof(SavingsAccount.Nickname))!;
nickname.SetValue(account, "Rainy day fund");

FieldInfo? auditTrail = t.GetField("_auditTrail", BindingFlags.NonPublic | BindingFlags.Instance);
auditTrail?.SetValue(account, "rewritten-by-reflection");
```

Verified:

```
Read  Balance via PropertyInfo.GetValue -> 5000
Wrote Nickname via PropertyInfo.SetValue -> Rainy day fund
Read  private field _auditTrail         -> created
Wrote private field _auditTrail         -> rewritten-by-reflection
```

Use `nameof(...)` rather than `"Balance"` wherever the member is known at compile time. It is a string in the IL either way, but a rename or typo becomes a **build error**. This one habit removes a large share of reflection bugs.

5.3 Invoking by name

```
MethodInfo? method = account.GetType().GetMethod("AddInterest");
object? result = method.Invoke(account, [12]);
```

Verified:

```
Found: Decimal AddInterest(Int32)
Invoke(account, [12]) returned 120.00
Balance is now 2120.00
```

Two failure modes you must know.

Wrong argument types fail at run time:

```
  ArgumentException: Object of type 'System.String' cannot be converted to type 'System.Int32'.
```

And an exception thrown *inside* the target is **wrapped**:

```
  TargetInvocationException -> InnerException is the real one:
    InvalidOperationException: Insufficient funds: balance is 2120.00.
```

`TargetInvocationException` is the single most important exception to understand here. If you let it escape, every `catch (InvalidOperationException)` upstream silently stops matching. Always unwrap:

```
catch (TargetInvocationException ex) when (ex.InnerException is not null)
{
    ExceptionDispatchInfo.Capture(ex.InnerException).Throw();  // preserves the stack trace
}
```

`ExceptionDispatchInfo` rather than `throw ex.InnerException` — the latter resets the stack trace. Example 12 does this properly.

5.4 Overloads

`GetMethod("Foo")` throws `AmbiguousMatchException` when `Foo` is overloaded. Specify the signature:

```
MethodInfo mathMax = typeof(Math).GetMethod(nameof(Math.Max), [typeof(int), typeof(int)])!;
```

5.5 Documentation

- `MethodInfo` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.methodinfo
- `PropertyInfo` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.propertyinfo

---

6. `BindingFlags` — Public vs Non-Public

6.1 Definition

`BindingFlags` is a `[Flags]` enum that filters which members a reflection lookup returns. **If a member is "missing", this is almost always why.**

6.2 The important values

Flag	Meaning
`Public`	public members
`NonPublic`	private, protected, internal, protected internal
`Instance`	per-object members
`Static`	per-type members
`DeclaredOnly`	this type only — no inherited members
`FlattenHierarchy`	include inherited **static** members
`IgnoreCase`	case-insensitive name matching

The default when you pass no flags is `Public | Instance | Static`. So:

- private members are **missing** unless you ask for `NonPublic`;
- inherited members are **included** unless you ask for `DeclaredOnly`.

You must pass **at least one** of `Public`/`NonPublic` **and** at least one of `Instance`/`Static`, or you get an empty result. An empty array from a type you know has members is nearly always a `BindingFlags` mistake.

6.3 Verified counts

```
GetMethods() with no arguments means Public | Instance | Static:
  count = 14 (includes inherited object methods)

Public | Instance | DeclaredOnly  -> 4
NonPublic | Instance | DeclaredOnly -> 1

The private methods reflection can see:
  private String Normalise(String note)

Static members need BindingFlags.Static - they are NOT returned by Instance:
  public static String ProductCode()
```

6.4 Accessibility is not security

This deserves its own statement, because it is a genuine and common misunderstanding:

> `private` ***is a compiler rule, enforced at compile time. Reflection with*** `BindingFlags.NonPublic` ***reads private state, and that is allowed. Accessibility is an encapsulation feature, not a security boundary.***

Code running in your process can read any field of any object. There is no supported in-process sandbox in modern .NET (Code Access Security was a .NET Framework feature and is gone). If something must be protected from code in the same process, the answer is a process or OS boundary, not an access modifier.

6.5 Legitimate and illegitimate uses of `NonPublic`

Legitimate: deserialization restoring objects with no public setters; ORM materialisation; test fixtures setting up otherwise unreachable state; diagnostics.

Illegitimate: routing around another library's invariants in production code. It compiles, it works, and it breaks on their next patch release — private members are not part of any contract.

6.6 Audience questions

Q: Why did `GetMethods()` return 14 methods on a class with 4? Inherited members. `object` contributes `ToString`, `Equals`, `GetHashCode`, `GetType`; the base class contributes more; property accessors are methods too. Use `DeclaredOnly`.

Q: Can reflection call a private method? Yes — `GetMethod(name, BindingFlags.NonPublic | BindingFlags.Instance)` then `Invoke`. See §6.4 for why this is allowed.

Q: Should I unit-test private methods this way? Generally no — test through the public surface, or make the unit a separate type. The pragmatic exception is a complex private algorithm, where `InternalsVisibleTo` plus `internal` is cleaner than reflection.

6.7 Documentation

- `BindingFlags` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.bindingflags

7. Dynamic Object Creation — `Activator.CreateInstance`

7.1 Definition

`Activator.CreateInstance` creates an instance of a type represented by a `Type` object, choosing a constructor by matching the arguments you supply.

7.2 Why it exists

`new Foo()` requires the compiler to know `Foo`. When the type comes from a string, a config file or a scanned assembly, there is no name to write. `Activator` is the bridge from **"a `Type` I discovered"** to **"a live object"**.

7.3 How it works internally

1. Validate the `Type` — not null, not abstract, not an interface, not open generic.
2. Enumerate `ConstructorInfo`s matching the `BindingFlags`.
3. Use a `Binder` to pick the best match for the argument types, with coercions.
4. Allocate the object and run the chosen constructor with the unboxed arguments.
5. Return it as `object` — you then cast.

Steps 2–3 are why it is slower than `new`. `new` is a single `newobj` IL instruction with the constructor resolved at compile time.

7.4 The forms

```
// (a) generic — needs a public parameterless constructor
var current = Activator.CreateInstance<CurrentAccount>();

// (b) Type plus constructor arguments
object? savings = Activator.CreateInstance(typeof(SavingsAccount), "SB-77", 15000m, 0.07m);

// (c) the realistic case — the type name is a string
Type? discovered = Type.GetType("Metaprogramming.ReflectionDemo.FixedDepositAccount");
object? fd = Activator.CreateInstance(discovered!, "FD-12", 100000m, 24);

// (d) reach a non-public constructor
object? internalOnly = Activator.CreateInstance(typeof(LedgerEntry), nonPublic: true);

// (e) via ConstructorInfo — explicit, and cacheable
ConstructorInfo ctor = typeof(SavingsAccount).GetConstructor([typeof(string), typeof(decimal),
    typeof(decimal)])!;
var viaCtor = (SavingsAccount)ctor.Invoke(["SB-88", 250m, 0.03m]);
```

Verified:

```
(a) Activator.CreateInstance<CurrentAccount>()  -> Current CA-0000 balance=0
(b) Activator.CreateInstance(type, args)       -> Savings SB-77 balance=15000 rate=0.07
(c) from a string type name                   -> FixedDeposit FD-12 principal=100000 term=24m
(d) nonPublic:true reaches a private .ctor      -> LedgerEntry LE-fc5fb2
(e) ConstructorInfo.Invoke                     -> Savings SB-88 balance=250 rate=0.03
```

Form **(e)** is the one to use on a hot path: cache the `ConstructorInfo` (or better, build a delegate from it) and skip the constructor search on every call.

7.5 `Activator.CreateInstance` vs `new`

	`new SavingsAccount(...)`	`Activator.CreateInstance(type, ...)`
Type known	compile time	run time

	`new SavingsAccount(...)`	`Activator.CreateInstance(type, ...)`
IL	one `newobj`	a method call that searches and binds
Type-checked	**yes**	no
Speed	fastest	much slower
Wrong args	build error	`MissingMethodException`
Use when	the type is known	and only when it is not

7.6 Every failure mode, triggered deliberately

Verified output from example 02:

```
MissingMethodException  : Constructor on type 'Metaprogramming.ReflectionDemo.SavingsAccount' not found.
Abstract type           : Constructor on type 'Metaprogramming.ReflectionDemo.BankAccount' not found.
Private .ctor, nonPublic:false -> MissingMethodException (use the nonPublic overload)
Ctor threw              : TargetInvocationException -> ArgumentOutOfRangeException: Opening balance cannot be
    negative. (Parameter 'balance')
Null Type               : ArgumentNullException (paramName=type)
```

Exception	Cause
`MissingMethodException`	no constructor matches the arguments
`MissingMethodException`	the type is abstract, or an interface
`MissingMethodException`	the only constructor is non-public and you did not pass `nonPublic: true`
`TargetInvocationException`	the constructor itself threw — check `InnerException`
`ArgumentNullException`	the `Type` was null
`TypeLoadException`	the type could not be loaded

Note the abstract-type message says "Constructor … not found", which is **misleading** — the real problem is that the type is abstract. Check `type.IsAbstract` yourself and throw a clear error. A host that reports *"PluginX is abstract and cannot be instantiated"* is far kinder than one that reports *"Constructor on type PluginX not found"*.

Every one of these is a **run-time** failure. That is the price of reflective construction.

7.7 Real-world applications

- DI containers (example 07) — though they use `ActivatorUtilities`, which also resolves the constructor's dependencies.
- Deserializers creating your DTOs.
- Plugin hosts (example 04).
- EF Core materialising entities.
- Test frameworks instantiating test classes.

7.8 Common mistakes

1. **Using it when the type is known.** Use `new`.
2. **Not checking `IsAbstract` / `IsInterface` first**, then getting the confusing message above.
3. **Calling it per item in a loop** without caching a `ConstructorInfo` or delegate.
4. **Letting `TargetInvocationException` escape**, hiding the real error.
5. **Assuming a parameterless constructor exists.** Many types have none.
6. **Expecting `Activator` to supply dependencies.** It cannot — it knows nothing about your container. That is what `ActivatorUtilities` is for (Part 4 §2).

7.9 Audience questions

Q: Is `Activator.CreateInstance<T>()` as fast as `new T()`? No. Even the generic form goes through the activation machinery. For `T : new()` the JIT can optimise some cases, but do not assume parity — measure if it matters.

Q: How do DI containers make this fast? They resolve the constructor **once**, build a compiled factory delegate (expression tree or emitted IL), cache it, and never search again. Example 07 §8 shows the engine; Part 3 §4 shows the technique.

Q: Can it create a generic type at run time? Yes, but close it first: `typeof(List<>).MakeGenericType(typeof(int))`, then `CreateInstance` that. Passing an open generic throws.

Q: What about types with only a private constructor, like a singleton? `nonPublic: true` reaches it — see form (d). Which is another reminder that a private constructor is not a guarantee, merely a strong hint.

7.10 Documentation

- `Activator.CreateInstance` — https://learn.microsoft.com/en-us/dotnet/api/system.activator.createinstance

8. Attributes and Custom Attributes

8.1 Definition

An **attribute** is a class deriving from `System.Attribute` whose application to a declaration stores a row of metadata on that declaration. It is **declarative data**, not behaviour.

8.2 Why attributes exist

To attach information to code in the place it belongs, where it cannot drift away from what it describes, and to let separate machinery act on it. The alternative — an external configuration file listing members by name — is exactly the drift-prone design attributes replace.

8.3 How it works internally

Applying an attribute writes a `CustomAttribute` row recording the attribute type and its constructor arguments. **The attribute object is not stored** — only the recipe. `GetCustomAttributes` runs the constructor on demand, so:

```
  ReferenceEquals(first, second) = False
```

Two calls, two objects. (Part 1 §5.5.)

8.4 Declaring one

```
[AttributeUsage(AttributeTargets.Class, AllowMultiple = false, Inherited = false)]
internal sealed class TableAttribute(string name) : Attribute
{
    public string Name { get; } = name;              // positional (constructor) argument
    public string Schema { get; set; } = "dbo";      // named argument
}
```

Applied as:

```
[Table("invoice_lines", Schema = "sales")]
internal sealed class InvoiceLine { /* ... */ }
```

- **Positional arguments** are constructor parameters, required, in order.
- **Named arguments** are settable properties or fields, optional, in any order.
- Every argument must be a **compile-time constant** (or a `Type`, or an array of those). You cannot pass `DateTime.Now` or a `new`ed object — a real constraint that pushes complex configuration toward a fluent API.

By convention the class is named `XAttribute` and used as `[X]`; the compiler appends the suffix for you.

8.5 `AttributeUsage` — the three settings

Setting	Purpose	Default
`ValidOn` (constructor arg)	which declarations it may be applied to	`AttributeTargets.All`
`AllowMultiple`	may it repeat on one target?	`false`
`Inherited`	do derived types/overrides see it?	`true`

Verified:

```
TagAttribute:
  ValidOn       = Class
  AllowMultiple = True
  Inherited     = False
ValidateRangeAttribute:
  ValidOn       = Property
  AllowMultiple = False
  Inherited     = True
```

`AttributeTargets` is a `[Flags]` enum — `Class`, `Method`, `Property`, `Field`, `Parameter`, `Constructor`, `Assembly`, `Module`, `Enum`, `Event`, `Interface`, `Delegate`, `ReturnValue`, `GenericParameter`, `All`. Combine with `|`. Applying an attribute to a disallowed target is a **compile error**, which is the point: it is compiler-enforced documentation of intent.

8.6 `AllowMultiple`

```
[Tag("billing")]
[Tag("generated-nightly")]
[Tag("pii-free")]
internal sealed class InvoiceLine { }
```

```
InvoiceLine carries 3 [Tag] attributes:
  billing
  generated-nightly
  pii-free
```

With `AllowMultiple = false` a second application is **compile error CS0579** — the constraint is enforced by the compiler, not discovered at run time.

The trap: `GetCustomAttribute<T>()` (singular) *throws* when several are present:

```
  AmbiguousMatchException - use GetCustomAttributes<T>() instead.
```

8.7 `Inherited`

Both switches must agree. `BaseEntity` declares `[Audited]` (`Inherited = true`) and `[Table]` (`Inherited = false`); `DerivedEntity` declares neither:

```
inherit: false - look only at the type's own metadata:
  [Audited] found = False
  [Table]   found = False

inherit: true - also walk the base chain:
  [Audited] found = True   <- Inherited=true on the attribute, and we asked for inheritance
  [Table]   found = False  <- Inherited=false on the attribute, so inheritance is refused
```

> *The attribute must be declared `Inherited = true` **and** the call must pass `inherit: true`. Either one off means no result.*

Note also that `inherit: true` behaves differently per member kind, and does **not** walk interface implementations — an attribute on an interface member is not found on the implementing class this way. You must check the interface explicitly. (Example 12 deliberately puts `[Cacheable]` on the *interface* and reads it from the interface's `MethodInfo`, which is why it works there.)

8.8 Reading attributes

API	Constructs?	Use for
`IsDefined(type)`	no	cheapest existence check
`GetCustomAttribute<T>()`	yes	exactly one expected
`GetCustomAttributes<T>()`	yes	`AllowMultiple = true`

API	Constructs?	Use for
`GetCustomAttributesData()`	**no**	untrusted assemblies, tooling

`GetCustomAttributesData()` matters for two reasons: no attribute constructor runs (so untrusted code cannot execute), and the attribute type need not even be loadable.

8.9 The payoff — an attribute-driven engine

```
internal sealed class Payment
{
    [ValidateRange(0.01, 1_000_000)]
    public decimal Amount { get; set; }

    [ValidateText(ExactLength = 3, Required = true)]
    public string CurrencyCode { get; set; } = string.Empty;

    [ValidateText(Required = true)]
    public string Reference { get; set; } = string.Empty;

    public string? Notes { get; set; }   // no attributes: skipped silently
}
```

The engine never names `Payment`:

```
foreach (PropertyInfo property in type.GetProperties(BindingFlags.Public | BindingFlags.Instance))
{
    if (!property.CanRead) continue;
    object? value = property.GetValue(instance);

    var range = property.GetCustomAttribute<ValidateRangeAttribute>();
    if (range is not null && value is not null)
    {
        double numeric = Convert.ToDouble(value);
        if (numeric < range.Minimum || numeric > range.Maximum)
        {
            errors.Add(property.Name + " must be between " + range.Minimum + " and " + range.Maximum
                + " (was " + numeric + ")");
        }
    }
    // ... text rules
}
```

Verified:

```
VALID payment: Amount=2500 CurrencyCode="INR" Reference="PAY-001"
  OK - no violations

INVALID payment: Amount=-5 CurrencyCode="RUPEES" Reference=""
  ERROR: Amount must be between 0.01 and 1000000 (was -5)
  ERROR: CurrencyCode must be exactly 3 characters (was 6)
  ERROR: Reference is required
```

This is, in miniature, ASP.NET Core model validation.

8.10 Real-world attribute readers

Attribute	Read by	To decide
`[HttpGet]`, `[Route]`	ASP.NET Core, at startup	the routing table
`[Required]`, `[EmailAddress]`	MVC model binding	validation errors
`[JsonPropertyName]`, `[JsonIgnore]`	`System.Text.Json`	the JSON shape
`[Table]`, `[Column]`, `[NotMapped]`	EF Core, at model build	the schema
`[Fact]`, `[Theory]`	xUnit	which methods are tests
`[Obsolete]`	the **compiler**	whether to warn

Attribute	Read by	To decide
`[Serializable]`, `[Flags]`, `[DllImport]`	the runtime	low-level behaviour

8.11 Common mistakes

1. Singular `GetCustomAttribute<T>()` on a multiple-allowed attribute → `AmbiguousMatchException`.
2. Forgetting one of the two `Inherited`/`inherit` switches.
3. Expecting interface-member attributes to be inherited by implementations. They are not; check the interface.
4. Putting behaviour in the attribute. Attributes are data.
5. Omitting `[AttributeUsage]` and getting surprising defaults.
6. Reading attributes on a hot path without caching.
7. Trying to pass a non-constant attribute argument — a compile error.

8.12 Audience questions

Q: Can an attribute change behaviour on its own? No. It is inert until something reads it. `[JsonIgnore]` does nothing unless `System.Text.Json` looks for it. This is the most common misconception about attributes.

Q: Why are two `GetCustomAttribute` results different objects? Because only the constructor *arguments* are stored; each read constructs a fresh instance. Cache if you need identity or performance.

Q: Can I put an attribute on a record's positional parameter? Yes, with an explicit target: `[property: Required]` puts it on the generated property; `[param: ...]` on the parameter. Part 1 §5.3 does exactly this.

8.13 Documentation

- Writing custom attributes — https://learn.microsoft.com/en-us/dotnet/standard/attributes/writing-custom-attributes
- `AttributeUsageAttribute` — https://learn.microsoft.com/en-us/dotnet/api/system.attributeusageattribute
- `AttributeTargets` — https://learn.microsoft.com/en-us/dotnet/api/system.attributetargets

---

9. Practical Example — Dynamic Plugin Discovery

Code: `04-Plugin-Discovery` **(three projects)**

9.1 The architecture

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ PluginDiscovery.Contracts ■   IReportPlugin, [ReportPlugin]
■   (the shared contract)   ■   Referenced by BOTH sides. Tiny. Stable.
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
          ■
    ■■■■■■■■■■■■■■■■■■■■
    ■                  ■
■■■■▼■■■■■■■■■■■■■  ■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■   ...Host     ■  ■ ...Plugins.Sales            ■
■ (the app)     ■  ■ MonthlySalesPlugin          ■
■               ■  ■ TopCustomersPlugin          ■
■ scans a       ■  ■ ExperimentalForecastPlugin  ■
■ folder,       ■  ■ UnannotatedHelperPlugin     ■
■ loads, runs   ■  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■      built to  bin/.../plugins/*.dll

The host has NO compile-time reference to the plugin assembly:
    <ProjectReference ... ReferenceOutputAssembly="false" />
It builds the plugin (so the demo works) but cannot see its types.
```

9.2 The contract

```
public interface IReportPlugin
{
    string Name { get; }
    string Render(IReadOnlyDictionary<string, string> parameters);
}

[AttributeUsage(AttributeTargets.Class, AllowMultiple = false, Inherited = false)]
public sealed class ReportPluginAttribute(string displayName) : Attribute
{
    public string DisplayName { get; } = displayName;
    public string Version { get; set; } = "1.0.0";
    public int Order { get; set; } = 100;
    public bool Enabled { get; set; } = true;
}
```

Why both an interface and an attribute? The interface says *what the plugin can do*; the attribute says *what the host should know before deciding to create it*. The host can list, filter and order plugins from metadata **without constructing any of them** — which matters when construction might be slow or might fail.

9.3 The whole discovery mechanism

```
private static List<PluginDescriptor> DescribePluginsIn(Assembly assembly)
{
    var found = new List<PluginDescriptor>();

    foreach (Type type in SafeGetTypes(assembly))
    {
        if (type.IsAbstract || type.IsInterface || !typeof(IReportPlugin).IsAssignableFrom(type))
        {
            continue;
        }

        var attribute = type.GetCustomAttribute<ReportPluginAttribute>();
        if (attribute is null || !attribute.Enabled)
        {
            continue;
        }

        found.Add(new PluginDescriptor(type, attribute.DisplayName, attribute.Version, attribute.Order));
    }

    return found;
}
```

Twenty lines, and every technique in Part 2 is in them: `GetTypes`, `IsAssignableFrom`, `IsAbstract`, `GetCustomAttribute`, and then `Activator` to build.

9.4 Verified workflow

```
Plugin folder: ...\PluginDiscovery.Host\bin\Debug\net10.0\plugins
Candidate assemblies: PluginDiscovery.Plugins.Sales.dll

Loaded PluginDiscovery.Plugins.Sales v1.0.0.0
  into load context: PluginALC:PluginDiscovery.Plugins.Sales
  collectible      : True
  accepted MonthlySalesPlugin as "Monthly Sales Summary" v2.1.0
  accepted TopCustomersPlugin as "Top Customers" v2.1.0
  skipped ExperimentalForecastPlugin - [ReportPlugin(Enabled = false)]
  skipped UnannotatedHelperPlugin - implements IReportPlugin but has no [ReportPlugin]

  --- monthly-sales ---
    Monthly sales for 2026-11
      South 418,500.00
      West  302,750.00
      North 211,000.00
      TOTAL 932,250.00
      (rendered by PluginDiscovery.Plugins.Sales v1.0.0.0)
```

Note the two rejections. A good host reports **why** it declined something — a plugin author whose DLL is silently ignored has nothing to debug.

9.5 Dependency isolation and `AssemblyLoadContext`

Each plugin gets its own load context:

```
internal sealed class PluginLoadContext : AssemblyLoadContext
{
    private readonly AssemblyDependencyResolver _resolver;
    private static readonly string[] SharedAssemblies = ["PluginDiscovery.Contracts"];

    public PluginLoadContext(string pluginPath)
        : base(name: "PluginALC:" + Path.GetFileNameWithoutExtension(pluginPath), isCollectible: true)
        => _resolver = new AssemblyDependencyResolver(pluginPath);

    protected override Assembly? Load(AssemblyName assemblyName)
    {
        // Share the contract: returning null defers to the Default context.
        if (assemblyName.Name is not null && SharedAssemblies.Contains(assemblyName.Name))
        {
            return null;
        }

        string? path = _resolver.ResolveAssemblyToPath(assemblyName);
        return path is null ? null : LoadFromAssemblyPath(path);
    }
}
```

Two jobs, and the second is the subtle one:

1. **Isolate private dependencies** so plugin A can use Newtonsoft 11 and plugin B version 13.
2. **Deliberately NOT isolate the shared contract**, so both sides use the *same* `IReportPlugin` type.

Verified:

```
Host's   IReportPlugin is from PluginDiscovery.Contracts in context Default
Plugin's IReportPlugin is from PluginDiscovery.Contracts in context Default

Same CLR type? True
Cast works?    True
```

If you get this wrong the context loads its own copy of the contracts, the plugin implements a *different* `IReportPlugin`, and the cast fails with `InvalidCastException` whose message names **the same type twice**:

> *Unable to cast object of type 'MonthlySalesPlugin' to type 'IReportPlugin'*

…when `MonthlySalesPlugin` obviously implements `IReportPlugin`. That baffling message is the signature symptom of a duplicated contract assembly. Type identity in .NET is **(assembly identity + load context + full name)**, not just the name.

9.6 Unloading

`isCollectible: true` makes `Unload()` possible. Verified:

```
Ran "monthly-sales" inside collectible context PluginALC:PluginDiscovery.Plugins.Sales
Unload() has been requested. Now the GC has to actually collect it.
  collection attempt 1: alive = True
  collection attempt 2: alive = False

Load context unloaded: True
```

`Unload()` **is a request, not an action.** The context dies only when nothing reachable refers to it, its assemblies, or any object whose type came from it.

What keeps a context alive — all common mistakes:

- any live instance of a plugin type;
- a cached `Type`, `MethodInfo` or `Assembly` from the plugin;
- a delegate pointing at plugin code (an event handler left subscribed);
- a static field in the **host** holding any of the above.

Note the `[MethodImpl(MethodImplOptions.NoInlining)]` on the helper in the example: if the JIT inlined it, the locals could stay rooted in the caller's frame and the context would never collect. That is how delicate this is.

And unloading only frees **managed state**. It cannot undo side effects the plugin already caused, and a plugin that P/Invoked a native library may have pinned it.

9.7 Trusted vs untrusted assemblies — the security section

This is the most important paragraph in Part 2, and the one most often got wrong:

> `AssemblyLoadContext` ***is an isolation boundary for TYPE IDENTITY and VERSIONS. It is NOT a security sandbox.***

Once you call `LoadFromAssemblyPath`, that code runs with your process's full privileges. It can:

- read and write any file your process can;
- open sockets, start processes, read environment variables and secrets;
- use reflection against your own private state (§6.4);
- call `Environment.Exit` and take the process down.

There is no supported in-process sandbox in modern .NET. Code Access Security and AppDomain partial trust were .NET Framework features and have been removed. If you must run untrusted code, isolate it where the OS can enforce it: a separate process with a reduced token, a container, a VM, or a WASM runtime.

Practical hardening for a trusted plugin model:

Measure	Why
Verify a strong name or Authenticode signature before loading	establishes provenance
Keep the contracts assembly tiny and version it deliberately	it is your ABI
Inspect with `MetadataLoadContext` first and reject early	no plugin code runs during vetting
Wrap every plugin call in `try`/`catch`	one bad plugin must not kill the host
Log path, version and hash of everything loaded	you will need this after an incident
Use `ActivatorUtilities`, not bare `Activator`	plugins can then take dependencies you control

9.8 Version compatibility

The contracts assembly is your ABI. Rules that save pain:

- **Adding a member to an existing interface is a breaking change** for every existing plugin. Use a *new* interface (`IReportPluginV2`) and check for both.
- Prefer adding optional **properties on the attribute** over changing the interface — old plugins keep working and simply take the defaults.
- Record a plugin's expected contract version and refuse mismatches with a clear message rather than letting a `MissingMethodException` surface later.

9.9 Audience questions

Q: Can I load a plugin without restarting the app? Loading, yes — that is example 04. *Replacing* one needs a collectible context, dropping every reference, unloading, and loading the new DLL; and the unload is cooperative, so it may not happen promptly. Most production systems take the simpler path: run plugins in a child process and restart that.

Q: Is `AssemblyLoadContext` a security boundary? No. See §9.7. Isolation of type identity, not of privileges.

Q: Why not just `Assembly.LoadFrom`? It loads into the default context: no dependency isolation (plugin versions collide with yours) and no possibility of unloading. Fine for a trusted first-party DLL, wrong for a plugin model.

Q: MEF instead? MEF (`System.Composition`) packages this pattern with a catalog and richer composition. Worth it for a large plugin surface. Understanding the 20 lines in §9.3 first means MEF stops being magic.

9.10 Documentation

- Understanding `AssemblyLoadContext` — https://learn.microsoft.com/en-us/dotnet/core/dependency-loading/understanding-assemblyloadcontext
- Assembly unloadability — https://learn.microsoft.com/en-us/dotnet/standard/assembly/unloadability
- Create an app with plugin support — https://learn.microsoft.com/en-us/dotnet/core/tutorials/creating-app-with-plugin-support
- `AssemblyLoadContext` — https://learn.microsoft.com/en-us/dotnet/api/system.runtime.loader.assemblyloadcontext

---

Part 2 summary

1. Introspection reads the metadata every assembly must already carry.
2. The hierarchy is Assembly → Module → Type → Member, and the API mirrors it exactly.
3. All `*Info` types derive from `MemberInfo`, so the object graph is fully navigable.
4. Prefer `typeof`; use `Type.GetType` only when the name is genuinely data — and remember it needs an **assembly-qualified** name for other assemblies.
5. `BindingFlags` decides what you see. A missing member is almost always a flags bug.
6. `private` is encapsulation, **not security**.
7. `Activator.CreateInstance` bridges `Type` → object, and every one of its failures is a run-time failure.
8. Attributes are **inert declarative data**; something has to read them.
9. Interface + attribute + reflection + `Activator` = a plugin system in 20 lines.
10. `AssemblyLoadContext` isolates **type identity**, not **privileges**.

Part 3 — Runtime Code Generation

Session time: 12 minutes (slides 16–23) Runnable examples: `05-DynamicMethod`, `06-ReflectionEmit`

> ***Everything in this part was verified by running it on .NET 10.0.11 (SDK 10.0.400).*** *This matters more here than anywhere else in the session: much of the published material about* `Reflection.Emit` *describes .NET Framework APIs that* ***do not exist*** *on modern .NET. Where this guide contradicts an older tutorial, the difference is usually that.*

1. What Is Runtime Code Generation?

1.1 Definition

Runtime code generation is creating new executable code — new IL, compiled to machine code by the JIT — while the process is running.

Part 2 **read** metadata that already existed. Part 3 **writes** code that did not.

1.2 In simple English

Part 2 was reading a book's table of contents. Part 3 is writing a new chapter while the book is being read, and then reading that chapter.

1.3 Why applications generate code dynamically

There is exactly one good reason, and several bad ones.

The good reason: speed, when the shape is only known at run time.

Reflection is flexible but slow per call, because every call re-does argument binding and access checks. If you must perform the same reflective operation millions of times, you can pay the reflection cost **once** to generate a specialised method, then call that method directly. You convert a per-call cost into a one-off cost.

This is why it appears where it does:

User	Generates
`System.Text.Json` (reflection mode)	fast property accessors per type
EF Core	entity materialisers and property accessors
DI containers	factory delegates per service
Moq, NSubstitute, FakeItEasy	mock **types**
Castle DynamicProxy	proxy **types**
`RegexOptions.Compiled`	a matcher for your specific pattern
ASP.NET Core	model binders, request delegates

A secondary reason: you need a TYPE that did not exist. A mock or proxy must *be* an `IFoo` to be passed where an `IFoo` is expected. No amount of reflection creates a type; only code generation does.

The bad reasons: because it is clever; to avoid writing a `switch`; to build a scripting language you could have taken off the shelf.

1.4 Runtime support requirements

Code generation needs a runtime that can compile code at run time. Two flags tell you:

```
RuntimeFeature.IsDynamicCodeSupported   // can I create code at all?
RuntimeFeature.IsDynamicCodeCompiled    // will it be JIT-compiled to machine code?
```

Verified on this machine:

```
RuntimeInformation.FrameworkDescription : .NET 10.0.11
RuntimeFeature.IsDynamicCodeSupported   : True
RuntimeFeature.IsDynamicCodeCompiled    : True
```

Platform	`IsDynamicCodeSupported`	`IsDynamicCodeCompiled`	Effect
CoreCLR (server, desktop, container)	`true`	`true`	works, fast
Native AOT	`false`	`false`	`Reflection.Emit` **throws** `PlatformNotSupportedException`
Mono interpreter (iOS, some WASM)	`true`	`false`	works, but runs **interpreted** — the speed benefit is gone
Mono with JIT (Android)	`true`	`true`	works

The practical rule for library authors: check `IsDynamicCodeSupported` and keep a reflection-based fallback. That is exactly what `System.Text.Json` and the DI container do — which is why they still function under AOT, just more slowly or via source generation.

1.5 Audience questions

Q: Is this the same as `eval` in JavaScript or Python? Conceptually similar — new code at run time — but much lower level. There is no C# compiler involved: you emit IL instruction by instruction. The closest thing to `eval` in .NET is hosting Roslyn (`CSharpScript.EvaluateAsync`), which compiles real C# at run time. That is a different, heavier tool.

Q: Is generated code slower than compiled code? After the JIT has compiled it, no — the JIT does not know or care that the IL was generated rather than loaded from a file. The costs are the generation itself and the first-call JIT.

Q: Can I debug generated code? Barely. There is no source. You can attach symbols to a *persisted* assembly (§5), which is the main practical reason to persist one.

1.6 Documentation

- Emitting dynamic methods and assemblies — https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/emitting-dynamic-methods-and-assemblies
- `RuntimeFeature.IsDynamicCodeSupported` — https://learn.microsoft.com/en-us/dotnet/api/system.runtime.compilerservices.runtimefeature.isdynamiccodesupported

---

2. Reflection vs Reflection.Emit

2.1 The comparison

	Reflection (`System.Reflection`)	**Reflection.Emit (`System.Reflection.Emit`)**
Purpose	inspect and invoke existing code	create new code
Direction	read	write
Key APIs	`Type`, `MethodInfo`, `PropertyInfo`, `Activator`	`DynamicMethod`, `ILGenerator`, `TypeBuilder`, `MethodBuilder`
Input	an assembly's metadata	IL instructions you emit
Output	information, or an `Invoke` result	a callable method or a new `Type`
Speed	slow **per call**	slow **once**, then near-direct per call
Complexity	low — a day to learn	high — IL, the evaluation stack, verification
Type safety	checked at run time, throws clearly	**not always checked at all** (§8)
Native AOT	mostly works	**does not work**
Debuggability	poor	worse
Use when	you need to *know* something	you need to *do* something, repeatedly and fast

2.2 They are partners, not rivals

Almost all real Emit code starts with reflection:

```
PropertyInfo property = typeof(Employee).GetProperty("Salary")!;  // REFLECTION: find the target
MethodInfo getter = property.GetGetMethod()!;                     // REFLECTION

var method = new DynamicMethod("GetSalary", typeof(decimal), [typeof(Employee)]);
ILGenerator il = method.GetILGenerator();
il.Emit(OpCodes.Ldarg_0);
il.Emit(OpCodes.Callvirt, getter);                                // EMIT: use what reflection found
il.Emit(OpCodes.Ret);

var fast = method.CreateDelegate<Func<Employee, decimal>>();      // from here on: no reflection
```

Reflection **discovers** the target; Emit **bakes it in**. The result is used indefinitely with no further reflection.

2.3 The measured numbers

Verified, Release build, 2,000,000 iterations, reading one `decimal` property (`05-DynamicMethod`):

```
technique                              total ms    relative to direct
-----------------------------------------------------------------------
direct C# property access                     8        1.0x
PropertyInfo.GetValue                        67        8.4x
MethodInfo.Invoke                            59        7.4x
cached delegate (typed)                      10        1.2x
emitted IL (typed, no box)                   10        1.2x
emitted IL (object->object, boxes)           21        2.6x
```

Read these five rows carefully — there are four lessons, and the third is the one most talks get wrong.

1. **Reflection per call costs ~7–8× a direct access.** Real, but note the units: 67 ms for *two million* reads is ~33 nanoseconds each. "Reflection is slow" is true per call and often irrelevant in absolute terms. If you do it once per web request, it is free. If you do it per row of a million-row import, it is your bottleneck.
1. **A cached delegate and emitted IL land in the same speed class** (both 1.2×), and both are close to direct access. The win comes from *caching the work*, not from IL specifically.

1 `MethodInfo.CreateDelegate` **gets you emit-level speed with no IL at all.** Compare rows 4 and 5: identical. **Try this before writing a single** `OpCode`**.** This is the single most useful practical takeaway in Part 3.

1 **Boxing and signature shape dominate.** Rows 5 and 6 are the same property, the same technique, differing only in signature: `object → object` pays a `castclass` and a `decimal` box on every call, and that alone costs more than the choice between reflection and emit. Optimising "reflection vs emit" while ignoring boxing is optimising the wrong thing.

> ***Caveat on these numbers.*** *They come from* `Stopwatch` *in a loop on one machine, in one run, in Release. They are honest to an order of magnitude and no better. Use* ***BenchmarkDotNet*** *for anything you intend to act on.*

2.4 Audience questions

Q: Is Reflection.Emit faster than reflection? The *generated code* is far faster per call. *Generating* it is slower than a single reflective call. So: faster if you amortise over many calls, slower if you call once. And a cached delegate usually gets you the same benefit for a fraction of the risk.

Q: Where is the break-even? Depends on the operation, but the order of magnitude is thousands of calls. Below that, reflection. If you cannot estimate the call count, you do not have a performance problem yet.

Q: Does .NET itself use Reflection.Emit? Yes — historically and currently, in the places listed in §1.3. But note that several of those have been *migrating away* from it toward source generators precisely because of AOT. The trend is away from runtime emit.

---

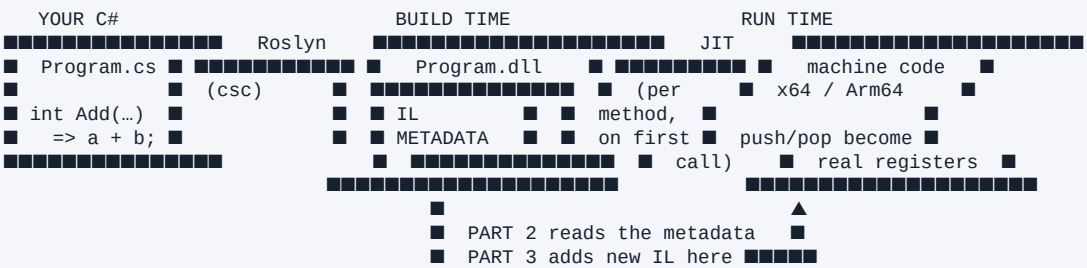
3. CIL / MSIL — The Language You Are Generating

3.1 Definition

CIL (Common Intermediate Language), also called **MSIL** or just **IL**, is the stack-based instruction set that every .NET language compiles to. It is defined by **ECMA-335**.

The three names mean the same thing. "CIL" is the standard's term, "MSIL" is the older Microsoft one, "IL" is what people say.

3.2 The compilation pipeline



Why the extra step? One IL assembly runs on x64, Arm64, Windows, Linux and macOS. The JIT specialises for the actual CPU at run time — it can use AVX-512 on a machine that has it without you shipping a separate build.

3.3 Exceptions to the simple pipeline

The diagram above is the common case. Be accurate about the rest:

Mode	How IL becomes machine code
JIT (default)	per method, on first call, at run time
ReadyToRun (R2R)	AOT-compiled at *publish* time, with IL kept as a fallback; the JIT can still re-optimise. Speeds up startup.
Tiered compilation (default on)	tier-0 quickly, then tier-1 optimised for hot methods — so a method is compiled *more than once*
Native AOT	compiled fully ahead of time; **no JIT ships**, so no runtime code generation at all
Mono interpreter	IL is interpreted, not compiled

Tiered compilation is worth a mention when discussing benchmarks: it is why warm-up loops matter, and why a naive first measurement is always wrong.

3.4 The evaluation stack

IL has no registers you can name. Every instruction works on an **evaluation stack**.

```
instruction           stack (top on the right)      effect
──────────────────────────────────────────────────────────────────────────────
                      [ ]                           start: empty
ldarg.0               [ a ]                         push argument 0
ldarg.1               [ a, b ]                      push argument 1
add                   [ a+b ]                       pop 2, push their sum
ret                   [ ]                           pop the return value, return
```

Two rules govern everything, and nearly every Emit bug is one of them:

1. **Balance.** At `ret`, the stack must hold exactly the return value (or nothing, for `void`). Leaving something behind, or taking too much, is invalid IL.

2 **Types.** Operand types must suit the opcode. `add` on an `int` and a string reference is invalid — see §8 for what actually happens, which is worse than an error.

A useful habit: write the expected stack state as a comment after every `Emit` call, as the examples in this package do. It turns an invisible bug into a visible one.

3.5 The instruction set the syllabus requires

`ldarg` — load argument

Pushes a method argument.

Instruction	Loads
`ldarg.0`	argument 0
`ldarg.1`	argument 1
`ldarg.2`, `ldarg.3`	arguments 2, 3
`ldarg.s <n>`	argument *n* (n ≤ 255)
`ldarg <n>`	any argument

> ***The numbering trap.*** *For a **static** method, `ldarg.0` is the first real argument. For an **instance** method, `ldarg.0` is `this`, and the first real argument is `ldarg.1`. Getting this wrong is the most common beginner Emit bug. A `DynamicMethod` declared as in §4 is static; a `MethodBuilder` instance method is not. Compare the IL in example 05 (static) with example 06's constructor and `Scale` method (instance) — the difference is visible.*

`ldc.i4` — load constant (int32)

Instruction	Bytes	Range
`ldc.i4.0` … `ldc.i4.8`	1	0–8
`ldc.i4.m1`	1	–1
`ldc.i4.s <sbyte>`	2	–128…127
`ldc.i4 <int32>`	5	any 32-bit value

All push the same thing; they differ only in encoding size. (`ldc.i8`, `ldc.r4`, `ldc.r8` exist for `long`, `float`, `double`.)

`add` and `mul` — arithmetic

Pop two operands, push one result.

```
before: [ 6, 7 ]   →   mul   →   after: [ 42 ]
```

`sub`, `div`, `rem` follow the same pattern. Variants: `add.ovf` throws on overflow (what C#'s `checked` emits); `div.un` is unsigned.

`call` and `callvirt` — invoke a method

Pops the arguments (and the receiver, for instance methods), pushes the return value if there is one.

	`call`	`callvirt`
Target decided	now, at emit time	at run time, from the object's real type
Null check	no	**yes** — throws `NullReferenceException`
Used for	static and non-virtual instance methods	virtual and interface methods

`newobj` — allocate and construct

One instruction that allocates the object, runs the constructor with the popped arguments, and pushes the reference.

```
before: [ 3, 4 ]   →   newobj Point::.ctor(int,int)   →   after: [ Point ]
```

This single instruction is what `new`, `Activator.CreateInstance`, a DI container and a deserializer all ultimately have to perform. Emitting it once and caching the delegate is how containers avoid paying reflection costs per resolve.

`ret` — return

Pops the return value (if the signature has one) and returns. Every path must end in `ret` (or `throw`, or an unconditional branch to one).

Others you will meet immediately

Instruction	Does
`ldfld` / `stfld`	load / store an instance field
`ldsfld` / `stsfld`	load / store a static field
`castclass T`	checked reference cast; throws `InvalidCastException`
`isinst T`	like `as` — pushes null instead of throwing
`box T` / `unbox.any T`	value type ↔ heap object
`ldstr "text"`	push a string literal
`ldnull`	push null
`br`, `brtrue`, `brfalse`, `beq`, `blt`	branches (used with `ILGenerator.DefineLabel`)
`ldloc` / `stloc`	load / store a local (declared with `DeclareLocal`)
`nop`	nothing — a placeholder

3.6 Audience questions

Q: Is IL the same as machine code? No. IL is a portable, CPU-independent, stack-based instruction set, and it carries full type information. Machine code is CPU-specific, register-based, and has no type information. The JIT translates the first into the second.

Q: Can I read the IL of my own C#? Yes, and you should before writing any: ILSpy, `ildasm`, or sharplab.io. Write the C# you want, read its IL, then emit that. Far more reliable than reasoning from first principles.

Q: Is IL human-writable? `ilasm` accepts IL source, and people do write it, but it is a specialised skill. `ILGenerator` is the normal route from a .NET program.

Q: Why a stack machine and not registers? A stack machine is simpler to verify and to target from a compiler, and it does not commit to a register count — leaving the JIT free to allocate real registers for the actual CPU.

3.7 Documentation

- `OpCodes` — every instruction — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.opcodes
- Managed code — https://learn.microsoft.com/en-us/dotnet/standard/managed-code

4. Lightweight Code Generation — `DynamicMethod`

4.1 Definition

A `DynamicMethod` is a single method with **no declaring type**, created and compiled at run time. It is the cheapest form of runtime code generation.

4.2 The four APIs

API	Role
`new DynamicMethod(name, returnType, parameterTypes)`	declare the signature
`GetILGenerator()`	get the writer for its body
`ILGenerator.Emit(OpCode, …)`	append one instruction
`CreateDelegate<T>()`	finish it and get something callable

4.3 The canonical example, instruction by instruction

```
var method = new DynamicMethod(
    name: "Add",                                // for stack traces and debugging only
    returnType: typeof(int),
    parameterTypes: [typeof(int), typeof(int)]);

ILGenerator il = method.GetILGenerator();

il.Emit(OpCodes.Ldarg_0);
il.Emit(OpCodes.Ldarg_1);
il.Emit(OpCodes.Add);
il.Emit(OpCodes.Ret);

var add = method.CreateDelegate<Func<int, int, int>>();
```

Verified output, with the stack state after each instruction:

```
Signature: int Add(int a, int b)

  IL                | evaluation stack after the instruction (top on the right)
  ------------------+-----------------------------------------------------------
  ldarg.0           | [ a ]                 push argument 0
  ldarg.1           | [ a, b ]              push argument 1
  add               | [ a+b ]               pop two, push their sum
  ret               | [ ]                   pop the return value and return

add(2, 3)    = 5
add(40, 2)   = 42
add(-7, 7)   = 0
```

Line by line:

Line	Why
`name: "Add"`	Cosmetic. Appears in stack traces; has no semantic effect.
`returnType`, `parameterTypes`	The signature. The IL you emit must match it, and `CreateDelegate`'s type argument must match it too.
`GetILGenerator()`	The body writer. Calling it twice returns the same generator.
`Ldarg_0`, `Ldarg_1`	The two arguments. This method is **static**, so argument 0 is the first real argument.
`Add`	Pops two, pushes one. The stack goes from depth 2 to depth 1.
`Ret`	Pops the `int` return value. Stack depth 0. Balanced.

Line	Why
`CreateDelegate<Func<int,int,int>>()`	Hands the IL to the runtime and returns a delegate. From here the per-call cost is a normal delegate invocation.

4.4 Constants

Target: `int f(int x) => (x * 2) + 10;`

```
IL                | stack after
------------------+---------------------
ldarg.0           | [ x ]
ldc.i4.2          | [ x, 2 ]        one-byte form for small constants
mul               | [ x*2 ]
ldc.i4.s 10       | [ x*2, 10 ]     short form, operand fits in a signed byte
add               | [ x*2+10 ]
ret               | [ ]

f(0) = 10
f(1) = 12
f(16) = 42
f(100) = 210
```

Note `il.Emit(OpCodes.Ldc_I4_S, (sbyte)10)` — the cast matters, because `Emit` has overloads per operand type and picking the wrong one produces wrong IL.

4.5 Calling an existing method

Target: `int f(int a, int b) => Math.Max(a, b) * 10;`

```
IL                             | stack after
-------------------------------+----------------------
ldarg.0                        | [ a ]
ldarg.1                        | [ a, b ]
call int32 Math::Max(int32,int32)| [ max ]        popped 2 args, pushed 1 result
ldc.i4.s 10                    | [ max, 10 ]
mul                            | [ max*10 ]
ret                            | [ ]

f(3, 9)  = 90
f(42, 1) = 420
```

This is the most valuable technique in practice: **keep the emitted IL tiny and `call` into normal C# helpers for anything non-trivial.** Debuggable C# does the work; the generated method is just the glue.

4.6 Constructing an object

Target: `object f(int x, int y) => new Point(x, y);`

```
IL                          | stack after
----------------------------+----------------------
ldarg.0                     | [ x ]
ldarg.1                     | [ x, y ]
newobj Point::.ctor(int,int)| [ Point ]      popped 2 args, pushed the new reference
ret                         | [ ]

f(3, 4) = Point(3, 4)
f(0, 0) = Point(0, 0)
```

4.7 The realistic use — a generated accessor

Target: `object f(object o) => (object)((Employee)o).Salary;`

```
  ldarg.0                     | [ o ]           the untyped instance
  castclass Employee          | [ Employee ]    checked cast, throws if wrong type
  callvirt get_Salary()       | [ decimal ]     run the property getter
  box decimal                 | [ object ]      value type -> heap object
  ret                         | [ ]

  emitted getter  -> 94500
  PropertyInfo    -> 94500
  direct property -> 94500
```

`object → object` is the signature a serializer or mapper needs, because it does not know the types at compile time. It is also why it is slower than the typed version (§2.3, rows 5 and 6): the cast and the box are real work on every call.

4.8 Where the "runtime" in runtime code generation really shows

The opcode is **data**, so one factory can build all four operators from a string that did not exist at compile time:

```
public static Func<int, int, int> Build(string op)
{
    OpCode opcode = op switch
    {
        "+" => OpCodes.Add,
        "-" => OpCodes.Sub,
        "*" => OpCodes.Mul,
        "/" => OpCodes.Div,
        _ => throw new ArgumentException("Unsupported operator: " + op, nameof(op))
    };

    var method = new DynamicMethod("Op" + op, typeof(int), [typeof(int), typeof(int)]);
    ILGenerator il = method.GetILGenerator();
    il.Emit(OpCodes.Ldarg_0);
    il.Emit(OpCodes.Ldarg_1);
    il.Emit(opcode);
    il.Emit(OpCodes.Ret);

    return method.CreateDelegate<Func<int, int, int>>();
}
```

Verified:

```
  Build("+")(84, 2) = 86
  Build("-")(84, 2) = 82
  Build("*")(84, 2) = 168
  Build("/")(84, 2) = 42
```

That is the honest shape of the technique: **a small interpreter of your data, turned into machine code once, then reused.** A `switch` over the operator would be simpler here — the technique pays off when the "data" is a whole expression, a mapping configuration, or a type's property list.

4.9 Advantages and limitations

Advantages: cheapest form; collectible (garbage-collected when unreferenced); can be given restricted skip-visibility to reach private members; near-direct speed once compiled.

Limitations: one method only — no fields, no types, no second method; cannot be saved to disk; unverified IL (§8); poor debugging; needs dynamic code support.

4.10 Common mistakes

1. `ldarg` **numbering** on instance vs static methods (§3.5).
2. **Forgetting** `ret`**.** `InvalidProgramException` on first call.
3. **Unbalanced stack.** Same symptom, harder to spot.
4. **Mismatched delegate type** in `CreateDelegate`.
5. **Wrong** `Emit` **overload** — `Ldc_I4_S` wants an `sbyte`.

6 **Generating per call** instead of caching the delegate. This single mistake can make the "optimisation" slower than the reflection it replaced.

7 `call` **where** `callvirt` **is needed** — a virtual method invoked non-virtually silently calls the wrong override.

4.11 Audience questions

Q: What is the difference between `DynamicMethod` **and** `MethodBuilder`**?** `DynamicMethod` is a standalone method with no declaring type — cheap, collectible, not savable, and it cannot define fields or types. `MethodBuilder` is a method *on a* `TypeBuilder`, so you can build a whole type with state and several methods, and it can be persisted. Use `DynamicMethod` for accessors and converters; `MethodBuilder` when you need a **type** (proxies, mocks).

Q: Can `DynamicMethod` **access private members?** Yes, with the `restrictedSkipVisibility` constructor overload. This is how serializers set private backing fields. It is a capability, not a security hole — as Part 2 §6.4 notes, private was never a security boundary.

Q: Are dynamic methods garbage-collected? Yes — that is a real advantage over a non-collectible dynamic assembly. When no delegate or reference remains, it can be collected.

Q: Should I use this or an expression tree? Expression tree, almost always. Same speed class, type-checked as you build, printable, and it fails loudly rather than silently (§8). Part 4 §8 covers them. Use `DynamicMethod` only when you need IL that expression trees cannot express.

4.12 Documentation

- `DynamicMethod` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.dynamicmethod
- `ILGenerator` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.ilgenerator

---

5. Full Assembly Generation

5.1 The builder hierarchy

```
  AssemblyBuilder   one logical assembly, with a name and a version
      ■
      ■■■ ModuleBuilder    one physical container of types (normally exactly one)
              ■
              ■■■ TypeBuilder     one class, struct, interface or enum
                      ■
                      ■■■ FieldBuilder        a field
                      ■■■ ConstructorBuilder  a constructor
                      ■■■ MethodBuilder       a method (holds an ILGenerator)
                      ■■■ PropertyBuilder     a property (needs get/set methods)
```

This is the Part 2 metadata hierarchy, built in reverse. Part 2 **read** Assembly → Type → Member; here you **write** the same structure.

One rule governs it: `TypeBuilder.CreateType()` "bakes" the type. Before that call the type does not exist and cannot be instantiated. After it, nothing more can be added.

5.2 What .NET 10 actually supports — verified

This is the part where older documentation will mislead you. Verified by running example 06:

```
AssemblyBuilderAccess members that exist on this runtime:
  AssemblyBuilderAccess.Run
  AssemblyBuilderAccess.RunAndCollect

Does the runtime AssemblyBuilder have a Save(string) method? no
Concrete runtime builder type: RuntimeAssemblyBuilder
AssemblyBuilder is declared in: System.Private.CoreLib
```

> `AssemblyBuilderAccess` ***on .NET 10 has only*** `Run` ***and*** `RunAndCollect`**.** `Save` *and* `RunAndSave` *were* ***.NET Framework*** *members. They do not exist. Code like this does* ***not compile*** *on modern .NET:* ``csharp // .NET Framework only — DOES NOT COMPILE on .NET Core / .NET 5+ var ab = AssemblyBuilder.DefineDynamicAssembly(name, AssemblyBuilderAccess.RunAndSave); ab.Save("Output.dll"); ``

Member	Meaning
`Run`	in-memory, executable, **not** collectible
`RunAndCollect`	in-memory, executable, **collectible** (can be unloaded)

Use `RunAndCollect` if the generated code's lifetime is bounded — otherwise a `Run` assembly lives for the life of the process. Note that **caching a generated** `Type` **keeps its whole load context alive**, exactly as in Part 2 §9.6.

5.3 Building a complete type, stage by stage

Goal — generate the equivalent of this C#, without a compiler:

```
public class Scaler : IScaler
{
    private readonly int _factor;
    public Scaler(int factor) { _factor = factor; }
    public int Factor => _factor;
    public int Scale(int value) => _factor * value;
}
```

Stage 1 — the assembly

```
var assemblyName = new AssemblyName("Metaprogramming.Generated") { Version = new Version(1, 0, 0, 0) };
AssemblyBuilder assemblyBuilder = AssemblyBuilder.DefineDynamicAssembly(assemblyName,
    AssemblyBuilderAccess.Run);
```

```
Stage 1  AssemblyBuilder.DefineDynamicAssembly("Metaprogramming.Generated", Run)
         IsDynamic = True, Location = "" (empty: it is not a file)
```

Stage 2 — the module

```
ModuleBuilder moduleBuilder = assemblyBuilder.DefineDynamicModule("MainModule");
```

Stage 3 — the type, declaring the interface

```
TypeBuilder typeBuilder = moduleBuilder.DefineType(
    name: "Metaprogramming.Generated.Scaler",
    attr: TypeAttributes.Public | TypeAttributes.Class | TypeAttributes.Sealed | TypeAttributes.BeforeFieldInit,
    parent: typeof(object));

typeBuilder.AddInterfaceImplementation(typeof(IScaler));
```

`AddInterfaceImplementation` is the key design move — see §6.

Stage 4 — the field

```
FieldBuilder factorField = typeBuilder.DefineField(
    "_factor", typeof(int), FieldAttributes.Private | FieldAttributes.InitOnly);
```

`FieldAttributes.InitOnly` **is** `readonly`.

Stage 5 — the constructor

```
ConstructorBuilder constructorBuilder = typeBuilder.DefineConstructor(
    MethodAttributes.Public | MethodAttributes.HideBySig
        | MethodAttributes.SpecialName | MethodAttributes.RTSpecialName,
    CallingConventions.Standard,
    [typeof(int)]);

constructorBuilder.DefineParameter(1, ParameterAttributes.None, "factor");

ILGenerator ctorIl = constructorBuilder.GetILGenerator();
ConstructorInfo objectCtor = typeof(object).GetConstructor(Type.EmptyTypes)!;

ctorIl.Emit(OpCodes.Ldarg_0);             // [ this ]
ctorIl.Emit(OpCodes.Call, objectCtor);    // [ ]          base.ctor()
ctorIl.Emit(OpCodes.Ldarg_0);             // [ this ]
ctorIl.Emit(OpCodes.Ldarg_1);             // [ this, factor ]
ctorIl.Emit(OpCodes.Stfld, factorField);  // [ ]          this._factor = factor
ctorIl.Emit(OpCodes.Ret);                 // [ ]
```

Two things that bite:

- **Every constructor must chain to a base constructor.** Omitting `call object::.ctor()` produces a type that fails in confusing ways. C# does this for you invisibly; here it is your job.
- `ldarg.0` **is** `this`**.** The `factor` parameter is `ldarg.1`. Contrast with §4.3, where `ldarg.0` was the first argument because that method was static.

`SpecialName | RTSpecialName` are required for constructors — they tell the runtime this is a constructor, not a method called `.ctor`.

Stage 6 — the interface method

```
MethodBuilder scaleMethod = typeBuilder.DefineMethod(
    "Scale",
    MethodAttributes.Public | MethodAttributes.Virtual | MethodAttributes.Final
        | MethodAttributes.HideBySig | MethodAttributes.NewSlot,
    typeof(int),
    [typeof(int)]);

scaleMethod.DefineParameter(1, ParameterAttributes.None, "value");

ILGenerator scaleIl = scaleMethod.GetILGenerator();
scaleIl.Emit(OpCodes.Ldarg_0);              // [ this ]
scaleIl.Emit(OpCodes.Ldfld, factorField);   // [ _factor ]
scaleIl.Emit(OpCodes.Ldarg_1);              // [ _factor, value ]
scaleIl.Emit(OpCodes.Mul);                  // [ _factor*value ]
scaleIl.Emit(OpCodes.Ret);                  // [ ]

typeBuilder.DefineMethodOverride(scaleMethod, typeof(IScaler).GetMethod(nameof(IScaler.Scale))!);
```

An interface implementation **must be** `Virtual`. `DefineMethodOverride` binds your method to the interface slot explicitly — the runtime would usually infer it from a matching name and signature, but being explicit is safer and is *required* when the names differ.

Stage 7 — the property

```
MethodBuilder factorGetter = typeBuilder.DefineMethod(
    "get_Factor",
    MethodAttributes.Public | MethodAttributes.HideBySig | MethodAttributes.SpecialName,
    typeof(int), Type.EmptyTypes);

ILGenerator getterIl = factorGetter.GetILGenerator();
getterIl.Emit(OpCodes.Ldarg_0);
getterIl.Emit(OpCodes.Ldfld, factorField);
getterIl.Emit(OpCodes.Ret);

PropertyBuilder factorProperty = typeBuilder.DefineProperty(
    "Factor", PropertyAttributes.HasDefault, typeof(int), null);
factorProperty.SetGetMethod(factorGetter);
```

Exactly as Part 2 §2.4 said: **a property is metadata binding a name to accessor methods.** You write the method; the property just points at it.

Stage 8 — bake it

```
Type created = typeBuilder.CreateType();
```

```
Stage 8  CreateType()  ->  Metaprogramming.Generated.Scaler
```

5.4 Saving a generated assembly on .NET 10 — `PersistedAssemblyBuilder`

Saving **is** supported, through a different class. Verified end to end:

```
var persisted = new PersistedAssemblyBuilder(
    new AssemblyName("Metaprogramming.Saved") { Version = new Version(1, 0, 0, 0) },
    coreAssembly: typeof(object).Assembly);

ModuleBuilder module = persisted.DefineDynamicModule("Metaprogramming.Saved.dll");
TypeBuilder typeBuilder = module.DefineType("Metaprogramming.Saved.MathHelpers",
    TypeAttributes.Public | TypeAttributes.Class | TypeAttributes.Abstract | TypeAttributes.Sealed);

MethodBuilder add = typeBuilder.DefineMethod(
    "Add", MethodAttributes.Public | MethodAttributes.Static, typeof(int), [typeof(int), typeof(int)]);
ILGenerator addIl = add.GetILGenerator();
addIl.Emit(OpCodes.Ldarg_0);
addIl.Emit(OpCodes.Ldarg_1);
addIl.Emit(OpCodes.Add);
addIl.Emit(OpCodes.Ret);

typeBuilder.CreateType();
persisted.Save(outputPath);
```

Verified:

```
Wrote a real assembly to disk:
  path  : ...\06-ReflectionEmit\bin\Debug\net10.0\Metaprogramming.Saved.dll
  exists: True
  bytes : 2048

Loaded that file back in and invoked it:
  assembly  : Metaprogramming.Saved v1.0.0.0
  IsDynamic : False  (false - it came from a file)
  Add(20,22)= 42
  Square(9) = 81
```

A real 2 KB DLL, loaded with `Assembly.LoadFrom` and invoked.

Details that matter:

- It lives in the `System.Reflection.Emit` assembly, part of the **shared framework** — **no package reference needed** on `net10.0`.
- `Save` overloads: `Save(string)` and `Save(Stream)`. There is also `GenerateMetadata(...)` for building a PE file yourself (to add a strong name or debug directory).
- **It is for WRITING, not running.** Do not expect to execute its types directly; save, then load the file — or use the `Run` builder instead.
- `Save` needs complete IL: call `CreateType()` on **every** `TypeBuilder` first.
- Note `TypeAttributes.Abstract | Sealed` — that pair is how a `static class` is represented in metadata.

Why persistence is worth knowing about — it turns IL debugging from guesswork into inspection:

1. Open the result in **ILSpy** or `ildasm` and **read the IL you emitted**.
2. Run **ILVerify** over it to catch invalid IL properly, instead of waiting for `InvalidProgramException` (§8).
3. Build-time tools can emit an assembly for later consumption.

5.5 Documentation

- `AssemblyBuilder` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilder
- `AssemblyBuilderAccess` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilderaccess
- `PersistedAssemblyBuilder` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.persistedassemblybuilder
- `TypeBuilder` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.typebuilder
- `MethodBuilder` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.methodbuilder

---

6. Using the Generated Type — The Pattern That Makes This Practical

6.1 Two ways to call it

```
object? instance = Activator.CreateInstance(generated, 7);

// (a) Reflectively — works, but slow per call.
MethodInfo scale = generated.GetMethod("Scale")!;
scale.Invoke(instance, [6]);

// (b) Through the interface — ordinary, fast interface dispatch.
var scaler = (IScaler)instance!;
scaler.Scale(6);
```

Verified:

```
Activator.CreateInstance(generated, 7) -> Metaprogramming.Generated.Scaler

Reflective invocation:
  Scale(6)  = 42
  Scale(10) = 70
  Factor    = 7

Through the IScaler interface - no reflection at the call site:
  scaler.Scale(6)  = 42
  scaler.Scale(10) = 70

Each instance keeps its own _factor, as a real field should:
  new Scaler(2).Scale(5) = 10
  new Scaler(3).Scale(5) = 15
  new Scaler(100).Scale(5) = 500
```

6.2 The pattern

> ***Generate a type that implements an interface you already know, then talk to it through that interface.***
> *Reflection is used **once**, at creation. Never on the hot path.*

This is not a trick; it is the design behind every real user of `TypeBuilder`:

Library	Generates a type implementing	You then use
Moq, NSubstitute	your interface	your interface
Castle DynamicProxy	your interface (or a subclass)	your interface
`DispatchProxy` (Part 4 §6)	your interface	your interface
EF Core lazy-loading proxies	your entity type	your entity type

6.3 The generated type is indistinguishable from a compiled one

Part 2's reflection code works on it unchanged:

```
Name            : Metaprogramming.Generated.Scaler
Assembly        : Metaprogramming.Generated (IsDynamic=True)
Module          : <In Memory Module>
BaseType        : Object
Interfaces      : IScaler
IsSealed        : True

Constructors:
  .ctor(Int32 factor)

Methods (declared only):
  Int32 Scale(Int32 value)  [virtual]
  Int32 get_Factor()

Properties:
  Int32 Factor { get; }

Fields (including private):
  private readonly Int32 _factor
```

Generated metadata and compiled metadata are **the same kind of thing**. That is the closing idea of Part 3: Part 2 read these tables; Part 3 wrote them.

---

7. Practical Example — Dynamic Arithmetic

Covered in §4.8. The summary of what distinguishes a dynamic method from a normal C# method:

	Normal C# method	`DynamicMethod`
Exists	from compile time	from when you create it
Declaring type	a real type	**none**
Source	your `.cs` file	your `Emit` calls
Type-checked	by the compiler	not reliably (§8)
In metadata	yes, in the assembly	no — it is not in any assembly you can save
Collectible	no	**yes**
Debuggable	fully	barely
Speed once JIT-ed	full	**the same**
Can be created from run-time data	no	**yes** — the whole point

---

8. What Happens When the IL Is Wrong

This section matters more than any other in Part 3, because the failure modes are **asymmetric** and the dangerous one is silent.

8.1 Case A — a missing `ret`

```
var missingRet = new DynamicMethod("MissingRet", typeof(int), [typeof(int), typeof(int)]);
ILGenerator il1 = missingRet.GetILGenerator();
il1.Emit(OpCodes.Ldarg_0);
il1.Emit(OpCodes.Ldarg_1);
il1.Emit(OpCodes.Add);
// deliberately NO il1.Emit(OpCodes.Ret);
```

Verified:

```
Case A: a method that never returns (no 'ret' at the end).
  CreateDelegate succeeded - nothing has been verified yet.
  Calling it...
  InvalidProgramException on FIRST CALL: Common Language Runtime detected an invalid program.
```

Note **when** it failed: not at `Emit`, not at `CreateDelegate`, but on the **first call** — because that is when the JIT compiles the method. Your bug surfaces far from the line that caused it.

8.2 Case B — a type mismatch, and this one is worse

```
var typeMismatch = new DynamicMethod("TypeMismatch", typeof(int), [typeof(int)]);
ILGenerator il2 = typeMismatch.GetILGenerator();
il2.Emit(OpCodes.Ldarg_0);
il2.Emit(OpCodes.Ldstr, "not a number");   // push a STRING
il2.Emit(OpCodes.Add);                     // add an int and a string reference?
il2.Emit(OpCodes.Ret);
```

Verified:

```
Case B: a stack TYPE mismatch - 'add' on an int32 and a string reference.
This one is worse than an exception. Watch what actually happens.
  No exception. broken(1) returned 299021057, and again 299021057.
  That number is the string's managed reference added to 1 as an integer.
  It is meaningless, it is not stable across runs, and nothing warned us.
```

No exception. A plausible-looking number. Nothing warned us.

> ***IL emitted by a fully-trusted*** `DynamicMethod` ***is not type-verified.*** *A mistake can silently produce garbage instead of failing. Memory-unsafe IL in this position can corrupt process state rather than throw.*

That asymmetry is the real reason to be cautious with `Emit`: your safety net is thinner than the C# compiler's, and in some cases there is no net at all. Compare with expression trees (Part 4 §8), where `Expression.Add(intNode, stringNode)` throws **immediately**, at build time of the tree.

8.3 How to debug emitted IL in practice

In order of usefulness:

1. **Write the C# first, read its IL.** Compile the method you want in a scratch project, open it in ILSpy or sharplab.io, and emit *that*. Never reason from first principles.
2. **Track the stack depth as you emit.** Keep a counter, or write the expected stack state in a comment on every line (as these examples do). Assert it returns to zero.
3. **Persist and verify.** Generate with `PersistedAssemblyBuilder`, `Save` it, and run **ILVerify** or `ildasm` over the file. This is the only way to get proper IL validation, and it is the strongest argument for knowing §5.4 exists.

4. **Keep emitted methods tiny.** Put real logic in normal C# and `call` it (§4.5). The less IL you write, the less IL can be wrong.
5. **Prefer expression trees.** They are type-checked as you build them.

---

9. Limits and Tradeoffs

9.1 `DynamicMethod` vs the builders

	`DynamicMethod`	`AssemblyBuilder`/`TypeBuilder`/`MethodBuilder`
Creates	one method	a whole **type**
Fields, properties, several methods	no	yes
Implements an interface	no	**yes**
Collectible	yes	only with `RunAndCollect`
Savable	no	yes, via `PersistedAssemblyBuilder`
Ceremony	~5 lines	~60 lines
Right for	accessors, converters, mappers, serializer fast paths	proxies, mocks, generated models

9.2 Costs you accept with either

Cost	Detail
Native AOT	Unavailable. `IsDynamicCodeSupported` is `false` and these APIs throw `PlatformNotSupportedException`.
Trimming	The trimmer cannot see what your IL will reference, or types that do not exist yet. Annotate with `DynamicallyAccessedMembers`, or do not trim.
First-use latency	Generating plus JIT-ing costs real milliseconds. Cache the generated `Type` or delegate; never regenerate per request.
Memory	A `Run` assembly is **never** collected. A cached generated `Type` keeps its whole context alive.
Debuggability	No source, limited breakpoints, opaque stack frames.
Maintainability	IL is write-mostly. Expect the next developer to fear it — and to be right to.
Safety	§8. Some invalid IL is not caught at all.

9.3 The decision order

Work down this list. Stop at the first one that works.

```
1. Plain C#, generics, or a cached delegate from MethodInfo.CreateDelegate
     -> covers the large majority of cases, at emit-level speed (§2.3)

2. Expression trees  (Part 4 §8)
     -> same speed class, readable, type-checked, printable
     -> can be BUILT and READ under AOT; only Compile() needs the JIT

3. DispatchProxy  (Part 4 §6)
     -> if you only need to intercept an INTERFACE

4. A Roslyn source generator
     -> if the shape is known at BUILD time: zero runtime cost, AOT-safe

5. Reflection.Emit
     -> only when the shape is known ONLY at run time AND you need a TYPE
```

Most people who reach for step 5 needed step 1 or 2.

Part 3 summary

1. Runtime code generation creates new IL while the process runs — the write half of metaprogramming.
2. The one good reason is **performance when the shape is only known at run time**: pay reflection once, then call generated code. The other is needing a **type** that did not exist.
3. It requires `RuntimeFeature.IsDynamicCodeSupported`, which is `false` **under Native AOT**.
4. IL is a **stack machine**. Balance the stack; match the operand types.
5. `ldarg.0` is the first argument in a **static** method and `this` in an instance method. This is the most common beginner bug.
6. `DynamicMethod` → one method, cheap, collectible, not savable. `TypeBuilder` → whole types; call `CreateType()` to bake.
7. On .NET 10, `AssemblyBuilderAccess` has only `Run` and `RunAndCollect`. Saving is done with `PersistedAssemblyBuilder` — verified here by writing, reloading and invoking a real DLL.
8. **Invalid IL may not be caught.** A missing `ret` throws on first call; a type mismatch silently returned garbage.
9. The practical pattern: **generate a type implementing a known interface, then use the interface.**
10. Measured: a cached delegate is as fast as emitted IL. **Try `CreateDelegate` before writing any IL.**

Part 4 — Real-World Applications

Session time: 9 minutes (slides 24–27) Runnable examples: `07-DependencyInjection`, `08-ExpressionTrees`, `09-Serialization`, `10-ASPNetCoreMVC`, `11-EntityFrameworkCore`, `12-DynamicProxies`

> ***The one idea that unifies this entire part: DISCOVER ONCE, CACHE, THEN EXECUTE COMPILED CODE.*** *Every framework below reflects at **startup** or **first use**, builds a fast plan, caches it, and never reflects again on the hot path. This is the answer to "isn't reflection slow?" and it is the single most transferable design lesson in the session.*

1. ASP.NET Core MVC

1.1 What the framework does for you

You write this, and never register it anywhere:

```
[ApiController]
[Route("api/reports")]
public sealed class ReportsController(IReportStore store) : ControllerBase
{
    [HttpGet]
    public ActionResult<IEnumerable<Report>> GetAll() => Ok(store.GetAll());

    [HttpGet("{id:int}")]
    public ActionResult<Report> GetById(int id) { /* ... */ }

    [HttpGet("search")]
    public ActionResult<IEnumerable<Report>> Search([FromQuery] string term, [FromQuery] int take = 5)
        => Ok(store.Search(term, take));

    [HttpPost]
    public ActionResult<Report> Create([FromBody] CreateReportRequest request) { /* ... */ }

    [NonAction]
    public string InternalHelper() => "not routable";
}
```

`Program.cs` contains one relevant line, which names no controller:

```
builder.Services.AddControllers();
```

1.2 Application parts — the search path

Verified (`10-ASPNetCoreMVC`):

```
ApplicationPartManager.ApplicationParts:
  AspNetCoreMvcDemo            (AssemblyPart)
```

An **application part** is a source of controllers. By default the list holds the entry assembly plus any assembly marked with `[ApplicationPart]`. Add a part and MVC will find controllers in another DLL — which is how a plugin can contribute endpoints, joining Part 2 §9 to this section.

1.3 Controller discovery

MVC's own mechanism, used directly in the example:

```
var controllerFeature = new ControllerFeature();
partManager.PopulateFeature(controllerFeature);
```

Verified:

```
Controller types found by reflecting over the application parts:
  Metaprogramming.AspNetCoreMvcDemo.Controllers.MetadataController
  Metaprogramming.AspNetCoreMvcDemo.Controllers.ReportsController
```

The rule MVC applies to a public class, in order:

1. skip it if it is abstract, generic, or marked `[NonController]`;
2. accept it if it carries `[Controller]` or `[ApiController]`;
3. otherwise accept it if its **name ends with** `"Controller"`.

> ***Inheriting from*** `ControllerBase` ***is NOT what makes a class a controller.*** *The name suffix or the attribute is.* `ControllerBase` *only gives you helpers like* `Ok()` *and* `NotFound()`*. This surprises people, and you can demonstrate it by renaming a class.*

1.4 Action discovery and the `ActionDescriptor` table

```
var actionProvider = app.Services.GetRequiredService<IActionDescriptorCollectionProvider>();
```

Verified:

```
IActionDescriptorCollectionProvider gives the routing table, version 0:

  METHOD  ROUTE TEMPLATE             CONTROLLER ACTION     PARAMETERS
  ------------------------------------------------------------------------------------
  GET     api/metadata/actions       Metadata   Actions
  GET     api/reports                Reports    GetAll
  POST    api/reports                Reports    Create     CreateReportRequest request
  GET     api/reports/search         Reports    Search     String term, Int32 take
  GET     api/reports/{id:int}       Reports    GetById    Int32 id
```

Where each column came from:

Column	Source
METHOD	the `[HttpGet]` / `[HttpPost]` attribute
ROUTE	`[Route]` on the class combined with the template on the method
CONTROLLER	the type name, minus the `Controller` suffix
ACTION	the reflected `MethodInfo.Name`
PARAMETERS	reflected `ParameterInfo`, including the `[FromQuery]`/`[FromBody]` binding source

Note `InternalHelper` is **absent** — `[NonAction]` excluded it. A public method on a controller is an action **by default**; opting out is explicit.

The table is a **cached data structure**. The `Version` property exists so that components can detect changes if parts are added at run time.

1.5 Startup-time vs request-time — the point people get wrong

This distinction is the heart of slide 24.

Done ONCE, at startup	Done PER REQUEST
scan application parts for controller types ← *reflection*	match the URL against the prepared route table — **no type scanning**
read routing, filter and binding attributes ← *reflection*	create the controller through DI
build the `ActionDescriptor` collection and route table	bind parameters using the prepared binders
build a cached activator and model binders per action	invoke the action through a **cached delegate**, not `MethodInfo.Invoke`

> *So* ***"MVC uses reflection, therefore MVC is slow per request" is wrong.*** *The reflection is amortised into startup.*

It is also why **startup time** matters for serverless and why minimal APIs plus source generation exist: to move even that startup reflection to build time.

1.6 The payoff — an app that describes itself

Because the API surface was *discovered* from metadata, the app can report it:

```
[HttpGet("actions")]
public ActionResult<IEnumerable<object>> Actions() =>
    Ok(provider.ActionDescriptors.Items.OfType<ControllerActionDescriptor>().Select(a => new
    {
        controller = a.ControllerName,
        action = a.ActionName,
        route = a.AttributeRouteInfo?.Template ?? "(conventional)",
        signature = a.MethodInfo.Name + "(" + string.Join(", ",
            a.MethodInfo.GetParameters().Select(p => p.ParameterType.Name + " " + p.Name)) + ")"
    }));
```

Verified over real HTTP:

```
GET /api/metadata/actions
  -> 200 OK
  -> [{"controller":"Metadata","action":"Actions","route":"api/metadata/actions",
       "httpMethods":["GET"],"signature":"Actions()", ...}, ...]
```

This is exactly how Swagger / OpenAPI generation works. No separate spec file; the spec is derived from the code.

1.7 Request-time behaviour, verified

```
GET /api/reports/2                     -> 200  {id:int} bound "2" to an int
GET /api/reports/999                   -> 404  the ACTION ran and chose NotFound
GET /api/reports/not-an-int            -> 404  the ROUTE CONSTRAINT rejected it; no action ran
GET /api/reports/search?term=sales&take=2 -> 200  [FromQuery] binding + default value
POST /api/reports  (valid body)        -> 201  Location: http://127.0.0.1:5199/api/reports/4
POST /api/reports  (invalid body)      -> 400  ValidationProblemDetails
```

Three of those are worth dwelling on:

- `/999` **vs** `/not-an-int` are both 404 but for completely different reasons. The first reached the action; the second never selected one, because the `:int` constraint failed during routing. Distinguishing these two is a real debugging skill.
- `CreatedAtAction(nameof(GetById), ...)` produced a `Location` header by looking the route up **by action name** in the same descriptor table. Metadata read back for URL generation, not just dispatch.
- **The 400** needed no validation code from us:

```
{"title":"One or more validation errors occurred.","status":400,
 "errors":{"Owner":["The Owner field is not a valid e-mail address."],
           "Title":["The field Title must be a string with a minimum length of 3 and a maximum length of 60."]}}
```

`[ApiController]` reflected over `[Required]`, `[StringLength]` and `[EmailAddress]` on the request model and short-circuited the request.

1.8 Advantages and limitations

Advantages: no registration to forget; routes live next to the code they route to; tooling (Swagger, analyzers, route debugging) all works off the same metadata.

Limitations: convention-based magic is invisible to newcomers; a typo in a route template is a run-time 404, not a build error; startup cost grows with assembly size; "find all usages" of an action returns nothing, because nothing calls it directly.

1.9 Audience questions

Q: Does MVC use `MethodInfo.Invoke` for every request? No. It builds and caches a faster invoker per action at startup. The reflection is in the discovery phase.

Q: Can I add a controller at run time? You can add an `ApplicationPart` and notify the change provider, and there are APIs for it — but it is advanced and fragile. The common approach is to register all candidate assemblies at startup.

Q: Why do minimal APIs exist if MVC is this flexible? Less startup reflection, less ceremony, and they are far friendlier to AOT and source generation. MVC's convention-based discovery is exactly the part AOT dislikes.

1.10 Documentation

- Routing to controller actions — https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing
- Application parts — https://learn.microsoft.com/en-us/aspnet/core/mvc/advanced/app-parts

2. Dependency Injection

2.1 The core insight

> ***A constructor signature is a machine-readable dependency list.***

```
internal sealed class OrderService(
    IOrderRepository repository,
    IPricingPolicy pricingPolicy,
    IAuditLog auditLog,
    IClock clock)
```

Verified:

```
OrderService declares one constructor. Reflection reads it as a shopping list:

  needs IOrderRepository       (parameter 'repository')
  needs IPricingPolicy         (parameter 'pricingPolicy')
  needs IAuditLog              (parameter 'auditLog')
  needs IClock                 (parameter 'clock')
```

Nobody wrote a configuration file listing those. The dependency graph **is** the constructor signature, and the signature is already in metadata. That is the whole insight DI is built on.

2.2 Registration and resolution

```
var services = new ServiceCollection();
services.AddSingleton<IClock, SystemClock>();
services.AddSingleton<IAuditLog, InMemoryAuditLog>();
services.AddScoped<IPricingPolicy, StandardPricingPolicy>();
services.AddTransient<IOrderRepository, InMemoryOrderRepository>();
services.AddTransient<OrderService>();

using ServiceProvider provider = services.BuildServiceProvider();
var orderService = provider.GetRequiredService<OrderService>();
```

One request resolves the entire graph transitively:

```
provider.GetRequiredService<OrderService>() produced a full object graph:
  OrderService
    +-- InMemoryOrderRepository
    +-- StandardPricingPolicy
    +-- InMemoryAuditLog
    +-- SystemClock
```

`GetService` **vs** `GetRequiredService`**:**

```
  GetService<IMissingService>()         = null
  GetRequiredService<IMissingService>() threw InvalidOperationException:
    No service for type 'Metaprogramming.DependencyInjectionDemo.IMissingService' has been registered.
```

Prefer `GetRequiredService`. A null that becomes a `NullReferenceException` later is strictly worse than an immediate, named exception — the same argument as `Type.GetType(throwOnError: true)` in Part 2 §4.2.

2.3 Service lifetimes

```
  scope 1:
    singleton: 1, 1
    scoped   : 1, 1
    transient: 1, 2
  scope 2:
    singleton: 1, 1
    scoped   : 2, 2
    transient: 3, 4
```

Lifetime	Instances	In ASP.NET Core
Singleton	one per provider	one for the whole application
Scoped	one per scope	**one per HTTP request**
Transient	a new one every resolution	every injection point

The classic bug: a singleton depending on a scoped service. The scoped instance gets captured for the life of the process — a *captive dependency*. If that scoped service is a `DbContext`, you now have a non-thread-safe object shared across requests, which is a genuinely nasty production failure.

`ValidateScopes` turns it into an exception instead of a slow leak:

```
services.BuildServiceProvider(new ServiceProviderOptions { ValidateScopes = true });
```

Verified:

```
  Cannot consume scoped service 'ScopedCounter' from singleton 'CaptiveDependencyService'.
```

This is on by default in the ASP.NET Core Development environment. Also consider `ValidateOnBuild = true`, which surfaces missing registrations at startup rather than on the first request that needs them.

2.4 How the container picks a constructor

```
ReportBuilder has three constructors:
  .ctor()
  .ctor(IClock)
  .ctor(IClock, IAuditLog)

Case A: only IClock is registered.
  chosen -> .ctor(IClock)

Case B: IClock AND IAuditLog are registered.
  chosen -> .ctor(IClock, IAuditLog)
```

> ***The rule: pick the constructor with the MOST parameters that the container can satisfy completely.***

Note the consequence demonstrated above: **adding a registration changed which constructor runs.** A registration somewhere else in the application can silently alter how your class is built. This is a real source of confusion and an argument for having exactly one constructor.

Ambiguity is an error, not a guess:

```
  InvalidOperationException: Unable to activate type 'AmbiguousService'. The following constructors are
      ambiguous:
    Void .ctor(IClock)
    Void .ctor(IAuditLog)
```

Two satisfiable constructors where neither parameter set contains the other. Fix by removing one, or by marking the intended one `[ActivatorUtilitiesConstructor]`.

2.5 `ActivatorUtilities` — DI for unregistered types

`InvoiceGenerator` needs one service from DI **and** a runtime value the container cannot know:

```
internal sealed class InvoiceGenerator(IClock clock, string invoiceNumber);
```

`Activator.CreateInstance` cannot help — it knows nothing about `IClock`.

```
var generator = ActivatorUtilities.CreateInstance<InvoiceGenerator>(provider, "INV-2026-0042");
```

Verified:

```
ActivatorUtilities.CreateInstance<InvoiceGenerator>(provider, "INV-2026-0042"):
  Invoice INV-2026-0042 dated 2026-11-10 for 2,500.00
```

And for a hot path, resolve the *plan* once:

```
ObjectFactory factory = ActivatorUtilities.CreateFactory(typeof(InvoiceGenerator), [typeof(string)]);
var invoice = (InvoiceGenerator)factory(provider, ["INV-0001"]);
```

```
ActivatorUtilities.CreateFactory - resolve the plan once, reuse it:
  Invoice INV-0001 dated 2026-11-10 for 1,000.00
  Invoice INV-0002 dated 2026-11-10 for 1,000.00
  Invoice INV-0003 dated 2026-11-10 for 1,000.00
```

When to use which:

API	Use when
`GetRequiredService<T>()`	the type is registered
`ActivatorUtilities.CreateInstance`	not registered, or needs extra arguments
`ActivatorUtilities.CreateFactory`	same, on a hot path — **cache the factory**

This is how ASP.NET Core activates controllers, and it is what a plugin host *should* use instead of the bare `Activator.CreateInstance` in Part 2 §9 — so plugins can take constructor dependencies.

2.6 Convention-based registration

```
foreach (Type implementation in assembly.GetTypes())
{
    if (implementation.IsAbstract || implementation.IsInterface) continue;
    if (!typeof(IEventHandler).IsAssignableFrom(implementation)) continue;
    if (!implementation.Name.EndsWith("Handler", StringComparison.Ordinal)) continue;

    services.AddTransient(typeof(IEventHandler), implementation);
}
```

Verified:

```
  registered EmailNotificationHandler
  registered InventoryReservationHandler
  skipped LegacyEventListener - does not match the naming convention

Dispatching one event to all discovered handlers:
  EmailNotificationHandler queued mail for OrderPlaced at 09:30
  InventoryReservationHandler reserved stock for OrderPlaced
```

Resolving `IEnumerable<IEventHandler>` returns **every** registration for that service type — the standard way to implement a dispatcher. This reflection happens **once at startup**; nothing scans types per request. Scanning libraries such as **Scrutor** package exactly this pattern.

2.7 A container in 60 lines — no magic left

```
private object Resolve(Type serviceType, HashSet<Type> resolving)
{
    if (_instances.TryGetValue(serviceType, out object? cached)) return cached;

    if (!_registrations.TryGetValue(serviceType, out Type? implementationType))
        throw new InvalidOperationException("No registration for " + serviceType.Name + ".");

    if (!resolving.Add(serviceType))
        throw new InvalidOperationException("Dependency cycle involving " + serviceType.Name + ".");

    // Step 1: read the constructor. This is the reflection.
    ConstructorInfo constructor = implementationType.GetConstructors()
        .OrderByDescending(c => c.GetParameters().Length).First();

    // Step 2: resolve each parameter recursively.
    object[] arguments = constructor.GetParameters()
        .Select(p => Resolve(p.ParameterType, resolving)).ToArray();

    // Step 3: construct.
    object instance = constructor.Invoke(arguments);

    resolving.Remove(serviceType);
    _instances[serviceType] = instance;
    return instance;
}
```

Verified — it resolves the same graph as the real container:

```
Resolved by MiniContainer (60 lines, at the bottom of this file):
  OrderService
    +-- InMemoryOrderRepository
    +-- StandardPricingPolicy
    +-- InMemoryAuditLog
    +-- SystemClock
```

Three ideas: a dictionary from service type to implementation type; `GetConstructors()` to find out what an implementation needs; recursion, then construct.

A production container adds lifetimes, scopes, disposal, thread safety, open generics, `IEnumerable<T>` resolution, and a compiled fast path. **The core idea is this.**

2.8 Do not over-claim: what the real container actually does

The syllabus specifically asks for this clarification, and it is worth getting right.

Verified by reading the provider's private `_engine` field:

```
Package: Microsoft.Extensions.DependencyInjection 10.0.0.0
RuntimeFeature.IsDynamicCodeCompiled = True

ServiceProvider's internal engine on THIS runtime: DynamicServiceProviderEngine
```

> ***It is accurate to say the container uses REFLECTION to read constructors. It is NOT accurate to say every container uses Reflection.Emit to call them.***

`Microsoft.Extensions.DependencyInjection` chooses an activation strategy at run time:

- a **reflection-based** engine, used immediately and always available;
- a **faster engine** built from compiled expression trees or emitted IL, which it switches to in the background once a service has been resolved enough times;
- the **reflection-only** engine when `RuntimeFeature.IsDynamicCodeCompiled` is `false` — the Native AOT case.

Other containers differ: some compile expression trees, some emit IL, some do neither. **"DI uses reflection" is the safe claim.** Anything more specific depends on the container, its version and the platform — so check, as the example does.

And one more thing worth saying out loud: **constructor injection does not *require* reflection.** You can wire an entire application by hand with `new`. Containers use reflection to spare you that wiring, not because the pattern demands it.

2.9 Common mistakes

1. Singleton depending on scoped (captive dependency) — enable `ValidateScopes`.
2. Multiple constructors, then surprise at which one runs.
3. Using the service locator pattern (`provider.GetService` everywhere) instead of constructor injection.
4. Registering a type both as itself and as an interface and expecting one instance — each registration is distinct. Register the concrete type and forward the interface to it.
5. Resolving scoped services from the root provider.
6. Not disposing scopes.

2.10 Audience questions

Q: Does dependency injection always use reflection? No. The *pattern* needs none — hand-wiring is pure constructor calls. The *containers* use reflection to read constructors, and some then generate code to call them. Compile-time DI containers exist that use no reflection at all.

Q: Is DI slow because of reflection? Resolution cost is dominated by graph size and lifetime bookkeeping, not reflection, because the plan is cached after first use. Transient services in a deep graph cost more than the reflection ever did.

Q: Why does it pick the greediest constructor? So that optional dependencies are supplied when available. The cost is the surprise in §2.4. One constructor per class avoids the whole question.

2.11 Documentation

- Dependency injection in .NET — https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection
- DI guidelines — https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection-guidelines
- `ActivatorUtilities` — https://learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinjection.activatorutilities

---

3. Entity Framework Core 10

3.1 Entity discovery

```
public class BloggingContext(DbContextOptions<BloggingContext> options) : DbContext(options)
{
    public DbSet<Blog> Blogs => Set<Blog>();
    public DbSet<Post> Posts => Set<Post>();
    public DbSet<Author> Authors => Set<Author>();
}
```

EF reflects over those `DbSet<T>` properties to discover the root entity types. Verified:

```
The DbSet<T> properties declared on BloggingContext, read with reflection:
  DbSet<Blog> Blogs
  DbSet<Post> Posts
  DbSet<Author> Authors

The entity types EF actually put in the model:
  Author   -> table "authors", key = AuthorKey
  Blog     -> table "Blogs", key = Id
  Post     -> table "blog_posts", key = Id
```

Three ways a type enters the model:

1. a `DbSet<T>` property on the context (the usual way);
2. `modelBuilder.Entity<T>()` in `OnModelCreating`;
3. **being reachable by navigation** from a type already in the model.

That third rule catches people out: adding a navigation property can pull a whole new type into the model and change your schema, with no `DbSet` in sight.

The model is built **once**, on first use, and cached for the application's lifetime — the same pattern as MVC in §1.5.

3.2 EF Core model metadata vs CLR metadata

The syllabus asks for this distinction explicitly, and the example makes it concrete.

```
CLR metadata for Post - what the C# compiler emitted:
  Int32       Id
  String      Title
  String      Content
  DateTime    PublishedOn
  Int32       Rating
  Int32       WordCount
  Int32       BlogId
  Blog        Blog
  Author      Author
  Int32?      AuthorId

EF Core model metadata for Post - what EF decided to map:
  Int32       Id              column "post_id"
  Int32?      AuthorId        column "AuthorId"
  Int32       BlogId          column "BlogId"
  String      Content         column "Content"
  DateTime    LastUpdatedUtc  column "last_updated_utc"  <- SHADOW: no CLR property exists
  DateTime    PublishedOn     column "PublishedOn"
  Int32       Rating          column "Rating"
  String      Title           column "Title"
```

The lists are **different**, in three distinct ways:

Member	CLR	EF model	Why
`WordCount`	yes	**no**	`[NotMapped]`
`Blog`, `Author`	yes	**not as properties**	they are **navigations**, tracked separately

Member	CLR	EF model	Why
`LastUpdatedUtc`	**no**	yes	a **shadow property**

> ***The EF Core model is a SEPARATE model that EF BUILDS BY READING CLR metadata**, then augments with its own conventions and your configuration. It is not a view over reflection, and it is not limited to what reflection can see. **Reflection is the input; `IModel` is the output, and the output is richer.***

A **shadow property** is the cleanest proof. Declared in the model only:

```
modelBuilder.Entity<Post>()
    .Property<DateTime>("LastUpdatedUtc")
    .HasColumnName("last_updated_utc");
```

There is no `post.LastUpdatedUtc` to assign, so it is reached by name through the change tracker:

```
entry.Property<DateTime>("LastUpdatedUtc").CurrentValue = ...;
```

3.3 Conventions, annotations, fluent API

The example configures three entities three different ways, deliberately.

Convention (`Blog`) — nothing configured, EF infers everything: a property named `Id` becomes the key; a collection navigation makes a one-to-many; `Post.BlogId` is inferred as the foreign key; the table is named after the `DbSet`.

Data annotations (`Post`) — explicit metadata on the type:

```
[Table("blog_posts")]
public class Post
{
    [Key] [Column("post_id")] public int Id { get; set; }
    [Required] [MaxLength(200)] public string Title { get; set; } = string.Empty;
    [NotMapped] public int WordCount => /* ... */;
}
```

Fluent API (`Author`) — no attributes on the class at all:

```
modelBuilder.Entity<Author>(author =>
{
    author.ToTable("authors");
    author.HasKey(a => a.AuthorKey);          // convention would NOT find this key
    author.Property(a => a.Name).HasColumnName("full_name").HasMaxLength(120).IsRequired();
    author.HasIndex(a => a.EmailAddress).IsUnique();
});
```

> ***Precedence, lowest to highest: CONVENTION < DATA ANNOTATION < FLUENT API.***

Why both exist: attributes are discoverable and sit next to the member; the fluent API keeps EF types out of a domain class, supports configuration attributes cannot express (attribute arguments must be compile-time constants — Part 2 §8.4), and can vary by environment.

Note the parallel with Part 1 §5: **convention is implicit metadata inferred from the declaration; annotations and fluent calls are explicit metadata you supplied.**

3.4 The schema EF generated

Verified, read back from SQLite itself:

```
CREATE TABLE "Blogs" (
    "Id" INTEGER NOT NULL CONSTRAINT "PK_Blogs" PRIMARY KEY AUTOINCREMENT,
    "Title" TEXT NOT NULL,
    "Url" TEXT NULL,
    "CreatedOn" TEXT NOT NULL
)

CREATE TABLE "authors" (
    "AuthorKey" INTEGER NOT NULL CONSTRAINT "PK_authors" PRIMARY KEY AUTOINCREMENT,
    "full_name" TEXT NOT NULL,
    "email" TEXT NOT NULL
)

CREATE TABLE "blog_posts" (
    "post_id" INTEGER NOT NULL CONSTRAINT "PK_blog_posts" PRIMARY KEY AUTOINCREMENT,
    "Title" TEXT NOT NULL,
    "Content" TEXT NOT NULL,
    "PublishedOn" TEXT NOT NULL,
    "Rating" INTEGER NOT NULL,
    "BlogId" INTEGER NOT NULL,
    "AuthorId" INTEGER NULL,
    "last_updated_utc" TEXT NOT NULL,
    CONSTRAINT "FK_blog_posts_Blogs_BlogId" FOREIGN KEY ("BlogId") REFERENCES "Blogs" ("Id") ON DELETE CASCADE,
    CONSTRAINT "FK_blog_posts_authors_AuthorId" FOREIGN KEY ("AuthorId") REFERENCES "authors" ("AuthorKey") ON
        DELETE SET NULL
)

CREATE UNIQUE INDEX "IX_authors_email" ON "authors" ("email")
```

Every name in that DDL traces back to a piece of metadata: `blog_posts`/`post_id` from attributes; `authors`/`full_name`/`email` from the fluent API; `Blogs` from convention; `last_updated_utc` from the shadow property; no `WordCount` column because of `[NotMapped]`; the unique index from `HasIndex(...).IsUnique()`.

3.5 Expression trees becoming SQL

This is where Part 4 §8 and EF Core meet, and it is the most impressive demo in the session.

```
IQueryable<Post> highRated = context.Posts.Where(p => p.Rating >= 4);
```

The tree EF is handed:

```
[Microsoft.EntityFrameworkCore.Query.EntityQueryRootExpression].Where(p => (p.Rating >= 4))
```

The SQL it produces (`ToQueryString()` does **not** execute):

```
SELECT "b"."post_id", "b"."AuthorId", "b"."BlogId", "b"."Content", "b"."last_updated_utc", "b"."PublishedOn",
    "b"."Rating", "b"."Title"
FROM "blog_posts" AS "b"
WHERE "b"."Rating" >= 0 AND "b"."Rating" >= 4
```

Note the extra `Rating >= 0`: that is the **global query filter** declared with `HasQueryFilter` in `OnModelCreating`. It is stored in the **model** as an expression tree and combined into every query over `Post`.

A projection with navigations:

```
context.Posts
    .Where(p => p.Rating >= 4 && p.Blog!.Title.Contains(".NET"))
    .OrderByDescending(p => p.PublishedOn)
    .Select(p => new { p.Title, Blog = p.Blog!.Title, Author = p.Author!.Name });
```

```
SELECT "b"."Title", "b0"."Title" AS "Blog", "a"."full_name" AS "Author"
FROM "blog_posts" AS "b"
INNER JOIN "Blogs" AS "b0" ON "b"."BlogId" = "b0"."Id"
LEFT JOIN "authors" AS "a" ON "b"."AuthorId" = "a"."AuthorKey"
WHERE "b"."Rating" >= 0 AND "b"."Rating" >= 4 AND instr("b0"."Title", '.NET') > 0
ORDER BY "b"."PublishedOn" DESC
```

EF translated the navigation into an `INNER JOIN`, the nullable navigation into a `LEFT JOIN`, and the anonymous type into a `SELECT` list — so it never loaded a whole `Post` entity.

> ***Look at how*** `string.Contains` ***was rendered:*** `instr("b0"."Title", '.NET') > 0`***.*** *That is a* ***SQLite*** *function, not* `LIKE`*. The* ***provider*** *chooses the translation, and the SQL Server provider emits something different for the same C#. This is why you should read the SQL your provider actually produces rather than assume —* `ToQueryString()` *exists for precisely that.*

And a reusable, composable predicate — the real reason this matters:

```
Expression<Func<Post, bool>> isRecommended = p => p.Rating >= 4;
IQueryable<Post> recommended = context.Posts.Where(isRecommended);
```

The **identical object** could instead be `.Compile()`d and run over a `List<Post>` in memory (Part 4 §8). One definition of "recommended", two execution engines. That is only possible because it is **data**, not a delegate.

3.6 The classic performance bug, made visible

```
GOOD: context.Posts.Where(p => p.Rating >= 4).ToList()
  -> WHERE "b"."Rating" >= 0 AND "b"."Rating" >= 4     ← filtering in SQL

BAD : context.Posts.ToList().Where(p => p.Rating >= 4)
  -> WHERE "b"."Rating" >= 0                            ← every row transferred
```

Both produce the same answer. On a large table only one is usable.

The cause is a **type** difference that is invisible in the source: `Where(Func<Post,bool>)` on `IEnumerable` and `Where(Expression<Func<Post,bool>>)` on `IQueryable` look identical. An accidental `.AsEnumerable()` or `.ToList()` before `.Where()` switches you from the second to the first, and filtering silently moves from the database into your process — after downloading every row.

3.7 When translation fails

```
context.Posts.Where(p => IsProofRead(p.Content)).ToList()
```

Verified:

```
  InvalidOperationException:
    The LINQ expression 'DbSet<Post>() .Where(p => p.Rating >= 0)
    .Where(p => Program.IsProofRead(p.Content))' could not be translated
```

EF Core **refuses** rather than silently downloading the table and filtering in memory. That refusal is deliberate: the silent version was the behaviour of older EF Core releases and caused serious production surprises.

Three fixes, in order of preference:

1. **Rewrite** the predicate using constructs EF can translate.
2. **Narrow in SQL first, then finish in memory, explicitly:**

```
context.Posts
    .Where(p => p.Rating >= 3)          // translated, runs in SQL
    .AsEnumerable()                      // the boundary, stated out loud
    .Where(p => IsProofRead(p.Content))   // ordinary LINQ to Objects
    .ToList();
```

1. **Push the logic into the database** as a function and map it with `modelBuilder.HasDbFunction`.

Also: what *is* translatable changes between EF versions and between providers. A query that works on SQL Server may fail on SQLite. **The provider decides.**

3.8 Change tracking

```
Loaded: "Reflection in practice" (rating 5)
  tracked state: Unchanged

After 'post.Rating = 3;' - we touched a plain C# property, nothing else:
  tracked state: Modified

  Modified properties EF detected by comparing against its snapshot:
    Rating: "5" -> "3"
```

EF kept the **original values** so it can generate a minimal `UPDATE`, obtained by reading the entity's properties when it materialised the object.

And on how values get in and out of your objects: **EF Core does not call `PropertyInfo.GetValue` per property per row.** It builds accessor delegates once per property — from expression trees on a JIT runtime — and reuses them. Same lesson as Part 3 §2.3: reflect once, then run compiled code.

3.9 The whole pipeline

```
  stage                    mechanism                        when
  ------------------------------------------------------------------------
  find DbSet<T> properties  System.Reflection                startup (once)
  read annotations          GetCustomAttributes              startup (once)
  apply conventions         EF's convention set              startup (once)
  apply fluent config       your OnModelCreating             startup (once)
  build IModel              EF model builder                 startup (once)
  build accessors           expression trees -> delegates    first use, cached
  translate a query         ExpressionVisitor chain -> SQL   per query shape, then cached
  materialise rows          cached accessor delegates        per row
```

Every technique in this session appears in that table: Part 2 reflection, Part 3 code generation, Part 4 expression trees.

3.10 Audience questions

Q: How does EF Core translate LINQ into SQL? The query arrives as an **expression tree** because `IQueryable<T>.Where` takes `Expression<Func<T,bool>>`, not `Func<T,bool>`. EF runs a chain of `ExpressionVisitor`s over it: identify the query root, bind member accesses to mapped properties and navigations, convert navigations to joins, translate method calls to provider-specific SQL functions, then print SQL. The translation is cached per query *shape*, so a loop executing the same query with different parameter values translates once.

Q: Does EF Core use reflection per row? No. Accessor delegates are compiled once per property and cached.

Q: Is the EF model the same as my classes? No — §3.2. It is a richer, separate model built *from* your classes.

Q: Why does the same LINQ work on SQL Server but not SQLite? Translation is provider-specific. Each provider supports a different set of functions. Always check with `ToQueryString()`.

3.11 Documentation

- EF Core — https://learn.microsoft.com/en-us/ef/core/
- Creating and configuring a model — https://learn.microsoft.com/en-us/ef/core/modeling/
- How queries work — https://learn.microsoft.com/en-us/ef/core/querying/how-query-works
- Client vs server evaluation — https://learn.microsoft.com/en-us/ef/core/querying/client-eval

---

4. Serialization

4.1 What a serializer must figure out

To turn an arbitrary object into JSON, a library must answer at run time: which members to write; what name each gets; each member's type; how to **read** each value; and, on the way back, which constructor or setters to use.

Every answer is in metadata, so the Part 2 API suffices:

```
  Int32             InvoiceId       -> "invoice_id"
  String            CustomerName
  DateOnly          IssuedOn
  Decimal           AmountDue
  InvoiceStatus     Status
  List`1            Lines
  String            InternalNotes   [JsonIgnore]
```

This is why serialization is the canonical metaprogramming example: the library author cannot know your types, so the library must ask the types themselves.

4.2 A 20-line serializer, to remove the magic

```
private static void WriteObject(StringBuilder builder, object instance)
{
    PropertyInfo[] properties = instance.GetType()
        .GetProperties(BindingFlags.Public | BindingFlags.Instance);

    builder.Append('{');
    bool first = true;
    foreach (PropertyInfo property in properties)
    {
        if (!property.CanRead) continue;
        if (property.GetCustomAttribute<JsonIgnoreAttribute>() is not null) continue;

        string name = property.GetCustomAttribute<JsonPropertyNameAttribute>()?.Name
            ?? char.ToLowerInvariant(property.Name[0]) + property.Name[1..];

        if (!first) builder.Append(',');
        builder.Append('"').Append(name).Append("\":");
        WriteValue(builder, property.GetValue(instance));
        first = false;
    }
    builder.Append('}');
}
```

`GetProperties()` → `GetValue()` → format → write a pair. That is the whole idea. What a real serializer adds on top is escaping, cycles, numbers, culture, nulls, polymorphism, dictionaries, streaming, async and speed. **Use `System.Text.Json`. Write `MiniJson` once, to understand it.**

4.3 Attributes that steer serialization

```
internal sealed class Invoice
{
    [JsonPropertyName("invoice_id")] public int InvoiceId { get; set; }
    [JsonPropertyOrder(-1)] public string CustomerName { get; set; } = string.Empty;
    [JsonConverter(typeof(IsoDateOnlyConverter))] public DateOnly IssuedOn { get; set; }
    [JsonConverter(typeof(JsonStringEnumConverter<InvoiceStatus>))] public InvoiceStatus Status { get; set; }
    [JsonIgnore] public string? InternalNotes { get; set; }
}
```

Two observations worth making in the talk:

- `[JsonPropertyName]` **beats** `PropertyNamingPolicy`**.** An explicit attribute always wins over a global convention — the same "explicit overrides implicit" rule as Part 1 §5 and EF Core §3.3.

- `InternalNotes` **exists on the object but never reaches the JSON.** One attribute keeps internal data out of an API response, enforced by the serializer rather than by every caller.

4.4 Reflection mode vs source-generated mode

```
// RUNTIME: reflects over Invoice on first use, then caches the plan.
string a = JsonSerializer.Serialize(invoice, options);

// COMPILE-TIME: the plan was generated during the build.
string b = JsonSerializer.Serialize(invoice, InvoiceJsonContextCompact.Default.Invoice);
```

The source-generated context has **no body we wrote**:

```
[JsonSourceGenerationOptions(WriteIndented = false, PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase)]
[JsonSerializable(typeof(Invoice))]
internal partial class InvoiceJsonContextCompact : JsonSerializerContext;
```

Proof the generator filled it in, read from the compiled assembly:

```
  type     : Metaprogramming.SerializationDemo.InvoiceJsonContext
  base     : JsonSerializerContext
  generated properties:
    JsonTypeInfo`1 Decimal
    JsonTypeInfo`1 Invoice
    JsonTypeInfo`1 InvoiceLine
    JsonTypeInfo`1 InvoiceStatus
    JsonTypeInfo`1 ListInvoiceLine
```

We declared `[JsonSerializable(typeof(Invoice))]` — **one** type. The generator produced **five** `JsonTypeInfo` properties, because it walked the whole object graph transitively: `Invoice` needs `List<InvoiceLine>`, which needs `InvoiceLine`, which needs `decimal`, and `Invoice` needs `InvoiceStatus`. That transitive closure is exactly why source generation can replace run-time discovery — and also why adding a property of a brand-new type requires a rebuild.

And the outputs:

```
Byte-for-byte identical: True
```

aspect	reflection mode	source-generated mode
when the plan is built	first call, at run time	during the build
mechanism	`System.Reflection`, then generated accessors	Roslyn generator emits C#
startup cost	pay on first use per type	none
trimming	unsafe: members may be trimmed	safe: all usage is visible
Native AOT	not supported	**supported**
new type support	automatic	add `[JsonSerializable]` and rebuild
polymorphism, dynamic shapes	more flexible	more restricted

Same goal, same output. The only real difference is WHEN the metaprogramming happened — Part 1 §4 made concrete.

4.5 The caching claim, measured honestly

The standard advice is "create `JsonSerializerOptions` once and reuse it", usually justified with "otherwise it re-reflects every call". That justification is only half right on current .NET. Measured three ways:

```
  5,000 serializations of the same object:
    one reused options instance          14 ms   1.0x
    new options, identical settings      19 ms   1.4x
    new options, DIFFERENT settings     101 ms   7.2x
```

- Reusing one instance is fastest, as expected.
- **New-but-identical options is only moderately slower.** `System.Text.Json` keeps a process-wide cache of serialization metadata that *equivalent* options instances share, so the expensive reflection is not repeated. You still pay for the allocation and the cache lookup.
- **New-and-different options is dramatically slower.** Nothing can be shared, so the type really is analysed again for every distinct configuration.

> ***The correct statement: reuse your options instance, and above all do not vary its settings per call.*** *The second mistake is the expensive one.*

4.6 Deserialization is harder

```
  InternalNotes : (null)  <- [JsonIgnore] means it was never written, so it cannot come back
```

Why the inbound direction is harder:

- Writing needs only a getter. Reading must **create** the object first.
- It must choose a constructor, then match JSON names to parameters.
- For `init`-only or `readonly` members it must use the constructor, not setters.
- Missing members, extra members and nulls all need a policy.

This is the same constructor-selection problem the DI container solves (§2.4) and that `Activator.CreateInstance` faces (Part 2 §7). Three unrelated-looking features, one underlying metadata question — a good point to make on slide 25.

Two run-time behaviours worth knowing:

```
  JsonException: The JSON value could not be converted to System.Int32.

And an UNKNOWN property is silently ignored by default, which surprises people:
  deserialized InvoiceId = 99, unknown member discarded
```

Set `JsonSerializerOptions.UnmappedMemberHandling = Disallow` to make unknown members an error.

4.7 Guidance

Source generation when: publishing Native AOT or trimmed; startup time matters (serverless, CLI); the set of types is known at build time (the usual case).

Reflection mode when: types are genuinely unknown until run time (a plugin's own models); you need broad polymorphic support; you are prototyping.

Where else this exact pattern appears: the regex source generator vs `RegexOptions.Compiled`; the logging source generator (`[LoggerMessage]`); the configuration binder generator; minimal API request-delegate generation. **The direction of travel in .NET is consistently from run time to build time.**

4.8 Documentation

- `System.Text.Json` overview — https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview
- Source generation — https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/source-generation

---

5. Plugin Architecture

Fully covered in **Part 2 §9** — the contract design, `AssemblyLoadContext`, type identity across contexts, unloading and its limits, version compatibility, and the security analysis.

The three points to repeat on slide 26:

1. **Interface + attribute + reflection + activation = a plugin system in 20 lines.**
2. **Unloading is cooperative.** `Unload()` is a request; one cached `Type` or subscribed delegate keeps the context alive forever.
3. `AssemblyLoadContext` **is not a security sandbox.** It isolates **type identity**, not **privileges**. There is no supported in-process sandbox in modern .NET; use a process, container or VM boundary for untrusted code.

The one upgrade to make here: a real host should activate plugins with `ActivatorUtilities` (§2.5) rather than bare `Activator.CreateInstance`, so plugins can declare constructor dependencies the host controls.

6. Dynamic Proxies

6.1 The problem

The service you want:

```
public decimal GetBalance(string account) => _balances[account];
```

What it becomes once the operational requirements land:

```
public decimal GetBalance(string account)
{
    _logger.LogInformation("GetBalance {Account}", account);      // logging
    var sw = Stopwatch.StartNew();                                // metrics
    if (_cache.TryGetValue(account, out var hit)) return hit;     // caching
    try
    {
        var result = _balances[account];                          // the actual work
        _cache[account] = result;
        return result;
    }
    catch (Exception ex) { _logger.LogError(ex, "..."); throw; }
    finally { _metrics.Record(sw.Elapsed); }                      // metrics
}
```

One line of business logic, ten lines of plumbing — **repeated in every method of every service**. These are **cross-cutting concerns**: orthogonal to the business logic, yet physically tangled into it.

6.2 `DispatchProxy`

`DispatchProxy` generates a type at run time that implements an interface and routes **every** call through one method you override.

```
public class LoggingProxy : InterceptingProxy
{
    public List<string> Log { get; } = [];

    protected override object? Invoke(MethodInfo? targetMethod, object?[]? args)
    {
        Log.Add("--> " + targetMethod!.Name + "(" + Format(args) + ")");
        var stopwatch = Stopwatch.StartNew();
        try
        {
            object? result = InvokeTarget(targetMethod, args);
            stopwatch.Stop();
            Log.Add("<-- " + targetMethod.Name + " returned " + result
                + " in " + stopwatch.Elapsed.TotalMilliseconds.ToString("N2") + " ms");
            return result;
        }
        catch (Exception ex)
        {
            Log.Add("!!! " + targetMethod.Name + " threw " + ex.GetType().Name);
            throw;
        }
    }
}
```

Created with:

```
object proxy = DispatchProxy.Create<TInterface, TProxy>()!;
```

Verified:

```
Calls made through the proxy (ordinary C#, no reflection at the call site):

  GetBalance("SB-1001")  = 25400.00
  Deposit("SB-1001", 500) = 25900.00
  GetAccountCount()       = 2

What the interceptor recorded, without AccountService knowing it exists:

  --> GetBalance(SB-1001)
  <-- GetBalance returned 25400.00 in 0.73 ms
  --> Deposit(SB-1001, 500)
  <-- Deposit returned 25900.00 in 0.26 ms
  --> GetAccountCount()
  <-- GetAccountCount returned 2 in 0.11 ms
```

`AccountService.cs` **contains no logging code at all.**

6.3 What was actually created

```
The object we are holding:
  type name      : generatedProxy_1
  full name      : generatedProxy_1
  base type      : LoggingProxy  <- our LoggingProxy
  implements     : IAccountService
  assembly       : ProxyBuilder
  IsDynamic      : True  <- generated at run time

It is NOT an AccountService:
  proxied is AccountService  = False
  proxied is IAccountService = True
  proxied is DispatchProxy   = True
```

`DispatchProxy` **did exactly what Part 3 §5 did by hand** — defined a type at run time, made it implement a known interface, created an instance. It wrote the `TypeBuilder` code for us, and every interface method it generated calls our `Invoke` override.

This is the bridge between Part 3 and Part 4: the generated-type pattern, packaged.

6.4 Attribute-driven caching

```
public interface IAccountService
{
    [Cacheable] decimal GetBalance(string accountNumber);
    decimal Deposit(string accountNumber, decimal amount);
    int GetAccountCount();
}
```

Verified:

```
Calling GetBalance three times with the same argument:
  call 1 -> 25400.00   (service hit count: 1)
  call 2 -> 25400.00   (service hit count: 1)
  call 3 -> 25400.00   (service hit count: 1)

  cache hits   : 2
  cache misses : 1
  the real service was read 1 time(s) for GetBalance
```

The attribute is the whole policy. Marking a new method `[Cacheable]` changes behaviour with no change to the proxy and none to the service.

> ***Worth stating clearly: metaprogramming provided the SEAM, not the POLICY.*** *This cache has no expiry, no size limit, no invalidation and no thread safety. A real one needs all four — use* `HybridCache` *or* `IMemoryCache` *and think about correctness. The same applies to* ***authorization****: a proxy can be where you enforce a policy, but it does not supply the policy or make it correct.*

The syllabus asks for exactly this clarification, and it is the honest framing.

6.5 Correct exception handling — the detail that bites

`targetMethod.Invoke(...)` wraps any exception from the target in `TargetInvocationException`. If a proxy lets that escape, **every** `catch (TimeoutException)` **upstream silently stops matching.**

```
protected object? InvokeTarget(MethodInfo targetMethod, object?[]? args)
{
    try
    {
        return targetMethod.Invoke(Target, args);
    }
    catch (TargetInvocationException ex) when (ex.InnerException is not null)
    {
        ExceptionDispatchInfo.Capture(ex.InnerException).Throw();
        throw; // unreachable
    }
}
```

Verified — the caller catches the *real* exception type:

```
  caught TimeoutException: Gateway timed out (call 2).
```

`ExceptionDispatchInfo` rather than `throw ex.InnerException`, because the latter resets the stack trace.

6.6 Composition

```
Chain: caller -> LoggingProxy -> CachingProxy -> AccountService

Log from the outer proxy:
  --> GetBalance(SB-1001)
  <-- GetBalance returned 25400.00 in 0.03 ms
  --> GetBalance(SB-1001)
  <-- GetBalance returned 25400.00 in 0.00 ms
  --> GetBalance(SB-1001)
  <-- GetBalance returned 25400.00 in 0.01 ms

  cache hits = 2, misses = 1, real service reads = 1
```

Each proxy is just another `IAccountService`, so they compose freely. **Order matters:** logging outside caching logs every call; logging inside caching would log only the misses.

6.7 Registering in DI

```
services.AddSingleton<AccountService>();
services.AddSingleton<IAccountService>(provider =>
{
    var inner = provider.GetRequiredService<AccountService>();
    return InterceptingProxy.Wrap<IAccountService, LoggingProxy>(inner);
});
```

Verified — the consumer has an ordinary constructor dependency and no idea:

```
MonthEndReport asked for IAccountService and got a proxy:
  injected type: generatedProxy_1
```

This is the **decorator pattern with a generated decorator**. Scrutor's `Decorate<TService, TDecorator>()` packages this registration.

6.8 Limitations — all verified

A. Interfaces only. `DispatchProxy` cannot proxy a class, sealed or not:

```
   ArgumentException: The type 'AccountService' must be an interface, not a class. (Parameter 'interfaceType')
```

Consequence: your services must be behind interfaces. Castle DynamicProxy can also subclass a class, but then only intercepts **virtual** members — a non-virtual method can never be intercepted by subclassing.

(.NET 10 also has a non-generic `DispatchProxy.Create(Type, Type)`, useful when the interface type is only known at run time.)

B. The proxy base class must not be sealed. The generated type derives from it:

```
ArgumentException: The base type '...' cannot be sealed. (Parameter 'TProxy')
```

It must also have an accessible parameterless constructor, which is why configuration happens after creation (`SetTarget`) rather than in a constructor.

C. Only calls through the interface are intercepted. Holding the real object bypasses the proxy entirely — and a call from one method of the service to another is **also not intercepted**, because `this` inside the service is the real object, not the proxy. **That self-call blind spot surprises people and breaks naive caching.**

D. Async needs deliberate handling. `Invoke` returns `object?`. For an `async` method that object is a `Task`, returned the instant the method starts — so a naive stopwatch around `InvokeTarget` measures only how long it took to *create* the `Task`. To time or catch properly you must await the returned task, which means inspecting the return type and handling `Task`, `Task<T>` and `ValueTask<T>` separately. (The timings in §6.2 are honest only because `IAccountService` is entirely synchronous.)

E. Per-call cost. Every call goes through `targetMethod.Invoke` — the slow reflective path measured in Part 3 §2.3. For a hot inner loop, cache a delegate per `MethodInfo`, or do not use a proxy.

F. Native AOT. `DispatchProxy` generates a type at run time, so it needs dynamic code support and does not work under Native AOT.

G. Debugging and clarity. A stack trace now runs through a generated type, and "go to definition" on an interface no longer reaches the code that runs. **Interception is powerful precisely because it is invisible, and that is also its main cost to the next person reading the code.**

6.9 Audience questions

Q: What are the limitations of dynamic proxies? §6.8, and the first three matter most: interfaces only, self-calls are not intercepted, and async needs explicit handling.

Q: `DispatchProxy` or Castle DynamicProxy? `DispatchProxy` is built in — no package, no licence question — but interface-only and minimal. Castle handles virtual class members, has a richer interceptor pipeline, and better async support. Most production AOP in .NET uses Castle, usually via Autofac interceptors.

Q: Is this AOP? It is one implementation of AOP's interception idea. Classical AOP (AspectJ) weaves at compile time; .NET IL weavers do it post-compile; proxies do it at run time with a wrapper object.

6.10 Documentation

- `DispatchProxy` — https://learn.microsoft.com/en-us/dotnet/api/system.reflection.dispatchproxy

7. Cross-Cutting Concerns — Choosing the Right Seam

7.1 The four seams

```
seam             scope                       reaches
-----------------------------------------------------------------------
Middleware       every HTTP request          the whole pipeline
MVC filters      controller actions          per action/controller
Dynamic proxies  any interface call          any layer, incl. services
Source generators compile time               anything, with zero runtime cost
```

7.2 Choosing by where the concern belongs

Concern	Seam	Why
Authentication, CORS, HTTPS redirect, request logging	**Middleware**	it is about the HTTP request, not about any one method
Model validation, authorization on an action, result shaping	**Filters**	they know MVC concepts: model state, action results
Retry on a flaky dependency, caching a repository, auditing domain operations	**Proxies / decorators**	the concern is about a service interface and applies however it is called, HTTP or not
High-volume logging, serialization, mapping	**Source generators**	the shape is known at build time; zero run-time cost

The test: would this concern still apply if the call arrived from a background job instead of HTTP? If yes, it belongs at the service seam (proxy or decorator), not in middleware.

7.3 Logging — the worked example

§6.2 is the logging example: `LoggingProxy` records entry, exit, elapsed time and exceptions for every method of any interface, with no change to the service.

7.4 Caching and authorization need real policies

Repeating §6.4 because the syllabus asks for it explicitly, and because it is the most important caveat in this section:

> ***Metaprogramming gives you the interception point. It does not give you a correct caching strategy or a correct authorization model.***

- **Caching** needs expiry, size limits, invalidation, stampede protection and thread safety. The demo cache in example 12 has none of these. Use `HybridCache` or `IMemoryCache`, and think hardest about *invalidation*.
- **Authorization** needs a policy: who the caller is, what they may do, and what happens when they may not. ASP.NET Core's authorization system supplies that. A proxy can be *where* the check runs; it is not the check.

An interception mechanism that silently applies a wrong policy is worse than no interception, because the wrongness is now invisible.

7.5 The answer that is right more often than any of them

A hand-written decorator class.

```
public sealed class CachingAccountService(IAccountService inner, IMemoryCache cache) : IAccountService
{
    public decimal GetBalance(string account) =>
        cache.GetOrCreate(account, _ => inner.GetBalance(account));

    public decimal Deposit(string account, decimal amount) => inner.Deposit(account, amount);
    public int GetAccountCount() => inner.GetAccountCount();
}
```

Twenty lines of ordinary C#: fully debuggable, trimmable, AOT-safe, obvious to the next reader, and no reflection per call. **Reach for a generated proxy when the number of interfaces makes hand-writing them genuinely impractical** — not before.

7.6 AOP concepts, briefly

Term	Meaning
Cross-cutting concern	a requirement that spans many modules (logging, security)
Aspect	the encapsulation of one such concern
Join point	a place where an aspect could apply (a method call)
Pointcut	the rule selecting join points (`[Cacheable]`, or "all methods of IFoo")
Advice	the code that runs (before / after / around)
Weaving	combining aspects with the code — at compile time, post-compile (IL weaving), or run time (proxies)

In .NET: `[Cacheable]` is a pointcut, `CachingProxy.Invoke` is around-advice, and `DispatchProxy.Create` is run-time weaving.

---

8. Expression Trees

8.1 The same lambda, two completely different things

```
Func<int, bool> asDelegate = n => n > 18;             // compiled IL — you can CALL it
Expression<Func<int, bool>> asExpression = n => n > 18; // an object GRAPH — you can READ it
```

Verified:

```
Source text is identical in both cases: n => n > 18

Func<int, bool>:
  runtime type : Func`2
  can call it  : asDelegate(21) = True
  can read it  : no - the body is compiled IL
  Method.Name  : <DelegateVersusExpression>b__1_0  (a compiler-generated method)

Expression<Func<int, bool>>:
  runtime type : Expression1`1
  NodeType     : Lambda
  Body         : (n > 18)
  Body.NodeType: GreaterThan
  Parameters   : Int32 n
  ToString()   : n => (n > 18)
  can call it  : not directly - Compile() first
```

> ***The difference is decided entirely by the TYPE you assign the lambda to.*** `Func<...>` → *compile the body to IL.* `Expression<...>` → *compile code that* ***builds a description*** *of the body.*

This is the one place where C# itself does compile-time metaprogramming on your behalf: it hands you your own source code as inspectable data.

8.2 The tree really is a tree

```
Expression<Func<Customer, bool>> predicate = c => c.Age > 18 && c.Country == "IN";
```

Verified:

```
Structure:
  AndAlso   (BinaryExpression, Boolean)
    GreaterThan   (BinaryExpression, Boolean)
      MemberAccess  -> Age : Int32
        Parameter     -> c : Customer
      Constant      -> 18 : Int32
    Equal   (BinaryExpression, Boolean)
      MemberAccess  -> Country : String
        Parameter     -> c : Customer
      Constant      -> IN : String
```

Every node is a strongly typed object:

```
  and.NodeType            = AndAlso
  left.NodeType           = GreaterThan
  member.Member.Name      = Age   (a Property on Customer)
  constant.Value          = 18 (Int32)
```

Because every node is an object, a library can ask *"which property is being compared, and to what?"* — **precisely the question a SQL translator must answer.**

8.3 `Compile()` — back to executable code

```
Func<Customer, bool> compiled = predicate.Compile();
```

```
predicate.Compile() returned a Func`2

  Asha   age 29 IN  -> True
  Ravi   age 16 IN  -> False
  Mia    age 34 US  -> False
```

`Compile()` **is expensive** — it generates and JITs code, using the same lightweight code generation as `DynamicMethod`. Compile once, cache the delegate, call it many times. Compiling inside a loop or per web request is the most common performance mistake with this API.

And a precision point: `Compile()` **needs runtime code generation, so it is unavailable under Native AOT. Expression trees can still be BUILT and READ there** — which is why EF Core can translate queries to SQL under AOT, because translation only *reads* the tree. It is `Compile()` that needs the JIT.

8.4 Building a tree by hand

```
ParameterExpression a = Expression.Parameter(typeof(int), "a");
ParameterExpression b = Expression.Parameter(typeof(int), "b");
BinaryExpression sum = Expression.Add(a, b);
ConstantExpression two = Expression.Constant(2, typeof(int));
BinaryExpression doubled = Expression.Multiply(sum, two);
Expression<Func<int, int, int>> lambda = Expression.Lambda<Func<int, int, int>>(doubled, a, b);
Func<int, int, int> fn = lambda.Compile();
```

Verified:

```
  Expression.Add(a, b)                    -> (a + b)
  Expression.Multiply(sum, two)           -> ((a + b) * 2)
  Expression.Lambda<Func<int,int,int>>(..)  -> (a, b) => ((a + b) * 2)

  fn(3, 4)   = 14
  fn(10, 11) = 42
```

Compare with Part 3, which built the same function in IL:

	Reflection.Emit	Expression tree
Code	`ldarg.0 / ldarg.1 / add / ldc.i4.2 / mul / ret`	`Add(a,b)` → `Multiply(…, 2)` → `Lambda`
Type-checked as you build	**no**	**yes —** `Expression.Add` **throws immediately on a type mismatch**
Printable	no	yes, via `ToString()`
Speed after compilation	same	same
Can create a **type**	yes	**no**

Recall Part 3 §8.2, where `add` on an `int` and a string **silently returned garbage**. `Expression.Add` would have thrown on the spot.

> ***Prefer expression trees over Reflection.Emit whenever you need a FUNCTION. Reach for Emit only when you need a TYPE, which*** `Expression` ***cannot create.***

8.5 A dynamic filter from runtime input

Filters that could have come from a query string (`?minAge=21&country=IN`):

```
Expression<Func<T, bool>> Build<T>(IReadOnlyDictionary<string, string> filters)
{
    ParameterExpression parameter = Expression.Parameter(typeof(T), "x");
    Expression? body = null;

    foreach ((string propertyName, string filter) in filters)
    {
        PropertyInfo? property = typeof(T).GetProperty(propertyName,
            BindingFlags.Public | BindingFlags.Instance | BindingFlags.IgnoreCase);
        if (property is null) throw new ArgumentException(/* ... */);

        MemberExpression member = Expression.Property(parameter, property);
        object typedValue = Convert.ChangeType(filter[1..], property.PropertyType);
        ConstantExpression constant = Expression.Constant(typedValue, property.PropertyType);

        Expression comparison = filter[..1] switch
        {
            ">" => Expression.GreaterThan(member, constant),
            "<" => Expression.LessThan(member, constant),
            "=" => Expression.Equal(member, constant),
            _ => throw new ArgumentException(/* ... */)
        };

        body = body is null ? comparison : Expression.AndAlso(body, comparison);
    }

    body ??= Expression.Constant(true);
    return Expression.Lambda<Func<T, bool>>(body, parameter);
}
```

Verified:

```
Filters supplied at runtime: Age>21, Country=IN

Expression built from those filters:
  x => ((x.Age > 21) AndAlso (x.Country == "IN"))

Matches:
  Asha (29, IN)
  Deepa (41, IN)
```

Note `Convert.ChangeType` then `Expression.Constant(typedValue, property.PropertyType)`: typing the constant correctly is what keeps the comparison valid. An `int` property compared against a `string` constant throws when you build the node — immediately, with a clear message.

Why an `Expression` and not a `Func`? Because the same object can either be compiled and run in memory, or handed to EF Core and turned into a SQL `WHERE` clause (§3.5). A `Func` could only ever do the first.

8.6 Rewriting with `ExpressionVisitor`

Trees are **immutable**. A visitor does not edit; it produces a new tree.

```
internal sealed class ConstantRaiser(int delta) : ExpressionVisitor
{
    protected override Expression VisitConstant(ConstantExpression node)
    {
        if (node.Type == typeof(int) && node.Value is int value)
        {
            return Expression.Constant(value + delta, typeof(int));
        }

        return base.VisitConstant(node);
    }
}
```

Verified:

```
Original : c => ((c.Age > 18) AndAlso (c.Country == "IN"))
Rewritten: c => ((c.Age > 21) AndAlso (c.Country == "IN"))   (every int constant raised by 3)

Nithya is 20 and in IN:
  original (Age > 18) -> True
  rewritten (Age > 21)-> False
```

A read-only visitor is just as useful:

```
  node census of the original tree:
    Parameter x 3
    MemberAccess x 2
    Constant x 2
    Lambda x 1
    AndAlso x 1
    GreaterThan x 1
    Equal x 1
```

`Parameter x 3` for a lambda with one parameter looks wrong but is correct: the single `ParameterExpression` object appears three times in the traversal — once in the lambda's parameter list and once inside each of the two `MemberAccess` nodes (`c.Age`, `c.Country`). A tree is a **graph with shared nodes**, not a strict tree, and a visitor walks each edge. Worth knowing before you write a visitor that counts or mutates.

`ExpressionVisitor` **is the standard way to process trees. EF Core's query pipeline is a long chain of visitors, each rewriting the tree a little closer to something that can be printed as SQL.**

8.7 `IQueryable` — why LINQ providers need expressions

```
// IEnumerable<T>.Where takes Func<T, bool>  — LINQ to Objects just runs it.
IEnumerable<Customer> asEnumerable = customers.Where(c => c.Age > 18 && c.Country == "IN");

// IQueryable<T>.Where takes Expression<Func<T, bool>> — the provider gets the TREE.
IQueryable<Customer> asQueryable = customers.AsQueryable()
    .Where(c => c.Age > 18 && c.Country == "IN")
    .OrderBy(c => c.Name);
```

Verified:

```
IQueryable<Customer>.Where(...).OrderBy(...)
  Where's parameter type: Expression<Func<Customer, bool>>
  nothing has executed yet. What exists is an expression tree:

  Metaprogramming.ExpressionTreesDemo.Customer[]
    .Where(c => ((c.Age > 18) AndAlso (c.Country == "IN")))
    .OrderBy(c => c.Name)

  provider: EnumerableQuery`1
  element : Customer
  result (on enumeration): Asha, Deepa
```

Swap the provider and the same tree has a different destiny:

Provider	What it does with the tree
`EnumerableQuery` (LINQ to Objects)	`Compile()` it and run it in memory
EF Core	translate it to **SQL**, run it in the database
an OData / HTTP provider	translate it to a **URL query string**

That substitutability is the entire reason LINQ providers can exist, and it works only because the query arrived as DATA rather than as compiled code.

8.8 The limits — two different failures, often confused

A. Not every C# construct can appear in an `Expression<TDelegate>`. The compiler **rejects** these at build time inside an expression lambda:

- statement bodies: `x => { var y = x * 2; return y; }`
- assignment: `x => x = 5`
- `await`
- `ref`/`out` arguments, unsafe code, lambdas using `dynamic`

- null-propagating `?.` and some pattern-matching forms

These are **compile errors**, not runtime failures. Note the precision: the underlying `System.Linq.Expressions` **API** *can* represent blocks and assignment (`Expression.Block`, `Expression.Assign`) — it is the **C# lambda syntax** that cannot.

B. Not every valid expression tree can be translated by a given provider. A separate, later failure, surfacing at **run time**:

```
customers.AsQueryable().Where(c => IsVipCustomer(c.Name)) ->
  LINQ to Objects ran it happily: 1 match(es).
  EF Core would throw InvalidOperationException: 'could not be translated'.
```

And EF Core really does (§3.7):

```
The LINQ expression '...Where(p => Program.IsProofRead(p.Content))' could not be translated
```

> ***The distinction worth teaching: "cannot be EXPRESSED as a tree" → a C# compile error. "cannot be TRANSLATED to SQL" → a provider exception at run time.*** *Different failures, different fixes — and easy to confuse, because both get described as "expression trees do not support that".*

C. `Compile()` is slow and needs dynamic code support. **D.** A tree is immutable: every "change" allocates a new tree. **E.** Debugging a compiled expression gives you no source to step through.

8.9 Audience questions

Q: What is the difference between reflection and expression trees? Reflection inspects **declarations** — what members exist. Expression trees represent **code** — what an operation does. Reflection tells you `Customer` has an `Age` property; an expression tree tells you someone wrote `c.Age > 18`. They are often used together: §8.5 uses reflection (`GetProperty`) to build the tree.

Q: Expression trees or Reflection.Emit? Expression trees, unless you need a *type*. Same speed class, type-checked as you build, printable, and they fail loudly rather than silently (Part 3 §8).

Q: Why is `IQueryable`'s `Where` different from `IEnumerable`'s? Only in the parameter type: `Expression<Func<T,bool>>` vs `Func<T,bool>`. That single difference is what lets a provider translate instead of execute — and why an accidental `.ToList()` changes where your filtering happens (§3.6).

Q: Can I convert a `Func` back into an `Expression`? No. The delegate is compiled IL; the structure is gone. This is why EF Core requires `Expression<Func<...>>` and why `Compile()` is a one-way door.

8.10 Documentation

- Expression trees (C# guide) — https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/expression-trees/
- `Expression<TDelegate>` — https://learn.microsoft.com/en-us/dotnet/api/system.linq.expressions.expression-1
- `ExpressionVisitor` — https://learn.microsoft.com/en-us/dotnet/api/system.linq.expressions.expressionvisitor
- `IQueryable` — https://learn.microsoft.com/en-us/dotnet/api/system.linq.iqueryable

---

Part 4 summary

1. **ASP.NET Core MVC** discovers controllers and actions by reflection at **startup**, then serves requests from a cached route table and cached invokers.
2. **DI** reads constructor signatures as dependency lists. Lifetimes matter; the greediest *satisfiable* constructor wins; `ActivatorUtilities` bridges DI and runtime arguments. **"DI uses reflection" is the safe claim** — the activation strategy varies by container, version and platform.
3. **EF Core** builds a **separate, richer model** by reading CLR metadata, then translates **expression trees** into provider-specific SQL. Shadow properties prove the model is not just a view over reflection.

4 **Serialization** is the canonical case: reflection mode and source-generated mode produce **byte-identical** output; the difference is *when* the work happened.

5 **Plugins** (Part 2 §9): `AssemblyLoadContext` isolates **type identity**, not **privileges**.

6 `DispatchProxy` packages Part 3's generated-type pattern. Interfaces only; self-calls are not intercepted; async needs care.

7 **Cross-cutting concerns**: choose the seam by where the concern belongs. A hand-written decorator is right more often than a generated proxy. **Metaprogramming supplies the seam, never the policy.**

8 **Expression trees** are code as data. Readable, type-checked, translatable — and the reason `IQueryable` can target a database. Prefer them to IL whenever you need a function.

And the thread running through all eight: DISCOVER ONCE, CACHE, THEN EXECUTE COMPILED CODE.

Part 2 · Part 3 · Quick Revision · Interview Questions

Quick Revision Notes

Read this in the 15 minutes before you present. Nothing new here — only what you must have at your fingertips.

The one sentence, if you forget everything else

> ***A .NET assembly is self-describing. Metaprogramming reads that description (Part 2) or writes new code from it (Part 3). Every framework does it ONCE at startup, caches the result, and then runs compiled code.***

One-sentence definitions

Term	Definition
Metadata	The compiler-emitted description of every type and member, stored in the assembly itself.
Metaprogramming	Code that operates on program structure or behaviour instead of on data.
Introspection	Reading metadata at run time.
Reflection	The .NET API for introspection and reflective invocation.
Reflection.Emit	The API for generating new IL at run time.
CIL / MSIL / IL	The portable, stack-based instruction set every .NET language compiles to.
Evaluation stack	The stack every IL instruction pushes to and pops from.
JIT	The compiler that turns IL into machine code, per method, at first call.
Attribute	A class deriving from `Attribute`; applying it stores inert declarative data on a declaration.
Expression tree	Code represented as an inspectable, immutable object graph.
Source generator	A Roslyn component that adds C# source during the build.
Shadow property	A property in the EF Core model with no corresponding CLR property.
Dynamic proxy	A runtime-generated type implementing an interface and routing every call through one method.
AssemblyLoadContext	An isolation boundary for assembly loading and type identity — **not** a security sandbox.

API cheat-sheet

Getting a `Type`

```
typeof(Customer)                 // compile time, checked, fastest — PREFER THIS
instance.GetType()               // runtime type of an object
Type.GetType("Ns.Customer")      // from a string; needs ASSEMBLY-QUALIFIED name for other assemblies
assembly.GetType("Ns.Customer")  // look in one specific assembly
```

Inspecting

```
assembly.GetTypes()              // all types (throws ReflectionTypeLoadException — catch it!)
type.GetMethods() / GetProperties() / GetFields() / GetConstructors()
type.GetMethod("Name", [typeof(int)])    // specify signature for overloads
type.IsAssignableFrom(other)     // use this, NOT ==
property.GetValue(obj) / SetValue(obj, v)
method.Invoke(obj, [args])       // wraps exceptions in TargetInvocationException
```

`BindingFlags`

```
Public | Instance          // the common case
NonPublic | Instance       // private/protected members
Public | Static            // static members
DeclaredOnly               // exclude inherited
```

Default with no flags = `Public | Instance | Static`**.** A "missing" member is almost always a `BindingFlags` bug.

Creating

```
Activator.CreateInstance<T>()                    // needs public parameterless ctor
Activator.CreateInstance(type, arg1, arg2)       // ctor chosen by argument types
Activator.CreateInstance(type, nonPublic: true)  // reaches a private ctor
ctor.Invoke([args])                              // explicit, and cacheable
ActivatorUtilities.CreateInstance<T>(provider, extraArgs)  // DI + your arguments
ActivatorUtilities.CreateFactory(type, argTypes)           // cache this on hot paths
```

Attributes

```
member.IsDefined(typeof(T))            // cheapest — constructs nothing
member.GetCustomAttribute<T>()         // one; THROWS AmbiguousMatchException if several
member.GetCustomAttributes<T>()        // all; use when AllowMultiple = true
member.GetCustomAttributesData()       // raw rows, NOTHING constructed — safe for untrusted
```

Generating

```
var dm = new DynamicMethod("Add", typeof(int), [typeof(int), typeof(int)]);
ILGenerator il = dm.GetILGenerator();
il.Emit(OpCodes.Ldarg_0);
il.Emit(OpCodes.Ldarg_1);
il.Emit(OpCodes.Add);
il.Emit(OpCodes.Ret);
var add = dm.CreateDelegate<Func<int, int, int>>();
```

Expression trees

```
Expression<Func<int, bool>> e = n => n > 18;
e.Body.NodeType                        // GreaterThan
Expression.Lambda<Func<int,int,int>>(body, a, b)
e.Compile()                            // EXPENSIVE — cache the delegate
```

Reflection vs Reflection.Emit

	Reflection	Reflection.Emit
Purpose	**inspect** and invoke	**create** code
Namespace	`System.Reflection`	`System.Reflection.Emit`
Cost	slow **per call**	slow **once**, then near-direct
Complexity	low	high
Type safety	runtime-checked, throws clearly	**not always checked at all**
Native AOT	mostly works	**does not work**
Use when	you need to *know* something	you need to *do* something, fast, repeatedly

They are partners: reflection *finds* the target, Emit *bakes it in*.

Compile-time vs runtime

	Source generators	Reflection	Reflection.Emit
Runs at	**build**	run	run
Output	C# source	information	IL
Runtime cost	**zero**	per call	once, then ~direct
Type-checked	**yes**	no	no
Native AOT	**yes**	partly	**no**
Trimming	safe	needs annotation	unsafe
Types unknown at build time	**no**	yes	yes

`DynamicMethod` workflow

```
1. new DynamicMethod(name, returnType, parameterTypes)
2. GetILGenerator()
3. Emit(...) one instruction at a time
4. CreateDelegate<TDelegate>()
5. CACHE THE DELEGATE  ← the step people forget
```

Runtime type-generation workflow

```
1. AssemblyBuilder.DefineDynamicAssembly(name, AssemblyBuilderAccess.Run)
2. .DefineDynamicModule("MainModule")
3. .DefineType("Ns.MyType", TypeAttributes.Public | Class, typeof(object))
4. .AddInterfaceImplementation(typeof(IKnownInterface))   ← the key design move
5. .DefineField / .DefineConstructor / .DefineMethod / .DefineProperty
6. GetILGenerator() on each, Emit the bodies
7. .CreateType()           ← bakes it; nothing can be added after
8. Activator.CreateInstance(created, args), then CAST TO THE INTERFACE
```

To save it: `new PersistedAssemblyBuilder(name, coreAssembly)` … `.Save(path)`.

IL instructions you must know

Instruction	Stack effect
`ldarg.0` / `ldarg.1`	push argument 0 / 1
`ldc.i4.N` / `ldc.i4.s n` / `ldc.i4 n`	push an int constant (1 / 2 / 5 bytes)
`add` / `sub` / `mul` / `div`	pop 2, push 1
`call`	pop args, push result — target fixed at emit time
`callvirt`	same, but virtual dispatch **and a null check**
`newobj ctor`	pop args, allocate + construct, push the reference
`ldfld` / `stfld`	load / store an instance field
`castclass T`	checked cast (throws); `isinst` is the `as` version
`box` / `unbox.any`	value type ↔ heap object
`ret`	pop the return value and return

The Add method, in full

```
ldarg.0     [ a ]
ldarg.1     [ a, b ]
add         [ a+b ]
ret         [ ]
```

The two rules

1. **Balance.** At `ret` the stack holds exactly the return value.
2. **Types.** Operand types must suit the opcode.

■ The `ldarg` trap

- **Static** method → `ldarg.0` is the **first argument**.
- **Instance** method → `ldarg.0` is `this`; the first argument is `ldarg.1`.

Verified .NET 10 facts (checked on this machine — not recalled)

Fact
`AssemblyBuilderAccess` has **only** `Run` and `RunAndCollect`. `Save`/`RunAndSave` are **.NET Framework only** and do not exist.
Saving is done with `PersistedAssemblyBuilder` (in the shared framework, no package). Verified: wrote a 2 KB DLL, reloaded it, invoked it → 42.
A missing `ret` → `InvalidProgramException` **on first call**, not at `Emit`.
`add` **on an int and a string produced NO error** — it silently returned garbage. Fully-trusted `DynamicMethod` IL is **not type-verified**.
`ServiceProvider`'s engine here is `DynamicServiceProviderEngine` — so say "DI uses reflection", not "DI uses Reflection.Emit".
A `DispatchProxy` base class **must not be** `sealed`; .NET 10 also has `DispatchProxy.Create(Type, Type)`.
SQLite renders `string.Contains` as `instr(...) > 0`, not `LIKE`. The *provider* decides.
`JsonSerializerOptions`: new-but-identical = 1.4×; new-and-**different** = **7.2×**. Equivalent instances share a process-wide cache.

Measured performance (Release, 2,000,000 property reads)

```
direct C# property access                    8 ms    1.0x
PropertyInfo.GetValue                       67 ms    8.4x
MethodInfo.Invoke                           59 ms    7.4x
cached delegate (typed)                     10 ms    1.2x
emitted IL (typed, no box)                  10 ms    1.2x
emitted IL (object->object, boxes)          21 ms    2.6x
```

Four takeaways:

1. Reflection ≈ 7–8× per call — but 67 ms for **two million** reads is ~33 ns each.
2. A **cached delegate is as fast as emitted IL**. Try `MethodInfo.CreateDelegate` **before** writing any IL.
3. **Boxing and signature shape dominate** — rows 5 vs 6 are the same technique.
4. Generation is not free. One call → reflection wins.

Real-world use cases

Framework	What it reflects	When
ASP.NET Core MVC	controllers, actions, routing attributes	**startup**
DI container	constructor parameters	startup + first resolve
EF Core	`DbSet<T>`, annotations, fluent config	**first use**, then cached
`System.Text.Json`	properties and attributes	**first use** per type, cached
xUnit / NUnit	`[Fact]` / `[Test]` methods	test discovery

Framework	What it reflects	When
Moq / Castle	generates proxy **types**	first use
Plugin hosts	types implementing a contract	on load

Common limitations

Limitation	Detail
No compile-time safety	a typo in a string is a run-time failure
Slow per call	unless you cache — and every framework caches
Native AOT	`Reflection.Emit`, `DispatchProxy`, `Expression.Compile()` all unavailable
Trimming	the trimmer cannot see reflective usage; annotate or do not trim
Debuggability	opaque frames, no source for generated code
Unverified IL	some invalid IL silently produces garbage
`private` **is not security**	`BindingFlags.NonPublic` reads it, legitimately
ALC is not a sandbox	isolates type identity, not privileges
Unload is cooperative	one cached `Type` keeps the context alive forever
Proxy self-calls	a service calling its own method bypasses the proxy
Translation ≠ expressibility	compile error vs provider runtime error are different failures

---

The decision order (say this on slide 28)

```
1. Plain C#, generics, or a cached delegate      ← covers most cases
2. Expression trees                              ← if you need a FUNCTION
3. DispatchProxy                                 ← if you only need to intercept an INTERFACE
4. A Roslyn source generator                     ← if the shape is known at BUILD time
5. Reflection.Emit                               ← only if you need a TYPE, known only at RUN time
```

Most people who reach for 5 needed 1 or 2.

Timing reminder

Part	Slides	Minutes	Cumulative
Opening	1–2	1:30	1:30
Part 1	3–7	5:00	6:30
Part 2	8–15	9:30	16:00
Part 3	16–23	11:30	27:30
Part 4	24–27	8:00	35:30
Closing	28	1:30	**37:00**

Checkpoints: Part 2 must START by 6:30. Part 3 must START by 16:00. Part 4 must START by 27:30. **If you are late at 16:00, cut Part 2's demo, not Part 3.**

The three lines you must not get wrong

1 **"**`AssemblyLoadContext` **is not a security sandbox."** It isolates type identity, not privileges.

2. **"`AssemblyBuilderAccess.Save` does not exist on .NET."** Use `PersistedAssemblyBuilder`.
3. **"DI uses reflection"** — not "DI uses Reflection.Emit". The engine is chosen at run time and falls back under AOT.

If someone asks something you do not know

> *"I don't want to guess at that — it's exactly the kind of thing that changes between versions. Let me check it against the docs and come back to you."*

Then actually do. That answer costs nothing and buys more credibility than a confident wrong one. There is a worked list in `Presenter-Guide/Audience-QA.md`.

Interview Questions

Technically accurate answers, with the depth an interviewer is usually probing for. Every factual claim about .NET 10 behaviour here was verified by running the code in `Code-Examples`.

Difficulty: ■ fundamentals · ■ working knowledge · ■ deep

Fundamentals

■ 1. What is metaprogramming?

Code that operates on program structure or behaviour rather than on data. In .NET it takes three forms: **compile-time generation** (Roslyn source generators), **runtime introspection** (Reflection) and **runtime code generation** (`Reflection.Emit`, expression trees).

Follow-up — why is it possible in .NET at all? Because every managed assembly is **self-describing**: the compiler emits metadata tables (`TypeDef`, `MethodDef`, `CustomAttribute`, …) alongside the IL, and the CLR needs them anyway. Reflection reads tables that are already there.

■ 2. What is reflection, and when would you use it?

The `System.Reflection` API for inspecting metadata and invoking members at run time.

Use it when **the set of types is not known when you write the code** — plugins, serializers, DI containers, test runners, ORMs.

Do **not** use it when the type is known. Generics, interfaces or a source generator are faster, type-checked and clearer.

■ 3. Difference between `typeof(T)`, `GetType()` and `Type.GetType(string)`?

	Resolved	Checked	Returns
`typeof(T)`	compile time	**yes**	the `Type` for a static type
`obj.GetType()`	run time	n/a	the **runtime** type of an instance
`Type.GetType("…")`	run time	no	the type, or `null`

They differ. If a variable is declared `object` but holds a `SavingsAccount`, `GetType()` returns `SavingsAccount`.

■ 4. What is an attribute? Does it do anything by itself?

A class deriving from `System.Attribute`. Applying it writes a row into the `CustomAttribute` metadata table.

It does nothing by itself. It is inert declarative data. `[JsonIgnore]` has no behaviour; `System.Text.Json` *looks for it* and decides to skip the property. If nothing reads an attribute, it is a comment the compiler stores.

This is the most common misconception about attributes and a very common interview question.

■ 5. `Activator.CreateInstance` vs `new`?

`new` is one `newobj` IL instruction with the constructor resolved at compile time: type-checked, fastest. `Activator.CreateInstance` searches constructors at run time, binds `object[]` arguments, then constructs: no type checking, much slower, and every failure is a run-time failure.

Use `Activator` **only** when the type genuinely is not known at compile time.

Working knowledge

■ 6. Is reflection slow?

Yes per call, and usually irrelevant in absolute terms. Measured here (Release, 2,000,000 reads of one `decimal` property):

```
direct C# property access                    8 ms    1.0x
PropertyInfo.GetValue                       67 ms    8.4x
MethodInfo.Invoke                           59 ms    7.4x
cached delegate (typed)                     10 ms    1.2x
emitted IL (typed, no box)                  10 ms    1.2x
```

So ~7–8×. But 67 ms for **two million** reads is about **33 nanoseconds** each. Once per web request: free. Per row of a million-row import: your bottleneck.

The real answer an interviewer wants: the cost is **per call**, and the fix is always the same — **do the reflection once, cache a delegate, then call compiled code**. That is what MVC, EF Core, `System.Text.Json` and the DI container all do.

Why is it slow? Per call it re-does member lookup, access checks, argument binding, boxing of arguments into `object[]`, and unboxing the result.

■ 7. Is `Reflection.Emit` faster than reflection?

The generated code is — roughly the same speed as direct access. **Generating it is not**: creating and JIT-ing a method costs far more than one reflective call.

So it wins only if you amortise over many calls (order of thousands).

The answer that shows judgement: in the measurement above, a **cached delegate from `MethodInfo.CreateDelegate` was exactly as fast as emitted IL** (both 1.2×). So try `CreateDelegate` first — you get emit-level speed with **no IL at all**. Reach for Emit only when the shape of the call is not fixed at compile time.

■ 8. Does .NET use reflection internally?

Extensively, and almost always at **startup or first use**, then cached:

Framework	Reflects over	When
ASP.NET Core MVC	controllers, actions, routing attributes	startup
DI container	constructor parameters	startup / first resolve
EF Core	`DbSet<T>` properties, annotations	first use, then cached
`System.Text.Json`	properties and attributes	first use per type, cached
xUnit / NUnit	`[Fact]` / `[Test]` methods	discovery
Moq / Castle	generates proxy **types**	first use

The important nuance: **it is used for *discovery*, not for *execution* on the hot path.** MVC does not call `MethodInfo.Invoke` per request; it builds a cached invoker at startup.

■ 9. Does dependency injection always use reflection?

No. Distinguish the pattern from the container.

- **Constructor injection** needs no reflection — you can wire an app by hand with `new`.
- **Containers** use reflection to read constructor signatures, so you do not have to wire it yourself.
- Some containers then **generate code** to call those constructors fast; some do not; compile-time DI containers use no reflection at all.

Verified for `Microsoft.Extensions.DependencyInjection` on .NET 10 by reading the provider's private `_engine` field:

```
ServiceProvider's internal engine on THIS runtime: DynamicServiceProviderEngine
```

It picks a strategy at run time: a reflection-based engine first, a faster expression-tree/IL-emitted engine once a service has been resolved enough times, and the reflection-only engine when `RuntimeFeature.IsDynamicCodeCompiled` is `false` (Native AOT).

> ***"DI uses reflection" is the safe claim. "DI uses Reflection.Emit" is not** — it depends on the container, its version and the platform.*

■ 10. Can reflection access private methods and fields?

Yes, with `BindingFlags.NonPublic`:

```
FieldInfo? f = type.GetField("_auditTrail", BindingFlags.NonPublic | BindingFlags.Instance);
f?.SetValue(instance, "changed");
```

And the important part of the answer:

> `private` ***is a compiler rule, enforced at compile time. Accessibility is an encapsulation feature, not a security boundary.***

Code in your process can read any field of any object. There is no supported in-process sandbox in modern .NET — Code Access Security and AppDomain partial trust were .NET Framework features and have been removed. If something must be protected from code in the same process, you need a process or OS boundary.

Legitimate uses: deserialization, ORM materialisation, test fixtures, diagnostics. **Illegitimate use:** routing around another library's invariants — it breaks on their next patch, because private members are part of no contract.

■ 11. Reflection vs expression trees?

	Reflection	Expression trees
Represents	**declarations** — what members exist	**code** — what an operation does
Example answer	"`Customer` has an `Age` property"	"someone wrote `c.Age > 18`"
Can execute	yes, slowly (`Invoke`)	yes, after `Compile()` — fast
Can be translated	no	**yes — this is how EF Core makes SQL**
Type-checked as built	n/a	**yes — factory methods throw immediately**

They are commonly used **together**: a dynamic filter builder uses reflection (`GetProperty`) to find the member, then builds an expression tree around it.

■ 12. `DynamicMethod` vs `MethodBuilder`?

	`DynamicMethod`	`MethodBuilder`
Declaring type	**none**	a `TypeBuilder`
Creates	one method	one method **of a type**
Fields, several methods, interfaces	no	**yes**
Collectible	**yes**	only with `RunAndCollect`
Savable	no	yes, via `PersistedAssemblyBuilder`
Ceremony	~5 lines	~60 lines
Right for	accessors, converters, serializer fast paths	**proxies, mocks** — anything that must *be* a type

The deciding question: **do you need a method, or a type?** A mock must *be* an `IFoo` to be passed where an `IFoo` is expected — only type generation can do that.

Deep

■ 13. What happens when generated IL is invalid?

It depends, and the asymmetry is the point. Both cases verified:

Case A — a missing `ret`:

```
CreateDelegate succeeded - nothing has been verified yet.
Calling it...
InvalidProgramException on FIRST CALL: Common Language Runtime detected an invalid program.
```

Not at `Emit`, not at `CreateDelegate`, but when the **JIT first compiles** the method. Your bug surfaces far from the line that caused it.

Case B — `add` on an `int` and a string reference:

```
No exception. broken(1) returned 299021057, and again 299021057.
```

No error at all. A plausible-looking garbage number — the string's managed reference added to 1 as an integer.

> ***IL emitted by a fully-trusted `DynamicMethod` is not type-verified.*** *A mistake can silently produce garbage instead of failing, and memory-unsafe IL can corrupt process state rather than throw.*

How to debug it: write the C# first and read its IL (ILSpy, sharplab.io); track expected stack depth as you emit; generate with `PersistedAssemblyBuilder`, `Save`, and run **ILVerify**; keep emitted methods tiny and `call` into real C#. Or use expression trees, where `Expression.Add(intNode, stringNode)` throws immediately.

■ 14. Difference between IL and machine code?

	IL	Machine code
Target	the CLR's virtual machine	a specific CPU
Model	**stack-based**	register-based
Type information	**full**	none
Portability	one assembly runs everywhere	per architecture
Produced by	the C#/F#/VB compiler	the **JIT**, or an AOT compiler

Why the extra step? One assembly runs on x64, Arm64, Windows, Linux and macOS, and the JIT can specialise for the actual CPU — using AVX-512 where available without you shipping a separate build. Being verifiable and type-rich also lets the runtime enforce memory safety.

Nuance worth adding: the pipeline is not always "IL → JIT". **ReadyToRun** pre-compiles at publish time with IL as a fallback; **tiered compilation** compiles a method *more than once* (tier-0 fast, tier-1 optimised for hot methods); **Native AOT** ships no JIT at all; the **Mono interpreter** interprets IL.

■ 15. Can `Reflection.Emit` work with Native AOT?

No. Under Native AOT there is no JIT, so there is nothing to compile generated IL. `RuntimeFeature.IsDynamicCodeSupported` is `false` and the `Reflection.Emit` APIs throw `PlatformNotSupportedException`.

Also unavailable: `DispatchProxy`, `Expression<T>.Compile()`, and any library depending on them (Moq, Castle DynamicProxy, reflection-mode `System.Text.Json` warns and may fail).

Two flags, and they are different:

```
RuntimeFeature.IsDynamicCodeSupported   // can I create code at all?
RuntimeFeature.IsDynamicCodeCompiled    // will it be JIT-compiled to machine code?
```

Platform	Supported	Compiled	Effect
CoreCLR	`true`	`true`	works, fast
Native AOT	`false`	`false`	**throws**
Mono interpreter (iOS)	`true`	`false`	works, but **interpreted** — no speed benefit

What to do instead: source generators. `[GeneratedRegex]`, `JsonSerializerContext`, `[LoggerMessage]` and the configuration binder generator all exist precisely to move this work to build time.

The library-author rule: check `IsDynamicCodeSupported` and keep a reflection-based fallback. That is exactly what `System.Text.Json` and the DI container do.

A precision point interviewers like: expression trees can still be **built and read** under AOT — which is why EF Core can translate LINQ to SQL there, since translation only reads the tree. It is `Compile()` that needs the JIT.

■ 16. Can a plugin be loaded without restarting the application?

Loading: yes. `AssemblyLoadContext` + `LoadFromAssemblyPath` + reflection discovery + activation. Verified in example 04.

Replacing or unloading: much harder. You need a **collectible** context (`isCollectible: true`), you must drop **every** reference, call `Unload()`, and then wait — because `Unload()` is a **request, not an action**. Verified:

```
Unload() has been requested. Now the GC has to actually collect it.
  collection attempt 1: alive = True
  collection attempt 2: alive = False
```

What keeps a context alive — all common mistakes:

- any live instance of a plugin type;
- a cached `Type`, `MethodInfo` or `Assembly`;
- a delegate pointing at plugin code (an event handler left subscribed);
- a static field in the **host** holding any of the above.

The example needs `[MethodImpl(MethodImplOptions.NoInlining)]` on its helper, because an inlined method's locals could stay rooted and the context would never collect. That is how delicate it is.

Also: unloading frees **managed state only**. It cannot undo side effects, and a plugin that P/Invoked a native library may have pinned it.

What production systems usually do: run plugins in a **child process** and restart that. Far more robust than in-process unloading.

■ 17. Is `AssemblyLoadContext` a security boundary?

No — and this is the most important "no" in the whole topic.

> `AssemblyLoadContext` ***is an isolation boundary for TYPE IDENTITY and VERSIONS. It is NOT a security sandbox.***

Once you call `LoadFromAssemblyPath`, that code runs with your process's **full privileges**. It can read and write any file your process can, open sockets, start processes, read environment variables and secrets, reflect against your private state, and call `Environment.Exit`.

There is no supported in-process sandbox in modern .NET. CAS and AppDomain partial trust were .NET Framework features and are gone.

For untrusted code: isolate where the OS can enforce it — a separate process with a reduced token, a container, a VM, or a WASM runtime.

For a trusted plugin model, harden it anyway: verify a strong name or Authenticode signature before loading; keep the contracts assembly tiny and versioned; vet with `MetadataLoadContext` first (no plugin code runs); wrap every plugin call in `try`/`catch`; log the path, version and hash of everything loaded.

What ALC *is* good for — and the failure that proves it: if the plugin context loads its own copy of the shared contract assembly, the plugin implements a *different* `IReportPlugin` and the cast fails with an `InvalidCastException` whose message **names the same type twice**. Type identity in .NET is **(assembly identity + load context + full name)**.

■ 18. How does EF Core translate LINQ into SQL?

The query arrives as **data**, not code, because `IQueryable<T>.Where` takes `Expression<Func<T,bool>>`, not `Func<T,bool>`. That single type difference is the whole mechanism.

The pipeline:

1. LINQ operators build an **expression tree**; nothing executes.
2. EF runs a chain of `ExpressionVisitor`s: identify the query root; bind member accesses to **mapped** properties and navigations; convert navigations to joins; translate method calls to provider-specific SQL functions; apply global query filters; produce a relational expression tree.
3. The provider prints SQL.
4. Results are materialised with **cached accessor delegates** (compiled once per property, not `PropertyInfo.GetValue` per row).
5. The translation is **cached per query shape**, so a loop with different parameter values translates once.

Verified:

```
SELECT "b"."Title", "b0"."Title" AS "Blog", "a"."full_name" AS "Author"
FROM "blog_posts" AS "b"
INNER JOIN "Blogs" AS "b0" ON "b"."BlogId" = "b0"."Id"
LEFT JOIN "authors" AS "a" ON "b"."AuthorId" = "a"."AuthorKey"
WHERE "b"."Rating" >= 0 AND "b"."Rating" >= 4 AND instr("b0"."Title", '.NET') > 0
ORDER BY "b"."PublishedOn" DESC
```

Note: `string.Contains` became `instr(...) > 0`, a **SQLite** function — not `LIKE`. **The provider chooses the translation**, so always check with `ToQueryString()`.

Why some queries fail: EF **refuses** rather than silently downloading the table:

```
The LINQ expression '...Where(p => Program.IsProofRead(p.Content))' could not be translated
```

That refusal is deliberate — the silent version was older EF Core behaviour and caused production incidents.

Keep two failures apart:

- *"cannot be EXPRESSED as a tree"* → a **C# compile error** (statement bodies, `await`, assignment).
- *"cannot be TRANSLATED to SQL"* → a **provider exception at run time**.

■ 19. What are the limitations of dynamic proxies?

All verified with `DispatchProxy` on .NET 10:

1. **Interfaces only.** `ArgumentException: The type 'AccountService' must be an interface, not a class.` Castle DynamicProxy can subclass a class, but then only intercepts **virtual** members.
2. **The proxy base class must not be** `sealed` — the generated type derives from it. It also needs an accessible parameterless constructor.
3. **Only calls through the interface are intercepted.** Crucially, **a self-call inside the service is not intercepted**, because `this` is the real object, not the proxy. This blind spot breaks naive caching and surprises everyone once.
4. **Async needs deliberate handling.** `Invoke` returns `object?`; for an `async` method that is a `Task` returned immediately, so a naive stopwatch measures only how long it took to *create* the task. You must inspect the return type and handle `Task`, `Task<T>` and `ValueTask<T>`.
5. **Per-call cost** — every call goes through `targetMethod.Invoke`, the slow path.
6. **No Native AOT.**
7. **Debuggability** — stack traces run through a generated type; "go to definition" no longer reaches the code that runs.

And the exception detail that catches people: `targetMethod.Invoke` wraps target exceptions in `TargetInvocationException`. Let it escape and every `catch (TimeoutException)` upstream silently stops matching. Unwrap with `ExceptionDispatchInfo.Capture(ex.InnerException).Throw()` — not `throw ex.InnerException`, which resets the stack trace.

The senior answer: a **hand-written decorator** is right more often than a generated proxy. Twenty lines of ordinary C#: debuggable, trimmable, AOT-safe, obvious. Use a generated proxy when the number of interfaces makes hand-writing impractical.

Design judgement — the questions that separate levels

■ 20. When would you NOT use metaprogramming?

1. **The type set is closed** → generics or a source generator.
2. **It runs once** → generation costs more than it saves.
3. **One implementation** → a reflective "engine" with one caller is a method with extra steps.
4. **The team will not maintain it** → IL and visitor chains are write-mostly.
5. **You are shipping Native AOT** → much of it does not run.
6. **The failure mode matters** → a compile error at 3pm beats a `NullReferenceException` at 3am.

> ***Metaprogramming trades compile-time safety for run-time flexibility. Make that trade when you need the flexibility, and not one step sooner.***

■ 21. Source generators or reflection?

Shape known at	Use
build time	**source generator** — zero runtime cost, AOT-safe, type-checked
run time	reflection, or runtime generation

Example 09 shows both producing **byte-identical** JSON. The difference is *when* the work happened. The direction of travel in .NET is consistently build-time: regex, logging, JSON, configuration binding all now have generator-based alternatives, driven by Native AOT and startup time.

What generators cannot do: handle types that do not exist at build time (plugins), or depend on run-time configuration.

■ 22. Your API endpoint is slow and the profiler shows reflection. What do you do?

A good answer walks the diagnosis, not straight to a fix:

1. **Where** is it — startup or per request? Startup reflection is normal and amortised.
2. **How many calls?** 33 ns each; if it is 10 calls per request, it is not your problem and the profiler is pointing at a symptom.
3. **Is it uncached?** The usual real culprit: `GetProperties()` in a loop, a new `JsonSerializerOptions` per call, or `Activator.CreateInstance` per item.
4. **Fix in this order:** cache the `*Info` objects → cache a **delegate** (`CreateDelegate`) → expression tree → source generator. Emit almost never.
5. **Check for boxing.** Measured: the same emitted getter was **2.6× vs 1.2×** purely because its signature boxed a `decimal`. Signature shape often dominates.
6. **Re-measure** with BenchmarkDotNet, not `Stopwatch` in a loop.

And the senior instinct: confirm reflection is actually the bottleneck. More often the real cost is an N+1 query, a missing index, or JSON over-serialization — and reflection is simply the most *visible* frame in the profile.

Glossary

Terms used in this session, defined precisely. Where a term is commonly misused, the misuse is called out.

A

Activator — `System.Activator`. Creates an instance from a `Type` object, choosing a constructor by matching the supplied arguments. The bridge from "a `Type` I discovered" to "a live object". Slower than `new` and type-checked only at run time.

ActivatorUtilities — `Microsoft.Extensions.DependencyInjection.ActivatorUtilities`. Like `Activator`, but resolves constructor parameters from an `IServiceProvider` while letting you supply the rest. Use `CreateFactory` to resolve the plan once and reuse it.

Advice *(AOP)* — the code an aspect runs at a join point: before, after, or around the intercepted call. `CachingProxy.Invoke` is around-advice.

AOP (Aspect-Oriented Programming) — organising cross-cutting concerns (logging, caching, authorization) into separate **aspects** instead of scattering them through business code. In .NET, implemented with middleware, filters, proxies, decorators or IL weaving.

Application part — an ASP.NET Core MVC abstraction for a source of controllers, usually an assembly. The **search path** for discovery; adding one lets MVC find controllers in another DLL.

Assembly — the unit of deployment, versioning and identity in .NET: a `.dll` or `.exe` containing IL, metadata and a manifest. One or more modules.

AssemblyBuilder — `System.Reflection.Emit.AssemblyBuilder`. Defines an in-memory dynamic assembly. **On .NET 10 it cannot save to disk** — use `PersistedAssemblyBuilder`.

AssemblyBuilderAccess — the enum controlling a dynamic assembly's capabilities. **On .NET 10 it has exactly two members: `Run` and `RunAndCollect`.** `Save` and `RunAndSave` were .NET Framework members and **do not exist**; code using them does not compile.

AssemblyLoadContext (ALC) — an isolation boundary for assembly loading. Lets two versions of a library coexist, and (when `isCollectible: true`) lets assemblies be unloaded. ■ **Not a security sandbox.** It isolates **type identity and versions**, not **privileges**. Loaded code runs with the host's full rights.

Assembly-qualified name — a type name including its assembly: `System.Text.Json.JsonSerializerOptions, System.Text.Json`. **Required** by `Type.GetType` for any type outside the calling assembly and `System.Private.CoreLib` — the most common cause of an unexpected `null`.

Attribute — a class deriving from `System.Attribute`. Applying it stores a row in the `CustomAttribute` metadata table. ■ **An attribute does nothing by itself.** It is inert declarative data until something reads it.

AttributeTargets — `[Flags]` enum listing where an attribute may be applied (`Class`, `Method`, `Property`, …). Violating it is a **compile error**.

AttributeUsage — the attribute applied *to* an attribute class, declaring `ValidOn`, `AllowMultiple` and `Inherited`. Defaults (`All`, `false`, `true`) are rarely what you meant, so declare it explicitly.

B

Backing field — the compiler-generated field behind an auto-property. Reflection shows it with an unspeakable name like `<AnnualRate>k__BackingField`, deliberately chosen so your code cannot collide with it.

BindingFlags — the enum filtering which members a reflection lookup returns. **Default with no flags is `Public | Instance | Static`.** A "missing" member is almost always a `BindingFlags` mistake. You must specify at least one of `Public`/`NonPublic` *and* one of `Instance`/`Static`.

box / unbox — IL instructions converting a value type to a heap object and back. Boxing allocates, and in measurements it frequently dominates the cost people attribute to reflection.

C

call / callvirt — IL call instructions. `call` fixes the target at emit time (static and non-virtual methods); `callvirt` resolves from the object's runtime type **and null-checks the receiver** (virtual and interface methods). Using `call` where `callvirt` is needed silently invokes the wrong override.

Captive dependency — a longer-lived service holding a shorter-lived one, classically a singleton capturing a scoped service. Detected by `ServiceProviderOptions.ValidateScopes`.

CIL (Common Intermediate Language) — the stack-based instruction set every .NET language compiles to, specified in ECMA-335. Also called **MSIL** or just **IL**; the three names mean the same thing.

CLR (Common Language Runtime) — the .NET execution engine: loading, JIT compilation, garbage collection, type safety, exceptions.

Collectible — an assembly or load context that can be garbage-collected and unloaded. Requires `AssemblyBuilderAccess.RunAndCollect` or `AssemblyLoadContext(isCollectible: true)`.

Convention (EF Core) — a rule EF applies automatically when nothing configured a detail (a property named `Id` becomes the key). Lowest precedence: **convention < data annotation < fluent API**.

Cross-cutting concern — a requirement spanning many modules — logging, caching, authorization, metrics, retries — orthogonal to business logic but easily tangled into it.

CustomAttributeData — `GetCustomAttributesData()` returns the raw attribute metadata rows **without constructing the attributes**. Use it for untrusted assemblies (no constructor code runs) and tooling.

D

Decorator — an object implementing an interface and wrapping another implementation of it, adding behaviour. A **hand-written decorator is usually preferable to a generated proxy**: debuggable, trimmable, AOT-safe, obvious.

DispatchProxy — `System.Reflection.DispatchProxy`. Generates a type at run time that implements an interface and routes **every** call through your `Invoke` override. ■ Interfaces only; the base proxy class must not be `sealed`; self-calls inside the target are **not** intercepted; async needs explicit handling; unavailable under Native AOT.

DynamicMethod — a single method with **no declaring type**, created at run time. The cheapest form of runtime code generation; collectible; cannot be saved; its IL is **not type-verified** when fully trusted.

E

ECMA-335 — the standard specifying the Common Language Infrastructure: IL, the metadata format, and the PE layout.

Evaluation stack — the stack every IL instruction operates on. Two rules: it must hold exactly the return value at `ret`, and operand types must suit the opcode.

Expression tree — code represented as an immutable, inspectable object graph (`Expression<TDelegate>`). Can be read, rewritten, compiled, or **translated** (to SQL, to a URL). The reason `IQueryable` can target a database. ■ Technically a **graph with shared nodes**, not a strict tree — one `ParameterExpression` object appears at every use site.

ExpressionVisitor — the base class for walking or rewriting an expression tree. Trees are immutable, so a visitor returns a **new** tree. EF Core's query pipeline is a chain of visitors.

F – I

Fluent API (EF Core) — configuring the model in `OnModelCreating` instead of with attributes. **Highest precedence.** Keeps framework types out of domain classes and expresses things attributes cannot (attribute arguments must be compile-time constants).

ILGenerator — the writer that appends IL instructions to a `DynamicMethod` or `MethodBuilder` body, via `Emit(OpCode, …)`.

ILVerify — a .NET tool that validates the IL in an assembly. Pair it with `PersistedAssemblyBuilder` to check emitted IL properly, instead of waiting for `InvalidProgramException` on first call.

Implicit vs explicit metadata — a **teaching distinction**, not a CLR category, about *who* produced the metadata: the compiler (implicit, from your declarations) or you (explicit, via attributes). The CLR has only metadata tables, one of which is `CustomAttribute`.

Introspection — reading a program's own structure at run time. In .NET, reflection.

InvalidProgramException — thrown when the JIT finds invalid IL, **on first call**, not when you emit it.

IQueryable<T> — a query whose operators take `Expression<Func<…>>` rather than `Func<…>`, so a provider receives the query as **data** and can translate it. That single type difference is the whole LINQ-provider mechanism.

J – M

JIT (Just-In-Time compiler) — compiles IL to machine code per method, at first call. Why generated IL runs at full speed, and why Native AOT (no JIT) cannot run it. Note that **tiered compilation** means a method may be compiled more than once.

Join point *(AOP)* — a place where an aspect could apply, typically a method call.

ldarg — the IL instruction family that pushes an argument. ■ **In a static method `ldarg.0` is the first argument; in an instance method it is `this` and the first argument is `ldarg.1`.** The most common beginner Emit bug.

ldc.i4 — the IL family that pushes an `int` constant, in 1-, 2- or 5-byte encodings (`ldc.i4.0`…`ldc.i4.8`, `ldc.i4.s`, `ldc.i4`). All push the same value.

Manifest — the part of an assembly's metadata recording its own identity, its referenced assemblies and their expected versions, and its resources.

MemberInfo — the abstract base of `Type`, `MethodInfo`, `ConstructorInfo`, `PropertyInfo`, `FieldInfo` and `EventInfo`. Why they all have `Name`, `DeclaringType` and `GetCustomAttributes`.

Metadata — the structured binary description of every type and member, emitted by the compiler into the assembly. **Required, not optional** — the CLR needs it. This is what makes .NET assemblies self-describing and reflection possible.

MetadataLoadContext — a reflection-like API for inspecting assemblies **without executing any of their code**. The right tool for vetting a plugin before loading it.

Metaprogramming — code that operates on program structure or behaviour rather than on data.

Middleware — an ASP.NET Core pipeline component handling every HTTP request. The right seam for concerns about *the request* (auth, CORS, request logging) rather than about a particular method.

Module — a physical container of types within an assembly. Multi-module assemblies are legal but essentially unused in modern .NET, so assembly ≈ module in practice.

MSIL — see **CIL**.

N – P

Native AOT — publishing a .NET app compiled fully ahead of time, with no JIT. ■ **`Reflection.Emit`, `DispatchProxy` and `Expression<T>.Compile()` do not work.** `RuntimeFeature.IsDynamicCodeSupported` is `false`. Note that

expression trees can still be **built and read** under AOT — only `Compile()` needs the JIT.

newobj — the single IL instruction that allocates an object, runs its constructor with the popped arguments, and pushes the reference. What `new`, `Activator`, a DI container and a deserializer all ultimately perform.

NonAction — `[NonAction]` excludes a public controller method from routing. Needed because a public method on a controller is an action **by default**.

PE file (Portable Executable) — the Windows executable format. A managed assembly is a PE file with extra sections holding the CLI header, metadata streams and IL.

PersistedAssemblyBuilder — `System.Reflection.Emit.PersistedAssemblyBuilder`. **How you save a generated assembly on modern .NET.** In the shared framework (no package needed). `Save(string)` / `Save(Stream)`. For **writing**, not running — save, then load the file. Verified here: wrote a 2 KB DLL, reloaded it, invoked it.

Pointcut *(AOP)* — the rule selecting which join points an aspect applies to. In example 12, the `[Cacheable]` attribute is the pointcut.

Property — ■ **not a storage location.** Metadata binding a name to getter and/or setter **methods**. Those accessors are real methods, flagged `IsSpecialName`.

Proxy — an object standing in for another, implementing the same interface and forwarding calls, with behaviour added around them.

R – S

ReadyToRun (R2R) — AOT compilation at publish time with IL kept as a fallback, so the JIT can still re-optimise. Improves startup without the restrictions of Native AOT.

Reflection — the `System.Reflection` API for reading metadata and invoking members at run time.

Reflection.Emit — the `System.Reflection.Emit` API for generating new IL at run time.

ReflectionTypeLoadException — thrown by `Assembly.GetTypes()` when any type fails to load. ■ Its `.Types` property still holds the ones that **did** load. Any host scanning assemblies it did not build must catch this.

ret — the IL instruction that pops the return value (if any) and returns. **Omitting it produces invalid IL**, discovered on first call.

RuntimeFeature.IsDynamicCodeCompiled — whether dynamically created code will be **JIT-compiled**. `false` on Mono interpreter builds: Emit *works* but runs interpreted, so the speed benefit disappears.

RuntimeFeature.IsDynamicCodeSupported — whether runtime code generation is possible at all. `false` under Native AOT. **Library code that emits IL should check this and keep a reflection fallback.**

Service lifetime — how long a DI container reuses an instance: **singleton** (one per provider), **scoped** (one per scope — one per HTTP request in ASP.NET Core), or **transient** (a new one every resolution).

Shadow property — an EF Core property that exists **in the model but not on the CLR class**, read and written through the change tracker by name. The cleanest proof that the EF model is a separate, richer model rather than a view over reflection.

Source generator — a Roslyn component that adds C# source **during the build**. Zero run-time cost, fully type-checked, trimming- and AOT-safe. ■ Does not exist at run time, and cannot handle types unknown at build time.

T – W

TargetInvocationException — the wrapper around any exception thrown *inside* a reflectively invoked member. ■ **Always unwrap it**, or every `catch` of the real exception type upstream silently stops matching. Use `ExceptionDispatchInfo.Capture(ex.InnerException).Throw()` — not `throw ex.InnerException`, which resets the stack trace.

Tiered compilation — the runtime compiling a method quickly first (tier-0) and re-compiling hot methods optimised (tier-1). Why benchmark warm-up loops matter.

Trimming — removing unused IL to shrink a deployment. ■ The trimmer **cannot see reflective usage**, so it may remove members your reflection needs. Annotate with `DynamicallyAccessedMembers`, use source generators, or do not trim.

TypeBuilder — the builder that defines a type at run time. `CreateType()` "bakes" it: before that call it cannot be instantiated; after it, nothing can be added.

Type identity — what makes two types the same to the CLR: **assembly identity + load context + full name**. Not just the name — which is why a duplicated contract assembly produces an `InvalidCastException` naming the same type twice.

Weaving *(AOP)* — combining aspects with code: at compile time, post-compile (IL weaving, as PostSharp/Fody do), or at run time (proxies). `DispatchProxy.Create` is run-time weaving.

Symbols you will see in reflection output

Appearance	Meaning
`List\1``	a generic type with 1 type parameter (backtick + arity)
`<AnnualRate>k__BackingField`	the compiler's backing field for an auto-property
`<>c`, `<>c__DisplayClass0_0`	compiler-generated closure classes for lambdas
`<GetAll>d__3`	a compiler-generated async/iterator state machine
`b__1_0`	a compiler-generated lambda method
`Outer+Inner`	a nested type — note the `+`, not a `.`
`.ctor` / `.cctor`	an instance / static constructor
`get_Salary` / `set_Salary`	property accessor methods
`generatedProxy_1`	a `DispatchProxy`-generated type
`<In Memory Module>`	the module of a dynamic assembly

References

Official Microsoft documentation, grouped by session part.

Link verification. Every URL below was checked with an HTTP request while this package was produced (2026-11-10): all returned HTTP 200. Nothing here is a guessed or reconstructed URL. Anything that could not be verified would be labelled **UNVERIFIED** — and no entry in this file carries that label.

Documentation moves. If a link 404s later, search `learn.microsoft.com` for the API name; the API reference pages (`/dotnet/api/...`) are the most stable.

Foundations

Topic	Link
What's new in .NET 10	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
Managed code and the CLR	https://learn.microsoft.com/en-us/dotnet/standard/managed-code
Assemblies in .NET	https://learn.microsoft.com/en-us/dotnet/standard/assembly/

Part 1 — Understanding Metaprogramming

Topic	Link
Reflection and attributes (C# guide)	https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes/
Source generators overview (compile-time metaprogramming)	https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/source-generators-overview
Writing custom attributes	https://learn.microsoft.com/en-us/dotnet/standard/attributes/writing-custom-attributes

Part 2 — Runtime Introspection

Topic	Link
Reflection in .NET (concepts)	https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/reflection
`System.Reflection` namespace	https://learn.microsoft.com/en-us/dotnet/api/system.reflection
`Type`	https://learn.microsoft.com/en-us/dotnet/api/system.type
`Assembly`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.assembly
`MethodInfo`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.methodinfo
`PropertyInfo`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.propertyinfo
`ConstructorInfo`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.constructorinfo
`FieldInfo`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.fieldinfo
`BindingFlags`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.bindingflags

Topic	Link
`Activator.CreateInstance`	https://learn.microsoft.com/en-us/dotnet/api/system.activator.createinstance
`AttributeUsageAttribute`	https://learn.microsoft.com/en-us/dotnet/api/system.attributeusageattribute
`AttributeTargets`	https://learn.microsoft.com/en-us/dotnet/api/system.attributetargets
`CustomAttributeData` (read attributes without constructing them)	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.customattributedata
`MetadataLoadContext` (inspect assemblies without executing them)	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.metadataloadcontext

---

Part 3 — Runtime Code Generation

Topic	Link
Emitting dynamic methods and assemblies	https://learn.microsoft.com/en-us/dotnet/fundamentals/reflection/emitting-dynamic-methods-and-assemblies
`System.Reflection.Emit` namespace	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit
`DynamicMethod`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.dynamicmethod
`ILGenerator`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.ilgenerator
`OpCodes` (every IL instruction)	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.opcodes
`AssemblyBuilder`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilder
`AssemblyBuilderAccess` — **note it has only `Run` and `RunAndCollect`**	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.assemblybuilderaccess
`PersistedAssemblyBuilder` — **how you save a generated assembly on modern .NET**	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.persistedassemblybuilder
`TypeBuilder`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.typebuilder
`MethodBuilder`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.emit.methodbuilder
`RuntimeFeature.IsDynamicCodeSupported`	https://learn.microsoft.com/en-us/dotnet/api/system.runtime.compilerservices.runtimefeature.isdynamiccodesupported

> ***The single most important documentation note for this part.*** *Many tutorials show `AssemblyBuilderAccess.RunAndSave` followed by `assemblyBuilder.Save("Out.dll")`. Those members are **.NET Framework only** and do not exist on .NET Core / .NET 5+. The modern replacement is `PersistedAssemblyBuilder`. Example 06 in this package prints the live enum members so you can confirm this on your own machine.*

Part 4 — Real-World Applications

Plugins and assembly loading

Topic	Link
Understanding `AssemblyLoadContext`	https://learn.microsoft.com/en-us/dotnet/core/dependency-loading/understanding-assemblyloadcontext
`AssemblyLoadContext` API	https://learn.microsoft.com/en-us/dotnet/api/system.runtime.loader.assemblyloadcontext
Assembly unloadability (and what prevents it)	https://learn.microsoft.com/en-us/dotnet/standard/assembly/unloadability
Tutorial: create an app with plugin support	https://learn.microsoft.com/en-us/dotnet/core/tutorials/creating-app-with-plugin-support

Dependency injection

Topic	Link
Dependency injection in .NET	https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection
DI guidelines (including captive dependencies)	https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection-guidelines
`ActivatorUtilities`	https://learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinjection.activatorutilities

ASP.NET Core MVC

Topic	Link
Routing to controller actions	https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/routing
Application parts	https://learn.microsoft.com/en-us/aspnet/core/mvc/advanced/app-parts

Entity Framework Core

Topic	Link
EF Core documentation	https://learn.microsoft.com/en-us/ef/core/
Creating and configuring a model	https://learn.microsoft.com/en-us/ef/core/modeling/
How queries work (LINQ to SQL translation)	https://learn.microsoft.com/en-us/ef/core/querying/how-query-works
Client vs server evaluation (why translation fails)	https://learn.microsoft.com/en-us/ef/core/querying/client-eval

Serialization

Topic	Link
`System.Text.Json` overview	https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview
Source generation in `System.Text.Json`	https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/source-generation

Expression trees

Topic	Link
Expression trees (C# guide)	https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/expression-trees/
`Expression<TDelegate>`	https://learn.microsoft.com/en-us/dotnet/api/system.linq.expressions.expression-1
`ExpressionVisitor`	https://learn.microsoft.com/en-us/dotnet/api/system.linq.expressions.expressionvisitor
`IQueryable`	https://learn.microsoft.com/en-us/dotnet/api/system.linq.iqueryable

Dynamic proxies

Topic	Link
`DispatchProxy`	https://learn.microsoft.com/en-us/dotnet/api/system.reflection.dispatchproxy

---

Native AOT and trimming — the limits of all of the above

This is the section to read before shipping anything built on runtime metaprogramming.

Topic	Link
Native AOT deployment (see the **"Limitations of Native AOT deployment"** section)	https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/#limitations-of-native-aot-deployment
Native AOT diagnostics support	https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/diagnostics
Trim self-contained deployments	https://learn.microsoft.com/en-us/dotnet/core/deploying/trimming/trim-self-contained
Known trimming incompatibilities	https://learn.microsoft.com/en-us/dotnet/core/deploying/trimming/incompatibilities
Prepare libraries for trimming	https://learn.microsoft.com/en-us/dotnet/core/deploying/trimming/prepare-libraries-for-trimming
Fixing trim warnings	https://learn.microsoft.com/en-us/dotnet/core/deploying/trimming/fixing-warnings
`DynamicallyAccessedMembersAttribute` (tell the trimmer what reflection needs)	https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.codeanalysis.dynamicallyaccessedmembersattribute

---

Tools worth knowing

These are not Microsoft documentation pages, so they are listed separately and described rather than linked, to keep every URL in this file verified:

- **ILSpy** — open-source .NET decompiler. Open any assembly and read its IL or decompiled C#. The fastest way to learn what IL your C# actually produces, and the way to inspect an assembly saved with `PersistedAssemblyBuilder`. Available from the ILSpy project on GitHub and the Visual Studio Marketplace.
- **ildasm / ilasm** — the IL disassembler and assembler shipped with the .NET SDK tooling and the Windows SDK.
- **ILVerify** — a .NET tool (`dotnet tool install -g dotnet-ilverify`) that validates IL in an assembly. Pair it with `PersistedAssemblyBuilder` to check emitted IL properly instead of waiting for `InvalidProgramException` at run time.
- **BenchmarkDotNet** — the correct way to measure any of the performance claims in this package. The `Stopwatch` loops in examples 05 and 09 are good enough to show an order of magnitude and no more.
- **SharpLab** (sharplab.io) — paste C#, see the IL or the lowered C#. Useful live during a talk to show what the compiler emits for a lambda or a record.

Suggested reading order

1. Reflection in .NET (concepts) — the whole of Part 2 in one page.
2. Writing custom attributes — short, and it anchors the "metadata" idea.
3. Expression trees (C# guide) — read this **before** `Reflection.Emit`; it is the better tool for most problems people reach for `Emit` to solve.
4. Emitting dynamic methods and assemblies — then the `PersistedAssemblyBuilder` reference page, because the conceptual page predates it.

5 Understanding `AssemblyLoadContext`, then assembly unloadability.

6 Native AOT limitations — so you know which of the above you can actually ship.